Planteo outline de "Nunca fue ajedrez" Libro de experiencias de investigación de Dr. Pedro E. Colla

[Portada]

Prefacio

Es un manual anotado,un journal de actividades

Introducción

Motivaciones

Antecedentes históricos

Los inicios teóricos y mecánicos (1912 - 1950)

El Ajedrecista (1912):

Construído por el ingeniero español Leonardo Torres y Quevedo, fue un autómata electromecánico capaz de jugar de forma totalmente automática un final de torre y rey contra el rey de un oponente humano. Se considera el primer juego de ajedrez automatizado de la historia.

Turochamp (1948):

Desarrollado en papel por Alan Turing junto a David Champernowne. Era un algoritmo con un sistema de puntuación heurístico que calculaba dos jugadas en el futuro. Al no existir computadoras capaces de ejecutarlo, Turing tuvo que simular el código a mano (tardando unos 30 minutos por movimiento) en una partida jugada en 1952.

El artículo de Claude Shannon (1950):

El matemático Claude Shannon publicó "Programming a Computer for Playing Chess" [LINK], el texto fundacional que definió las dos estrategias básicas de búsqueda para las computadoras de ajedrez:

- la Tipo A (fuerza bruta, evaluar todas las jugadas posibles)
- la Tipo B (búsqueda selectiva, evaluar solo las mejores jugadas).

Las primeras ejecuciones en computadoras (1951 - 1959)

El programa de Dietrich Prinz (1951):

El primer programa de ajedrez real ejecutado por una computadora digital. Fue escrito para la Ferranti Mark 1 en la Universidad de Mánchester. No jugaba partidas completas, pero era capaz de resolver problemas de **"jaque mate en dos jugadas"**.

El Ajedrez de Los Alamos (1956):

Desarrollado en la computadora MANIAC I por un equipo del Laboratorio Científico de Los Álamos (incluido Stanislaw Ulam). Debido a las limitaciones de memoria, redujeron el tablero a 6x6 (sin alfiles ni peones dobles). Fue el primer programa en jugar una partida completa y derrotar a un contrincante humano (un principiante que recibió ventaja).

El programa de Alex Bernstein (1957):

Creado en una IBM 704, fue el primer programa capaz de jugar una partida completa en un tablero estándar de 8x8. Utilizaba una estrategia Tipo B (selectiva), analizando las 7 mejores jugadas posibles en cada posición.

La era académica y los primeros torneos (1960 - 1977)

El programa Kotok-McCarthy (1962):

Desarrollado en el MIT bajo la tutela de John McCarthy. En 1966 protagonizó el primer match de ajedrez entre computadoras de la historia, enfrentándose vía telégrafo al programa del Instituto de Física Teórica y Experimental (ITEP) de la Unión Soviética. El programa soviético ganó el encuentro 3-1.

Mac Hack Six (1966-1967):

Escrito por Richard Greenblatt en el MIT para la computadora PDP-6. Fue un punto de inflexión absoluto: incorporó amplios conocimientos de aperturas y heurísticas de ajedrez. Se convirtió en el primer programa en competir contra humanos en torneos oficiales, logrando la primera victoria computacional contra un jugador federado y obteniendo un rating ELO de unos 1200 o 1400 puntos.

La serie "Chess" de Northwestern University (1970 - 1977):

Creado por Larry Atkin, David Slate y Keith Gorlen. Sus versiones (Chess 3.0, Chess 4.0, etc.) dominaron por completo los primeros Campeonatos de Ajedrez de Computadoras de la ACM. En 1977, Chess 4.6 ganó el segundo Campeonato Mundial de Ajedrez por Computadora y logró vencer por primera vez a un Gran Maestro (Michael Stean) en una partida de ajedrez rápido.

El Fenómeno Histórico de Microchess (1976)

Microchess es el primer programa de ajedrez comercialmente exitoso para microcomputadoras en la historia de la informática. Diseñado originalmente en 1976 por Peter R. Jennings para el microcomputador hobbista KIM-1 de MOS Technology, demostró al mundo que una máquina extremadamente limitada podía ejecutar una simulación interactiva inteligente, ayudando a cimentar el mercado del software comercial independiente.

Fuentes [1] (https://chessprogramming.org/Peter_Jennings), [2] (https://en.wikipedia.org/wiki/Microchess), [3] (https://es.wikipedia.org/wiki/Microchess), [4] (https://wiki.texto-plano.xyz/microchess), [5] (https://www.microsiervos.com/archivo/ordenadores/historia-microchess-juego-ajedrez-1k-primer-software-comercial-historia.html), [6] (https://www.schach-computer.info/wiki/index.php?title=Jennings,_Peter_En)

Orígenes e Historia

[FOTOGRAFIA DE KIM-1 con MicroChess]

En el año 1976, las computadoras domésticas daban sus primeros pasos. El KIM-1 de MOS Technology no era una computadora convencional, se trataba de una placa de circuito impreso sin caja, provista de un microprocesador MOS 6502 a 1 MHz, un teclado hexadecimal de 24 teclas, un display de 7 segmentos de 6 dígitos y la asombrosa cantidad de 1 Kilobyte (1024 bytes) de memoria RAM.

[1] (https://benlo.com/microchess/), [2] (https://es.wikipedia.org/wiki/Microchess)

Peter Jennings, un recién graduado universitario, se propuso el desafío de meter un juego de ajedrez completo en ese espacio. El 18 de diciembre de 1976 nació Microchess.

[1] (https://www.computerhistory.org/chess/early-microcomputer-chess/), [2] (https://www.schach-computer.info/wiki/index.php?title=Jennings,_Peter_En), [3] (https://es.wikipedia.org/wiki/Microchess), [4] (https://benlo.com/microchess/)

El primer hito comercial

[FOTOGRAFIA DE MICROCHESS]

Inicialmente, Jennings vendía el programa por $10 dólares enviando un manual físico que contenía el código fuente completo en lenguaje ensamblador. Los compradores debían teclear línea por línea los códigos hexadecimales en sus computadoras. Más adelante, cofundó Personal Software y comenzó a distribuirlo en casete, vendiendo decenas de miles de copias (y eventualmente millones con sus posteriores versiones gráficas para Apple II, TRS-80 y Commodore PET). Curiosamente, las regalías generadas por Microchess sirvieron para financiar el desarrollo y lanzamiento de VisiCalc, la primera hoja de cálculo de la historia.

[1] (https://www.schach-computer.info/wiki/index.php?title=Jennings,_Peter_En), [2] (https://es.wikipedia.org/wiki/Microchess), [3] (https://www.microsiervos.com/archivo/ordenadores/historia-microchess-juego-ajedrez-1k-primer-software-comercial-historia.html), [4] (https://www.computerhistory.org/chess/early-microcomputer-chess/), [5] (https://translate.google.com/translate?u=https://en.wikipedia.org/wiki/Microchess&hl=es&sl=en&tl=es&client=sge)

Filosofía de Diseño

La filosofía detrás de Microchess no era vencer a Grandes Maestros, sino lograr lo aparentemente imposible, la compresión absoluta y la optimización radical de un desarrollo de software dentro de recursos inimaginablemente reducidos. En lugar de un árbol de búsqueda masivo, Jennings se enfocó en:

[1] (https://www.schach-computer.info/wiki/index.php?title=Jennings,_Peter_En) [1] (https://translate.google.com/translate?u=https://en.wikipedia.org/wiki/Microchess&hl=es&sl=en&tl=es&client=sge) [1] (https://www.schach-computer.info/wiki/index.php?title=Jennings,_Peter_En), [2] (https://translate.google.com/translate?u=https://en.wikipedia.org/wiki/Microchess&hl=es&sl=en&tl=es&client=sge)

- Accesibilidad y entretenimiento: El creador afirmaba que parte de su éxito comercial radicaba en que el programa cometía errores humanos, lo que hacía divertido jugar contra él y ganarle.
- Demostración técnica: Servir como una "analogía del mundo real" para mostrar a familiares y amigos de qué era capaz un microordenador, rompiendo la percepción de que estas máquinas eran meros juguetes de luces parpadeantes.

- Heurística simplificada: Basarse en conceptos estratégicos del famoso libro *My System* de Aron Nimzowitsch para tomar decisiones sin sobrecargar la memoria. [1] (https://benlo.com/microchess/)

Implementación Técnica

El programa original de Microchess ocupaba exactamente 924 bytes de memoria. En ese espacio diminuto se incluyó el generador de movimientos, la lógica de validación, la función de evaluación, la interfaz de entrada/salida y las variables del juego. [1] (https://www.schach-computer.info/wiki/index.php?title=Jennings,_Peter_En)

- Lenguaje: Código máquina / Ensamblador puro del MOS 6502. [1] (https://es.wikipedia.org/wiki/Microchess)
- Interfaz de Usuario Básica: Sin gráficos. El jugador introducía su movimiento en formato numérico (por ejemplo, anotando las coordenadas de origen y destino por número de casilla) usando las teclas de la placa KIM-1, y el display de 7 segmentos respondía mostrando la jugada de la máquina. [1] (https://www.computerhistory.org/chess/early-microcomputer-chess/), [2] (https://translate.google.com/translate?u=https://en.wikipedia.org/wiki/Microchess&hl=es&sl=en&tl=es&client=sge)
- Estructura de Datos Ultra-Comprimida: En lugar de mapear un tablero clásico de 64 casillas completas (que consumiría demasiada memoria para la época), utilizaba una matriz compacta orientada al conteo y posición de piezas activas de cada bando. [1] (https://chessprogramming.org/MicroChess)

Características Principales

- Modo de Juego Humano vs. CPU: Permitía jugar partidas completas contra el computador.
- Velocidades de ejecución configurables: Las versiones iniciales podían responder al instante, tomar de 5 a 10 segundos o calcular de forma más profunda para una partida de aproximadamente una hora en total.
- Niveles de dificultad: Dependiendo de la versión, ofrecía desde 3 niveles (Microchess 1.5) hasta 8 niveles de juego en versiones posteriores.
- Herramienta de práctica: Era ideal para aprender aperturas estándar y practicar mates sencillos. [1] (https://www.ecured.cu/Microchess), [2] (https://translate.google.com/translate?u=https://en.wikipedia.org/wiki/Microchess&hl=es&sl=en&tl=es&client=sge)

Algoritmo Detallado

Para operar en menos de 1 KB de RAM sin desbordar la pila de llamadas (stack) del procesador 6502, Jennings ideó un algoritmo simplificado basado en variaciones de Minimax con poda Alpha-Beta rudimentaria: [1] (https://fvcalderan.github.io/myworks/articles/minimax_chess_engine.pdf), [2] (https://chessprogramming.org/MicroChess)

En un Árbol de Búsqueda (Búsqueda por Plies), Microchess calculaba usualmente hasta 3 plies de profundidad (3 medios movimientos: Jugada CPU $\rightarrow$ Respuesta Humana $\rightarrow$ Réplica CPU). [1] (https://en.wikipedia.org/wiki/Microchess), [2] (https://translate.google.com/translate?u=https://en.wikipedia.org/wiki/Microchess&hl=es&sl=en&tl=es&client=sge)

- Generación de movimientos: El motor barría las piezas secuencialmente para encontrar movimientos legales.

- Inversión de Tablero (Board Reversing): Para ahorrar código de evaluación duplicado para blancas y negras, el programa "espejaba" o invertía internamente el tablero para que el bando que estuviera calculando siempre fuera evaluado desde la misma perspectiva algorítmica. [1] (https://chessprogramming.org/MicroChess)

Se utilizaba una función de *Evaluación Estática* de manera que cuando el motor alcanzaba el límite de profundidad, ejecutaba una asignación de puntos preinicializada de 16 bytes que tasaba el valor de las piezas en unidades de medio peón:

Pieza	Valor en Microchess (Medios Peones)
Rey	11
Reina	10
Torre	6
Alfil	4
Caballo	4
Peón	2

A este valor material estático se le sumaban sutiles bonificaciones posicionales basadas en la movilidad (cuántas casillas libres tenía una pieza para moverse) y la ocupación del centro del tablero. El movimiento con la puntuación final ponderada más alta (Minimax) era el seleccionado. [1] (https://benlo.com/microchess/)

Evaluación Comparativa de Potencia Ajedrecística

Microchess debe ser juzgado bajo el prisma de su contexto técnico:

Criterio	Microchess (1976)	Sargon (1978) / Sargon II (1979)	Motores Modernos (Ej. Stockfish)
Fuerza Estimada (Elo)	~1100 Elo (Nivel Principiante)	~1500 - 1800 Elo (Nivel Club)	>3500 Elo (Superhumano)
Profundidad de Búsqueda	Hasta 3 plies	4 - 6 plies habitualmente	>30 plies con facilidad
Consumo de Memoria	< 1 KB RAM	~8 KB - 16 KB RAM	Gigabytes de Tablas de Transposición
Reglas Soportadas	Básicas (algunas omisiones)	Ajedrez Completo	Ajedrez Completo e IA

Daría la impresión, desde ésta perspectiva que MicroChess no es mucho mas que una bonita referencia histórica, sin embargo hay un aspecto que no ha podido ser batido, con esa arquitectura moderna era (¿es?) capaz de aportar aproximadamente un punto ELO por bit, Sargón aporta aprox. 0.09 puntos ELO por bit y Stockfish, con todo lo impresionante que es, aporta 0.0000035 puntos ELO por bit. El sistema ELO, del cual nos ocuparemos en detalle mas adelante, es de naturaleza logaritmica, pero aún asi la performance de MicroChess es impresionante. Algunas ideas generales han sido utilizadas posteriormente en éste trabajo.

Desempeño y Limitaciones

A nivel de juego puro, la crítica de la época y los historiadores posteriores (como Tim Harding) llegaron a catalogar el juego de Microchess como "terrible" en comparación con lo que vendría después. El motor cometía despistes graves y tácticos debido a su corta profundidad de cálculo y a que omitía cálculos complejos de finales. [1] (https://translate.google.com/translate?u=https://en.wikipedia.org/wiki/Microchess&hl=es&sl=en&tl=es&client=sge) Sin embargo, su potencia radicaba en la eficiencia. Mientras que competidores posteriores como Sargon requerían máquinas mucho más caras con al menos 8K o 16K de memoria, Microchess demostró que la optimización de código extrema podía suplir la falta de hardware, convirtiéndose en una leyenda irrepetible de la ingeniería informática. [1] (https://www.schach-computer.info/wiki/index.php?title=Jennings,_Peter_En), [2] (https://translate.google.com/translate?u=https://en.wikipedia.org/wiki/Microchess&hl=es&sl=en&tl=es&client=sge)

Sería interesante estudiar en mas detalle un fragmento de su código fuente en ensamblador 6502, ver cómo resolvía problemas específicos como el enroque o la promoción o incluso saber más sobre el duelo histórico entre Bobby Fischer y el Commodore ChessMate basado en este software? Pero dejamos las cuestiones técnicas para mas adelante, cuando las analizaremos con algún detalle.

Sargon: the old buddy (1978/1979)

[FOTOGRAFIA DE SARGON]

Sargón fue el primer motor de juego para ajedrez automático relativamente completo comercialmente accesible, si bien existieron esfuerzos anteriores estaban normalmente implementados en grandes plataformas de cómputos (mainframe, VAX y otras) que tenían escasas opciones de accederlas. **Sargon** fue desarrollado por **Dan y Kathe Spracklen** y presentado en 1978. Ese año ganó las cinco partidas del torneo de ajedrez para microcomputadoras celebrado en la **West Coast Computer Faire**, uno de los primeros hitos competitivos del ajedrez en computadoras personales. Su importancia no radica en haber inventado la búsqueda ajedrecística, sino en demostrar que una microcomputadora de 8 bits, con memoria y potencia muy limitadas, podía ejecutar un jugador competitivo para su época. También contribuyó a popularizar el ajedrez informático entre aficionados y programadores. Sargon fue escrito en **lenguaje ensamblador Z80**. Según la descripción de sus autores, ocupaba aproximadamente **8 KB de RAM**, distribuidos en unos 4 KB de lógica de juego, 2 KB de datos y 2 KB de interfaz gráfica.

Sus principales componentes eran:

- Representación del tablero y generación de movimientos:** rutinas especializadas por pieza, con comprobaciones de legalidad, jaques, enroque y captura al paso.
- **Búsqueda:** exploración en profundidad de un árbol de movimientos mediante **poda alfa-beta**, para descartar ramas que no podían mejorar la decisión.
- **Evaluación:** puntuación heurística de las posiciones, basada en material y consideraciones ajedrecísticas; incluía análisis de intercambios para reducir errores tácticos con un costo computacional contenido.
- **Interfaz:** visualización del tablero e interacción con el usuario, adaptadas a las capacidades de los equipos de la época.

No utilizaba redes neuronales ni aprendizaje automático: sus decisiones procedían de reglas, búsqueda y evaluación diseñadas explícitamente. Sargon ayudó a consolidar el ajedrez como demostración práctica de

búsqueda heurística en hardware restringido. El libro *Sargon: A Computer Chess Program* (1978)[LINK], que incluía su código fuente comentado, facilitó el estudio, la adaptación y la enseñanza de técnicas de programación de motores de ajedrez. La familia Sargon tuvo posteriores versiones y adaptaciones para diversas computadoras personales. Su legado resulta especialmente relevante para proyectos contemporáneos que combinan motores clásicos y evaluadores neuronales: permite observar qué aporta la búsqueda y qué aporta el mecanismo de evaluación, dos componentes conceptualmente separables. En algún sentido es el punto de partida para muchas de las iniciativas que se describen en el resto de éste material y de hecho se constituye en la "referencia" al momento de hacer evaluaciones preliminares de la fuerza ajedrecística de un enfoque.

Fuentes

- Computer History Museum, *Sargon: A Computer Chess Program*.
- Dan y Kathe Spracklen, *Sargon: A Computer Chess Program* (1978), texto y listado histórico.
- Computer History Museum, Dan y Kathe Spracklen en el campeonato ACM de 1978.

El juego

Para abordar el tema de explorar algoritmos y estrategias automáticas hay que, primero, entender las caracteristicas del juego. Su estructura, sus reglas, sus modos, sus convenciones y la infraestructura que rodea la forma de interpretar una partida.

El desafío

El contexto de los motores de ajedrez ha evolucionado significativamente en la medida que aparecen plataformas mas veloces y algoritmos mas novedosos. O meramente algoritmos que eran impensables en una plataforma Z80 de 8 bits y pocos K de memoria RAM son ahora triviales en un procesador de 64 bits con 32 GBytes de memoria y acceso a un procesador GPU de alta potencia, es una cuestión de perspectiva.

Pero en la esencia el problema no es, una vez actualizado, diferente a como lo planteo Claude Shannon, sigue habiendo dos tipos de algoritmos de juego

- la Tipo A (fuerza bruta, evaluar todas las jugadas posibles)
- la Tipo B (búsqueda selectiva, evaluar solo las mejores jugadas).

Y los dos tipos han evolucionado. Los de *Tipo A* se han rendido ante la evidencia de un juego que logra presentar el formidable desafío de 10^{130} combinaciones a explorar, imposible.

Claude Shannon lo formuló como una famosa analogía basada en sus conclusiones elementales: si convirtiéramos cada uno de los átomos del universo observable (aproximadamente 10^{80}) en una supercomputadora —incluso en un computador cuántico operando a la máxima velocidad permitida por las leyes de la física— y las pusiéramos a calcular combinaciones desde el Big Bang hasta hoy, apenas habrían analizado una fracción infinitesimal del total.

Sin embargo el enfoque de fuerza bruta es abordable si se limita el *horizonte* dentro del cual se analizan las jugadas posibles; planteado de ésta forma el problema pasa a ser que tan lejos permite el algoritmo y la tecnología mover ese horizonte hacia el futuro.

MicroChess o **Sargon** hacen eso, aplican un método **minimax alpha/beta* para tratar de reducir el árbol de posibilidades y, en un tiempo objetivo que se considere razonable examnar la mayor cantidad de jugadas posibles y elegir la mejor. Esos motores en sus ambientes de hardware originales no podían hacerlo mas que dos jugadas hacia adelante, hoy en hardware moderno el mismo algoritmo puede analizar hasta 7 u 8 jugadas hacia adelante. Pero alli aparece la siguiente limitación, como evaluamos el posible valor de cada jugada comparada con otras. Aparecen los otros componentes de la estrategia, el valor de la i-esima posición **Valor (V_i)** y la **Política (P_i o π_i), es decir que capacidad tiene una jugada para darme el mayor beneficio (usualmente tratar de ganar la partida) y entre las posibles cual es el criterio para elegir una movida en lugar de otra.

MicroChess o **Sargon** tenían analisis muy simplificados basado en una heurística determinada que asigna valor a las piezas, a la ocupación de ciertos sectores del tablero y a la chance que en la próxima jugada ocurra algo relevante (tal como comer una pieza de mas valor o dar jaque mate). Pero como es de imaginar heurísticas de ese tipo son relativamente limitadas.

Básicamente capturan la **"ideología"** ajedrecistica de quien las formula, sus preferencias si se quiere. Pero ciertamente es dudoso que provengan de un experto jugador a nivel *Maestro* o *Gran Maestro*. Como resultado estas estrategias sorprenden a un jugador casual, que quizás no realiza análisis de esa profundidad o con alguna heurística simple pero robusta, pero rara vez pueden sorprender, mucho menos ganar a un jugador de cierto nivel.

En la medida que la potencia de cálculo aumentó se desarrollan casi concurrentemente algoritmos sofisticados que permiten explorar mas posiciones o seleccionar aquellas posiciones mas prometedoras respecto a otras y al mismo tiempo algoritmos mas complejos para obtener el material para decidir que jugada es mejor que otra (V_i) y que jugada será mejor seleccionar como movida propia π_i.

El epítome de fuerza bruta fue el programa *Deep Blue*, fue una supercomputadora de ajedrez desarrollada por IBM que hizo historia al convertirse en el primer sistema informático en derrotar a un campeón del mundo vigente, Garry Kasparov, en un encuentro con ritmo de juego de torneo. Su desarrollo comenzó a finales de los años 80 bajo el nombre de ChipTest y luego Deep Thought en la Universidad Carnegie Mellon, de la mano de los científicos Feng-hsiung Hsu y Murray Campbell, quienes posteriormente fueron reclutados por IBM para culminar el proyecto.

¿Cómo funcionaba y en qué se basaba?

A diferencia de la Inteligencia Artificial moderna basada en redes neuronales y aprendizaje autónomo (como AlphaZero, al que veremos luego), Deep Blue funcionaba mediante fuerza bruta computacional combinada con reglas heurísticas.

- Poder de procesamiento masivo: Consistía en un superordenador masivamente paralelo compuesto por dos torres de dos metros de altura que albergaban más de 500 procesadores y 216 chips aceleradores específicamente diseñados para calcular movimientos de ajedrez.
- Velocidad de cálculo: La versión de 1997 era capaz de analizar y evaluar hasta 200 millones de posiciones por segundo.
- Algoritmo de búsqueda (Minimax): Calculaba los árboles de decisiones explorando masivamente las opciones de jugadas hacia adelante (profundidad de cálculo) y seleccionaba la mejor opción asumiendo que el rival también elegiría su mejor jugada.
- Función de evaluación humana: Para decidir si una posición final en su cálculo era "buena" o "mala", utilizaba una función programada con conocimientos conceptuales de ajedrez (valor de las piezas,

seguridad del rey, estructura de peones, espacio). El equipo contó con la asesoría de Grandes Maestros de ajedrez para afinar esta base de datos y su libro de aperturas.

En esencia se trata de cantidades masivas de potencia de cálculo para explorar millones de jugadas por segundo (**Sargon** puede, en un día bueno explorar algunos miles a 2 jugadas hacia adelante) pero usando el algoritmo *minimax* y una funcion de (V_i) y π_i dada por una combinación de posiciones recordadas y comparadas de partidas pasadas y preferencias de los entrenadores. El programa nunca se hizo público y solo se dispone de descripciones superficiales de su funcionamiento.

Pero estaba claro que el enfoque de *Deep Blue* ponía el sendero de la fuerza bruta muy lejos de la disponibilidad habitual de experimentadores con menos recursos.

La evolución progresó hasta la disponibilidad moderna de programas como *stockfish* [LINK] que puede mostrar niveles de juego que solo los grandes maestros puede batir, a veces.

Este programa, y algunos similares, pueden funcionar en computadores modernos, los que de por si son bastante mas poderosos que en la época de *Deep Blue* vs. Kasparov, con algoritmos muy especializados para reducir el espacio de exploración y con heurísticas mas sofisticadas para evaluar las movidas. El resultado es que *stockfish* puede evaluar el equivalente a 20 o 30 jugadas hacia adelante en sus versiones mas extremas (las hay menos potentes). Recientemente, luego de la irrupción de algoritmos como *AlphaZero* basado en redes neuronales *stockfish* también abraza el uso de técnicas de inteligencia artificial para mejorar sus capacidades de exploración y valoración de jugadas.

Pero todos los métodos explorados, y algunos mas no mencionados, se caracterizan por ser pertenecientes a la familia de *fuerza bruta*, pero que hay de la otra familia los de *busqueda selectiva*?

Por cierto que tambien hay exponentes en ésta familia, algunos muy famosos.

AlphaZero

[FOTO DE ALPHAZERO-KASPAROV]

AlphaZero es un programa de Inteligencia Artificial desarrollado por Google DeepMind que revolucionó los juegos de mesa de estrategia (ajedrez, shogi y go) al aprender a dominarlos de forma completamente autónoma. A diferencia de los motores tradicionales que dependen del conocimiento acumulado por humanos durante siglos, AlphaZero aprendió a jugar partiendo de cero, jugando millones de partidas contra sí mismo utilizando únicamente las reglas básicas del juego. AlphaZero nació a finales de 2017 en los laboratorios de Google DeepMind, la división de IA de Google. Es la evolución directa de AlphaGo (la IA que derrotó al campeón mundial del milenario juego de Go) y AlphaGo Zero. El equipo de DeepMind, liderado por Demis Hassabis, adaptó este sistema para crear un algoritmo más generalizado y aplicable a múltiples juegos complejos, publicando un histórico artículo científico donde demostraron que, en pocas horas de autoaprendizaje, la IA podía destruir a los campeones mundiales informáticos de ajedrez (Stockfish) y shogi (Elmo). AlphaZero no utiliza la "fuerza bruta" de los ordenadores tradicionales ni funciones de evaluación programadas por humanos. Se basa en dos pilares tecnológicos:

- Redes Neuronales Profundas: Tiene una red que evalúa las posiciones del tablero y otra que predice cuáles son las jugadas más prometedoras. En lugar de calcular millones de variantes absurdas, AlphaZero utiliza una intuición artificial similar a la humana para enfocarse solo en las mejores líneas.
- Aprendizaje por Refuerzo (Tabula Rasa): El programa comenzó sabiendo únicamente las reglas del ajedrez. Jugó contra copias de sí mismo a una velocidad asombrosa. Al principio hacía movimientos

aleatorios, pero la red neuronal se iba actualizando tras cada partida, aprendiendo qué estrategias llevaban a la victoria y cuáles a la derrota. En solo 4 horas de autoentrenamiento en ajedrez, superó el nivel de Stockfish (en ese momento).
- Búsqueda de Árboles de Monte Carlo (MCTS): En vez del algoritmo clásico Minimax que analiza posiciones de forma simétrica a gran profundidad, AlphaZero utiliza *Monte Carlo Tree Search (MCTS)* para simular los resultados más probables, calculando "solo" unas 80,000 posiciones por segundo (frente a los 70 millones de Stockfish), pero con una precisión y calidad estratégica radicalmente superiores.

AlphaZero no es un software comercial ni de código abierto; fue un proyecto de investigación científica interna de Google y no está disponible para que el público general lo descargue o lo use en su computadora. Sin embargo, el mundo del ajedrez no se quedó de brazos cruzados, el proyecto *Leela Chess Zero (Lc0)* resultó del esfuerzo de una comunidad de ajedrez y programadores la que creó este proyecto de código abierto para replicar primero el funcionamiento de AlphaZero y luego extenderlo. *Lc0* se entrena gracias a la potencia comunitaria de miles de usuarios que prestan sus tarjetas gráficas (GPUs). Cualquier aficionado puede descargar Lc0 de forma gratuita, conectarlo a una interfaz (como Arena o ChessBase) y usarlo en su propia PC para analizar partidas con el "estilo AlphaZero". El estilo de AlphaZero cambió para siempre la teoría del ajedrez moderno debido a sus marcadas diferencias con los motores clásicos:

Característica	**Motores Tradicionales (Ej. Stockfish clásico)**	**AlphaZero / Motores de Red Neuronal**
Origen del conocimiento	Heurísticas y conceptos de estrategia programados por Grandes Maestros humanos.	Aprendizaje autónomo absoluto jugando contra sí mismo
Velocidad de cálculo	Brutal. Evalúa decenas o cientos de millones de posiciones por segundo.	Eficiente. Evalúa decenas de miles de posiciones por segundo, pero muy selectas.
Estilo de juego	Ultra-calculador, materialista, defensivo y busca la perfección táctica concreta.	Profundamente posicional, intuitivo, dinámico y famoso por sacrificar piezas a largo plazo por iniciativa.
Hardware ideal	Procesadores comunes (CPU).	Tarjetas gráficas de alta potencia o procesadores de IA (GPU / TPU)

El éxito de AlphaZero fue tan rotundo que obligó a los motores tradicionales a evolucionar, hoy en día, el propio Stockfish ya no es puramente tradicional pues adoptó la tecnología de redes neuronales (llamada *NNUE*) fusionando lo mejor de la fuerza bruta clásica con la enorme comprensión estratégica inspirada por AlphaZero.

Se explorarán algunos de los algoritmos involucrados en la medida que se discutan implementaciones concretas, pero lo cierto es que claramente define una posible estrategia para un recorrido de aprendizaje y comprensión.

Carece de sentido tratar de emular o superar a *stockfish* (o por caso a *AlphaZero*), es un esfuerzo de muchas personas, con enormes recursos materiales, de tiempo y tecnológicos por lo que tratar de conseguir algo distinto es un desafío que requiere calibrar los recursos disponibles.

Stockfish

[FOTO DE STOCKFISH]

Ese enfoque, al no competir con *stockfish*, lleva a preguntarse cual es el diferenciador de éste proyecto.

La respuesta no es lineal ni simple, siquiera quizás robusta.

Se trata de aprender, explorar, comprender y eventualmente aplicar técnicas conocidas para familiarizarse con ellas, y quizás encontrar estrategias que permitan aplicarlas en computadores mas simples y de mucha menos potencia en forma efectiva. microcontroladores, problema de optimización de performance. O quizás solo haber disfrutado el camino de aprendizaje.

Planteo conceptual

El planteo conceptual es determinar que componentes, sub-sistemas o módulos, tendrá la implementación para permitir ayudar en los objetivos y luego definir como se conectarán entre si aplicando que convenciones.

[AQUI VIENE UN DIAGRAMA]

Los objetivos van mutando con el aprendizaje, por lo tanto es mas facil contarlos cuando el proyecto esté terminado que al comenzar. Pero si se puede describir el punto de partida y la idea general.

- Explorar los distintos algoritmos.
- Entender las interacciones entre ellos.
- Experimentar el problema de ajedrez como compuesto de dos partes:
    - Explorar el valor de las jugadas que son factibles.
    - Evaluar que criterios se pueden usar para determinar la mejor.
- Probar que combinación da una performance razonable en ambientes muy pequeños.
- Desarrollar un protocolo de evaluación de los distintos enfoques propuestos.

Uno de los aprendizajes mas tempranos consiste en comprender como es el ciclo de vida de un determinado esquema de juego, el que podría resumirse en algunos pasos que se repiten:

- Se parte de una idea o estrategia para medir V_i y π_i
- Implementarlo.
- Encontrar la forma de calibrarlo o entrenarlo.
- Probar el funcionamiento básico (**dry run**).
- Entender la performance y la capacidad de exploración que tiene.
- Calibrar la potencia relativa de cada algoritmo respecto a los restantes.
- Tomar alguna medida objetiva del potencial "*ajedrecistico*" que tiene.
- Optimizar la performance.
- Mover a las distintas plataformas, repetir la evaluación.
- .... tomar el siguiente

Finalmente,

¿como evaluar si una estrategia es "*suficientemente buena*" para declararla exitosa?

Esta determinación está plagada de intangibles, pero a los efectos de éste proyecto se tomará el criterio de *fortaleza competitiva recíproca*

''' Una combinación de algoritmos de evaluación y juego será mejor que otra cuando en competencia recíproca muestre tener un ELO relativo superior. '''

Y un criterio de *aceptabilidad*

''' Un motor será aceptable a los efectos de la experimentación cuando tenga un ELO superior al de un algoritmo Sargón histórico. '''

Se verá mas adelante los mecanismos de medición de fortaleza ajedrecistica relativa, en particular el sistema ELO.

Pero, para poder progresar, es necesario entender que es el ajedrez primero.

Las reglas del ajedrez

Para poder adentrarse en el desarrollo de un motor es claro que debe asegurarse que los algoritmos (*motores*) utilizados cumplan algunas reglas del ajedrez en cuanto a como conducir las partidas y como realizar el movimiento de las piezas.

El ajedrez es un juego de estrategia para dos jugadores (blancas y negras) cuyo objetivo principal es dar jaque mate al rey adversario. A continuación, tienes un resumen claro y estructurado de sus componentes y reglas fundamentales basadas en el reglamento oficial de la Federación Internacional de Ajedrez (FIDE):

El Tablero (El Casillero)

El tablero de ajedrez es una cuadrícula perfecta de 8x8 casillas, alternadas en colores claros y oscuros, sumando un total de 64 casillas. • Orientación: Se coloca de forma que cada jugador tenga una casilla clara (blanca) en su esquina inferior derecha ("blanca a la derecha"). • Columnas: Las líneas verticales se identifican con letras de la a a la h. • Filas: Las líneas horizontales se identifican con números del 1 al 8.

Cantidad y Tipo de Piezas

Cada jugador comienza la partida con un ejército de 16 piezas (32 piezas en total sobre el tablero).

Pieza	Cantidad por jugador	Valor sugerido (en puntos)
Peón	8	1
Caballo	2	3
Alfil	2	3
Torre	2	5
Dama (Reina)	1	9
Rey	1	Infinito (Invalorable)

El valor sugerido de las piezas es un factor que tiene que tomarse, como mucho, ligeramente. El real valor de una pieza es su fuerza táctica en un contexto de juego determinado. Cada pieza tiene bondades y debilidades estratégicas pero su valor depende del contexto. Por ejemplo, un peon a un paso de *coronar* no vale 1, aunque todavía no vale tanto como una Dama.

Movimiento de las Piezas

Ninguna pieza puede saltar sobre otra, a excepción del caballo. Tampoco pueden ocupar una casilla donde ya haya una pieza de su propio color.

- ♟ Peón: Avanza una casilla hacia adelante por turno. En su primer movimiento de la partida, tiene la opción de avanzar dos casillas. Captura en diagonal, avanzando una casilla hacia la izquierda o derecha. Nunca puede retroceder.
- ♞ Caballo: Se mueve en forma de "L" (dos casillas en una dirección y luego una en perpendicular). Es la única pieza que puede saltar sobre otras piezas.
- ♝ Alfil: Se mueve en línea recta por las diagonales tantas casillas como quiera. Cada jugador tiene un alfil para casillas claras y otro para oscuras.
- ♜ Torre: Se mueve en línea recta por las filas y columnas (horizontal y vertical) tantas casillas como desee.
- ♛ Dama: Es la pieza más poderosa. Puede moverse cualquier número de casillas en cualquier dirección: horizontal, vertical o diagonal.
- ♚ Rey: Se mueve solo una casilla en cualquier dirección. Es la pieza que se debe proteger a toda costa.

Condiciones de Terminación de la Partida

Una partida de ajedrez termina cuando ocurre alguna de las siguientes situaciones:

- Victoria (Ganar la partida)
    - Jaque Mate: El rey de un jugador está bajo ataque directo (en jaque) y no tiene ningún movimiento legal para escapar, cubrirse o capturar a la pieza atacante.
    - Abandono: Un jugador decide rendirse de forma voluntaria al ver que su posición es indefendible.
    - Por tiempo: En partidas con reloj, si a un jugador se le agota el tiempo asignado, pierde automáticamente (siempre que el rival tenga material suficiente para dar mate).
- Tablas (Empate)
    - Ahogado: Es el turno de un jugador cuyo rey no está en jaque, pero no puede realizar ningún movimiento legal con ninguna de sus piezas en el tablero.
    - Insuficiencia de material: Ninguno de los dos bandos tiene piezas suficientes para forzar un jaque mate (por ejemplo: Rey contra Rey, o Rey y Caballo contra Rey).
    - Triple repetición: La misma posición exacta del tablero se repite tres veces durante la partida.
    - Regla de los 50 movimientos: Pasan 50 jugadas consecutivas de cada jugador sin que se haya movido un solo peón ni se haya capturado ninguna pieza.
    - Mutuo acuerdo: Ambos jugadores deciden y acuerdan que la partida es un empate.

Modalidades de juego

Las modalidades en el ajedrez se definen principalmente por el control de tiempo asignado a cada jugador para toda la partida. Cuanto menos tiempo hay, el juego pasa de ser puramente estratégico y profundo a volverse un ejercicio de reflejos, intuición y pura táctica. De acuerdo con el reglamento oficial de la FIDE y las principales plataformas en línea (como Chess.com y Lichess), las modalidades se dividen de la siguiente manera:

⏱️ Modalidades Oficiales de Ritmo Corto/Medio

- Ajedrez Clásico (Pensado): Es el formato tradicional de los torneos de élite. Cada jugador suele tener 90 minutos o más para las primeras 40 jugadas, muchas veces con un "incremento" (segundos extra que se añaden al reloj tras cada movimiento). Las partidas pueden durar entre 3 y 7 horas, priorizando la máxima precisión y profundidad estratégica.
- Ajedrez Rápido (Rapid): Un punto medio ideal. El tiempo asignado es de más de 10 minutos pero menos de 60 minutos por jugador. Es un ritmo muy popular en torneos híbridos porque exige calcular rápido sin perder por completo la calidad de las jugadas.
- Blitz (Relámpago): Una de las modalidades más populares y frenéticas. El tiempo total es de 10 minutos o menos por jugador (el estándar más común es 3 minutos con 2 segundos de incremento, o 5 minutos limpios). Aquí los errores tácticos son frecuentes y el manejo del reloj es vital.

🚀 Modalidades Ultra Rápidas y En Línea

- Bullet (Bala): El tiempo es extremadamente escaso, usualmente menos de 3 minutos para toda la partida (el ritmo rey es 1 minuto por jugador). No hay tiempo para pensar; se juega por puro instinto, memoria muscular y velocidad con el ratón o la pantalla táctil.
- Hyperbullet / Ultrabullet: Modalidades exclusivas de internet donde el tiempo es de 30 o 15 segundos en total por jugador. Ganar por tiempo ("relojear" al rival) es casi más importante que dar jaque mate.

🗓️ Modalidades de Ritmo Extendido o Asincrónico

- Ajedrez por Correspondencia: El extremo opuesto al Bullet. Los jugadores tienen días o semanas para hacer un solo movimiento. Antiguamente se jugaba por correo postal; hoy se realiza mediante servidores web. Permite un análisis profundo, científico y casi perfecto de la posición.

🎭 Variantes Populares del Juego (Cambios de Reglas)

Además del tiempo, existen modalidades donde cambias las reglas o la configuración inicial del tablero:

- Ajedrez 960 (Fischer Random): Creado por el campeón mundial Bobby Fischer. Las reglas de movimiento son las mismas, pero la posición de las piezas en la primera fila se sortea aleatoriamente antes de empezar (manteniendo los peones al frente). Existen 960 combinaciones posibles. Su objetivo es anular la memorización de aperturas teóricas.
- Pasapiezas (Bughouse): Se juega en parejas (dos tableros contiguos). Cuando capturas una pieza del rival, se la pasas a tu compañero, quien en su turno puede elegir entre mover una de sus piezas o "soltar" la pieza capturada en cualquier casilla vacía de su propio tablero.
- Ajedrez a la Ciega (Blindfold): Los jugadores disputan la partida sin ver el tablero ni tocar las piezas. Deben llevar la posición en su mente y cantar sus jugadas utilizando la notación algebraica (ej. "e4", "Cc3").

Fortalezas y debilidades

Desde el punto de vista táctico, cada pieza tiene una "personalidad" única definida por su geometría de movimiento. Entender sus ventajas y limitaciones te permite coordinarlas mejor en el tablero:

- ♟ El Peón

    - Fortalezas: Son el "alma del ajedrez". Sirven como un escudo protector para tu propio rey, son excelentes para reclamar y controlar espacio en el centro, y son ideales para crear cadenas defensivas difíciles de romper. Su mayor fortaleza táctica es la promoción: si un peón llega al final, se transforma en Dama.
    - Debilidades: Son las piezas menos móviles y nunca pueden retroceder. Cada vez que avanzas un peón, dejas casillas débiles ("huecos") detrás de él que jamás podrá volver a defender.

- ♞ El Caballo

    - Fortalezas: Su movimiento en "L" le permite saltar sobre otras piezas, lo que lo convierte en el rey de las posiciones cerradas o bloqueadas. Es letal ejecutando dobletes (atacar dos piezas a la vez sin que ninguna pueda bloquear el ataque) y puede ubicarse en casillas fuertes del campo enemigo donde los peones rivales ya no lo puedan expulsar.
    - Debilidades: Tienen un alcance muy corto y tardan varios turnos en trasladarse de un lado al otro del tablero. En los bordes o esquinas pierden hasta el 75% de su potencia defensiva y de ataque.

- ♝ El Alfil

    - Fortalezas: Tienen un gran alcance a larga distancia y brillan en posiciones abiertas donde no hay peones estorbando. Son excelentes para clavar piezas enemigas y pueden cortar el tablero por la mitad controlando diagonales enteras desde la distancia.
    - Debilidades: Están limitados a la mitad del tablero, ya que un alfil que empieza en casillas claras jamás podrá tocar o defender una casilla oscura. Si la estructura de peones bloquea sus diagonales, se convierte en un "alfil malo", atrapado detrás de sus propias líneas.

- ♜ La Torre

    - Fortalezas: Tienen una enorme potencia en el final de la partida. Su objetivo táctico es dominar columnas abiertas (líneas verticales sin peones) para infiltrarse en la séptima u octava fila enemiga, donde suelen arrasar con los peones rezagados del rival.
    - Debilidades: Al inicio de la partida suelen estar atrapadas y desconectadas en las esquinas. Necesitan que el tablero se vacíe un poco o que los peones se muevan para poder activar su tremendo poder a larga distancia.

- ♛ La Dama

    - Fortalezas: Combina la fuerza de la Torre y el Alfil, lo que la convierte en la fuerza ofensiva más destructiva del juego. Su versatilidad la hace perfecta para crear ataques dobles, dar jaques continuos y castigar instantáneamente cualquier error de cálculo o casilla desprotegida del rival.
    - Debilidades: Es tan valiosa que no puede exponerse temprano en la partida. Si la sacas antes de tiempo, las piezas menores del rival (caballos y alfiles) la atacarán constantemente para ganar turnos de desarrollo mientras tú te ves obligado a moverla para salvarla.

- ♚ El Rey

    - Fortalezas: En la fase inicial y media es una pieza puramente defensiva, pero en el final de la partida se transforma en un atacante activo. Al quedar pocas piezas, el rey debe centralizarse para escoltar a sus propios peones hacia la promoción o frenar el avance de los peones rivales.
    - Debilidades: Su vulnerabilidad es absoluta durante la mayor parte del juego. Al moverse solo una casilla, es fácil de arrinconar o rodear si no cuenta con un enroque seguro o un escudo sólido de

peones.

Notaciones

[TABLERO CON REFERENCIA]

Las notaciones en el ajedrez son los sistemas de escritura utilizados para registrar los movimientos de una partida. Esto permite recrear juegos históricos, estudiar teoría, publicar libros y permitir que las computadoras procesen el juego. A lo largo de la historia han existido varios sistemas, pero hoy en día la Notación Algebraica es el estándar internacional oficial de la FIDE. A continuación, te presento una descripción detallada con ejemplos de los métodos de notación más importantes.

Notación Algebraica (El Estándar Moderno)

Este sistema trata al tablero como un plano cartesiano. Es el único sistema legal en torneos oficiales.

La Geometría del Tablero:

- Columnas (verticales): Se nombran con letras de la a a la h (de izquierda a derecha desde la perspectiva de las blancas).
- Filas (horizontales): Se nombran con números del 1 al 8 (desde la primera fila blanca hasta la octava fila negra).
- Casillas: Cada una tiene una dirección única uniendo su letra y número (ej. e4, c6, h8).

Identificación de las Piezas (en español):

Se utiliza la letra inicial mayúscula de la pieza. Los peones ♟ no tienen letra; si solo se escribe la casilla, se entiende que se movió un peón.

- ♚ R = Rey
- ♛ D = Dama
- ♜ T = Torre
- ♝ A = Alfil
- ♞ C = Caballo

Reglas de Escritura y Símbolos Especiales:

- Movimiento simple: Letra de la pieza + casilla de destino. (Ej: Cf3 significa "Caballo a la casilla f3").
- Movimiento de peón: Solo la casilla de destino. (Ej: e4 significa "Peón a la casilla e4").
- Captura: Se intercala una x entre la pieza y la casilla de destino. (Ej: Axb5 significa "Alfil captura en b5").
- Captura de peón: Se indica la columna de origen del peón, la x, y la casilla de destino. (Ej: exd5 significa "Peón de la columna e captura en d5").
- Jaque: Se añade un signo de más (+) al final de la jugada. (Ej: Dh5+).
- Jaque Mate: Se añade un signo de almohadilla o doble más (# o ++). (Ej: Dh7#).
- Enroque corto (flanco de rey): Se escribe 0-0.
- Enroque largo (flanco de dama): Se escribe 0-0-0.
- Promoción de peón: Se escribe la casilla de destino y la inicial de la pieza elegida. (Ej: e8=D).

⚠️ Resolución de Ambigüedades:

Si dos piezas idénticas (como dos caballos) pueden ir a la misma casilla, se debe especificar de dónde venía el que se movió, añadiendo su columna o fila de origen. • Ejemplo: Si tienes caballos en f3 y d2, y ambos pueden ir a e4: si mueves el de f3, escribes Cfe4. Si estuvieran en la misma columna (ej. d2 y d4) y van a e4, escribes C2e4.

Formato PGN (Portable Game Notation)

Es el estándar digital derivado de la notación algebraica. Es el formato de texto plano que leen páginas como Chess.com, Lichess o programas como ChessBase para guardar bases de datos de partidas. Utiliza la notación algebraica pero siempre con las iniciales de las piezas en inglés (K=King, Q=Queen, R=Rook, B=Bishop, N=Knight) para que sea universal. Además, agrupa los metadatos de la partida entre corchetes antes de las jugadas.

Ejemplo real de un archivo PGN (El famoso "Mate del Pastor"):

text [Event "Partida Amistosa"] [Site "Mar del Plata, ARG"] [Date "2026.10.31"] [White "Jugador Blanco"] [Black "Jugador Negro"] [Result "1-0"]

1. e4 e5 2. Ac4 Cc6 3. Dh5 Cf6 4. Dxf7# 1-0 Usa el código con precaución.

Notación Descriptiva (El Sistema Antiguo)

Fue el sistema dominante en los países de habla hispana e inglesa hasta la década de 1980. Si abres un libro de ajedrez viejo, estará escrito así. Es mucho más complejo porque las casillas cambian de nombre según quién esté jugando.

Reglas principales:

- Las columnas no se llaman de la 'a' a la 'h', sino por el nombre de la pieza que la ocupa al inicio: TD (Torre Dama), CD (Caballo Dama), AD (Alfil Dama), D (Dama), R (Rey), AR (Alfil Rey), CR (Caballo Rey), TR (Torre Rey).
- Las filas se cuentan del 1 al 8 desde el punto de vista del jugador que mueve. Por lo tanto, la fila 4 de las blancas es la fila 5 de las negras.
- Se describe el origen y el destino.

Ejemplo de comparación (Algebraica vs Descriptiva):

Imagina el primer movimiento típico del peón de rey: • En Algebraica: 1. e4 (Fácil, la casilla e4 es fija para ambos). • En Descriptiva: 1. P4R (Significa: "Peón a la cuarta casilla del Rey"). Si las negras responden avanzando su peón equivalentemente, en descriptiva también se escribe 1. ... P4R (porque para el negro, es su propia cuarta fila). En algebraica, ese movimiento negro sería 1. ... e5.

Notación FEN (Forsyth-Edwards Notation)

No registra los movimientos, sino que es un método de notación para describir una posición exacta del tablero en una sola línea de texto. Es una "fotografía" que las computadoras usan para guardar partidas suspendidas o puzzles tácticos.

- Lee el tablero fila por fila, desde la fila 8 hasta la fila 1 (de izquierda a derecha).
- Las piezas blancas van en Mayúsculas (P, N, B, R, Q, K) y las negras en minúsculas (p, n, b, r, q, k).
- Los números representan las casillas vacías consecutivas.

- Las filas se separan con una barra diagonal /.

Ejemplo:

La posición inicial del ajedrez en formato FEN se escribe así: rnbqkbnr/pppppppp/8/8/8/8/PPPPPPPP/RNBQKBNR w KQkq - 0 1 (Donde la w indica que juegan blancas, y KQkq indica que ambos bandos aún pueden enrocar en ambos lados). ¿Te gustaría que hagamos un pequeño ejercicio donde te escriba una jugada en notación algebraica y tú intentes deducir qué pieza se movió y qué ocurrió en el tablero?

Sistema ELO

La utilización del sistema ELO es central para poder realizar comparaciones objetivas sobre la potencia ajedrecística entre dos algoritmos, o entre el mismo algoritmo corriendo en plataformas diferentes donde la velocidad de procesamiento o las restricciones de memoria disponible determinan la profundidad alcanzable del análisis en determinados tiempos.

El Sistema de Puntuación ELO es un método matemático utilizado para calcular la habilidad relativa de los jugadores en juegos de suma cero, principalmente en el ajedrez. Fue inventado por el físico estadounidense de origen húngaro Arpad Elo para reemplazar los imprecisos sistemas de clasificación previos y fue adoptado oficialmente por la Federación Internacional de Ajedrez (FIDE) en 1970. No es un acrónimo; lleva ese nombre en honor a su creador.

Es importante entender que el sistema ELO es de naturaleza logarítmica (o si se lo prefiere, exponencial) de manera que una diferencia relativamente pequeña en los puntajes entre dos jugadores cambia profundamente las probabilidades de éxito cuando compiten.

🎯 ¿Para qué se usa?

- Medir el nivel real: Permite saber con precisión matemática qué tan fuerte es un jugador en comparación con toda la población activa.
- Emparejamiento (Matchmaking): En plataformas digitales como Chess.com o Lichess, se usa para emparejarte automáticamente con rivales de tu misma fuerza.
- Otorgar Títulos: La FIDE exige alcanzar ciertos umbrales de Elo (junto con otros requisitos) para otorgar títulos vitalicios:
  - Maestro FIDE (FM): 2300 puntos.
  - Maestro Internacional (IM): 2400 puntos.
  - Gran Maestro (GM): 2500 puntos.

📊 Cómo calcular la probabilidad de victoria basada en el Elo

Antes de actualizar los puntos de un jugador, el sistema necesita calcular la expectativa de puntuación (probabilidad de victoria) de un Jugador A frente a un Jugador B. Un resultado de 1 representa una victoria, 0.5 un empate y 0 una derrota. La fórmula matemática para hallar la expectativa del Jugador A (E_A) es: $E_A = \frac{1}{1+10^{\frac{R_B - R_A}{400}}}$ Donde: • R_A: Elo actual del Jugador A. • R_B: Elo actual del Jugador B. • El número 400 es un factor de escala estándar: una diferencia de 400 puntos de Elo significa que el jugador superior tiene aproximadamente un 91% de probabilidad de ganar.

📝 Ejemplo de Probabilidad:

Imagina una partida entre Jugador A (Elo 1800) y Jugador B (Elo 1600). La diferencia es de 200 puntos.

1. Aplicamos la fórmula: $E_A = \frac{1}{1+10^{\frac{1600-1800}{400}}} = \frac{1}{1+10^{-0.5}}$
2. $10^{-0.5} \approx 0.3162$
3. $E_A = \frac{1}{1+0.3162} \approx 0.76$ Resultado: El Jugador A tiene una expectativa del 76% (0.76 puntos esperados). Esto significa que si juegan 10 partidas, se espera matemáticamente que el Jugador A sume 7.6 puntos (por ejemplo, 7 victorias, 1 empate y 2 derrotas). Por consiguiente, la expectativa del Jugador B (E_B) es el complemento: $1 - 0.76 = 0.24$ (24%).

🧮 Cómo se calcula el intercambio de Elo tras una partida

Cuando dos jugadores terminan una partida, sus puntuaciones se actualizan según la diferencia entre el resultado real y el resultado esperado. La fórmula es: $R'_A = R_A + K \times (S_A - E_A)$ Donde: • R'_A: El nuevo Elo del Jugador A. • R_A: El Elo anterior del Jugador A. • S_A: El resultado real de la partida (1 si ganó, 0.5 si empató, 0 si perdió). • E_A: La expectativa de puntuación calculada en el paso anterior. • K (Factor K): Es una constante que determina el impacto de la partida. Varía según el jugador para dar estabilidad al sistema: • K = 40: Para nuevos jugadores (primeras 30 partidas) o menores de 18 años. El Elo sube y baja muy rápido. • K = 20: Para la mayoría de los jugadores de club y torneos estándar. • K = 10: Para jugadores de élite que han alcanzado un Elo de 2400 o más (evita fluctuaciones drásticas).

📝 Ejemplo de Cálculo de Intercambio (K = 20):

Siguiendo el caso anterior donde Jugador A (1800, $E_A = 0.76$) juega contra Jugador B (1600, $E_B = 0.24$): • Escenario 1: Gana el Jugador A (el favorito). • Nuevo Elo de A: $1800 + 20 \times (1 - 0.76) = 1800 + 20 \times (0.24) = \mathbf{1804.8}$ (+4.8 puntos). • Nuevo Elo de B: $1600 + 20 \times (0 - 0.24) = 1600 - 4.8 = \mathbf{1595.2}$ (-4.8 puntos). • Nota: Como el favorito ganó, el cambio de puntos es pequeño. • Escenario 2: Sorpresa, gana el Jugador B (el menos fuerte). • Nuevo Elo de A: $1800 + 20 \times (0 - 0.76) = 1800 - 15.2 = \mathbf{1784.8}$ (-15.2 puntos). • Nuevo Elo de B: $1600 + 20 \times (1 - 0.24) = 1600 + 15.2 = \mathbf{1615.2}$ (+15.2 puntos). • Nota: Las sorpresas se premian y castigan con muchos más puntos.

📈 Cómo calcular la diferencia de Elo basada en la proporción de partidas ganadas

Si dos jugadores disputan un bloque o match largo de muchas partidas (lo suficientemente grande para mitigar la varianza), podemos usar la proporción de puntos obtenidos para calcular la diferencia exacta de Elo (ΔR) que existe entre ellos. La fórmula matemática para despejar la diferencia de Elo a partir del porcentaje de puntos ganados o proporción (P) es la inversa de la fórmula logística: $\Delta R = 400 \times \log_{10}\left(\frac{P}{1-P}\right)$ Donde P es la proporción de puntos obtenidos (Puntos ganados / Puntos totales disputados).

📝 Ejemplo Basado en Proporción:

Dos amigos juegan un match de 20 partidas. El Jugador A obtiene 15 puntos (combinando victorias y empates) y el Jugador B obtiene 5 puntos.

1. Calculamos la proporción de A: $P = \frac{15}{20} = 0.75$ (75% de los puntos).
2. Aplicamos la fórmula inversa: $\Delta R = 400 \times \log_{10}\left(\frac{0.75}{1-0.75}\right)$
3. $\Delta R = 400 \times \log_{10}\left(\frac{0.75}{0.25}\right) = 400 \times \log_{10}(3)$
4. Como $\log_{10}(3) \approx 0.4771$:
5. $\Delta R = 400 \times 0.4771 \approx \mathbf{190.8}$ Resultado: Basado puramente en el desempeño de este match, el rendimiento del Jugador A estuvo 191 puntos de Elo por encima del Jugador B. Si el Jugador B tuviera

un Elo estimado de 1500, el Jugador A jugó al nivel de un Elo de 1691.

ELO aplicado a algoritmos

En el ajedrez computarizado y las plataformas en línea, el sistema Elo se aplica de forma idéntica en su matemática básica tanto para humanos como para bots, pero difiere drásticamente en cómo se gestionan sus ecosistemas y la velocidad de sus fluctuaciones. Mientras que el Elo de un jugador humano evoluciona lentamente a lo largo de los años debido a limitaciones físicas, fatiga y un número reducido de partidas, los motores de ajedrez (bots) pueden disputar miles de partidas en cuestión de horas. Esto permite que plataformas de evaluación especializadas, como la CCRL (Computer Chess Rating Lists), utilicen factores de escala extremadamente precisos y calibraciones constantes para medir diferencias milimétricas de rendimiento entre versiones de un mismo programa. El mayor desafío radica en que las poblaciones de humanos y bots rara vez comparten el mismo ecosistema cerrado, lo que genera una inflación o deflación del Elo relativo. Un bot de Chess.com con un Elo asignado de 2000 no necesariamente juega con la misma fuerza que un ser humano con 2000 de Elo en la FIDE, ya que los puntos Elo solo miden la habilidad en relación con los demás integrantes de ese grupo específico. Además, a diferencia de los humanos, los bots de competencia pura no sufren de inconsistencia psicológica: sus funciones de evaluación son deterministas, lo que significa que ante una misma ventaja táctica siempre rendirán igual, obligando a los desarrolladores a introducir algoritmos artificiales de "reducción de fuerza" o "errores programados" cuando se diseñan bots destinados a jugar de forma realista contra aficionados humanos. Comparar el Elo de los mejores humanos con el de las computadoras actuales demuestra que la brecha entre la inteligencia humana y los motores de ajedrez es absoluta e insalvable. Mientras que los humanos medimos el progreso en pequeños márgenes, las máquinas han entrado en una escala de fuerza completamente diferente. [1] (https://www.quora.com/What-kind-of-ELO-rating-would-a-computer-bot-like-Stockfish-need-to-beat-Magnus-Carlson-everytime), [2] (https://vocal.media/education/can-a-human-beat-the-best-chess-computer), [3] (https://mychessplan.com/stockfish-vs-leela-chess-zero-adult-improver-engine-audit-2026/) A continuación, se detalla la comparativa entre los picos humanos y las versiones más recientes de los motores dominantes (con datos actualizados a octubre de 2026):

🧠 El Techo Humano: Magnus Carlsen (https://www.google.com/search?kgmid=%2Fm%2F0291j3&q=Magnus+Carlsen)

- Elo Máximo Histórico (Ritmo Clásico): 2882 puntos (alcanzado por Magnus Carlsen (https://www.google.com/search?kgmid=%2Fm%2F0291j3&q=Magnus+Carlsen) en mayo de 2014 y replicado en agosto de 2019). [1] (https://www.guinnessworldrecords.es/world-records/68289-highest-chess-rating-men)
- Contexto: Este es el Elo oficial de la FIDE más alto jamás registrado en la historia del ajedrez (superando el pico de 2851 de Garry Kasparov). Ningún ser humano ha logrado romper la barrera de los 2900 puntos en ajedrez clásico presencial. [1] (https://en.wikipedia.org/wiki/Magnus_Carlsen), [2] (https://www.facebook.com/100088946902343/posts/did-you-know-that-magnus-carlsen-holds-the-record-for-the-highest-peak-classical/988116074163280/), [3] (https://www.chess.com/forum/view/chess-players/magnus-carlsen-70)
- Nota en línea: Aunque Carlsen ha rozado los 3340 puntos de Elo en plataformas de internet para ritmos ultra rápidos (Blitz), ese es un ecosistema cerrado inflado y no comparable con el Elo de torneo internacional. [1] (https://www.youtube.com/watch?v=4YzYk0shhn0&t=447)

💻 Las Bestias de Silicio: Motores Actuales

Para medir a los motores se utilizan listas de clasificación de computadoras especializadas, siendo la más famosa la CCRL (Computer Chess Rating Lists). A diferencia de los humanos, el Elo de los motores varía según la potencia del procesador (CPU) o la tarjeta gráfica (GPU) donde se ejecuten. [1] (https://computerchess.org.uk/ccrl/4040/rating_list_all.html), [2] (https://ramachess.com/stockfish-chess/elo/), [3] (https://ramachess.com/topchessengines/e/lczero.html), [4] (https://chess.stackexchange.com/questions/42730/why-is-leela-chess-zero-rated-low-on-ccrl-leaderboards)

Stockfish 19 (El Rey Absoluto)

- Elo Estimado: ~3650 - 3690 puntos (en el control de tiempo estándar de torneo 40/15).
- Versión actual: Stockfish 19 (lanzado en septiembre de 2026).
- Particularidad: Utiliza redes neuronales eficientemente actualizables (NNUE) combinadas con un cálculo bruto masivo que analiza millones de jugadas por segundo. En listas de Blitz informáticos con hardware de múltiples núcleos, llega a superar los 3800 puntos de Elo. [1] (https://www.facebook.com/c24live/posts/stockfish-is-simply-a-monster/1562998360427947/), [2] (https://chessnextmove.ai/blog/stockfish-19), [3] (https://www.uscfsales.com/blogs/chess-technology/the-best-chess-engine-in-the-world), [4] (https://ramachess.com/stockfish-chess/elo/)

Leela Chess Zero / Lc0 (La Red Neuronal Pura)

- Elo Estimado: ~3580 - 3670 puntos (corriendo en hardware óptimo con GPU dedicada).
- Particularidad: Inspirada en AlphaZero de DeepMind, Leela aprendió a jugar al ajedrez jugando millones de partidas contra sí misma. Su estilo es famoso por tener una "intuición posicional" más parecida a la humana, aunque respaldada por una fuerza de cálculo perfecta. [1] (https://checkmatex.app/blog/stockfish-vs-leela-vs-komodo-best-chess-engine-2026), [2] (https://chessnextmove.ai/blog/stockfish-18), [3] (https://mychessplan.com/stockfish-vs-leela-chess-zero-adult-improver-engine-audit-2026/), [4] (https://www.rankred.com/chess-engines/)

📊 Comparativa Directa de Datos

Competidor Tipo	Elo Máximo / Actual (Normalizado)	Diferencia Magnus Carlsen
Humano	2882	Base (0)
Leela Chess Zero Motor (Red Neuronal / GPU)	~3620	+738 puntos
Stockfish 19 Motor (Fuerza Bruta + NNUE)	~3650	+768 puntos

1

📉 ¿Qué pasaría en un enfrentamiento real? (Análisis Táctico)

La diferencia de más de 750 puntos de Elo entre Magnus Carlsen (https://www.google.com/search?kgmid=%2Fm%2F0291j3&q=Magnus+Carlsen) y Stockfish 19 se traduce, matemáticamente, en una disparidad abismal utilizando la fórmula de expectativa de Arpad Elo: [1] (https://chessnextmove.ai/blog/stockfish-18)

- Probabilidad de victoria: La probabilidad de que Magnus Carlsen (https://www.google.com/search?kgmid=%2Fm%2F0291j3&q=Magnus+Carlsen) le gane una sola partida a Stockfish 19 es prácticamente del 0%. [1] (https://www.quora.com/What-kind-of-ELO-rating-would-a-computer-bot-like-Stockfish-need-to-beat-Magnus-Carlson-everytime)
- Un Match de 100 partidas: Si Carlsen (https://www.google.com/search?kgmid=%2Fm%2F0291j3&q=Carlsen) jugara un match a 100 partidas contra el Stockfish actual, el resultado más probable sería de 95 a 98 derrotas para el humano y apenas 2 o 4 empates (tablas). Carlsen (https://www.google.com/search?kgmid=%2Fm%2F0291j3&q=Carlsen) no ganaría un solo juego. Las únicas tablas ocurrirían si el motor entra de forma forzada en una línea teórica donde el juego esté matemáticamente simplificado ("muerto") desde la apertura. [1] (https://www.quora.com/What-kind-of-ELO-rating-would-a-computer-bot-like-Stockfish-need-to-beat-Magnus-Carlson-everytime)
- Ventajas insalvables: Las computadoras no cometen errores tácticos de un solo movimiento, tienen acceso a "tablabases" de finales (saben perfectamente cómo ganar o empatar cualquier posición si quedan 7 piezas o menos en el tablero) y no sufren de cansancio, nervios ni presión psicológica. [1] (https://vocal.media/education/can-a-human-beat-the-best-chess-computer) En el ajedrez moderno, la IA ya no compite contra el ser humano; los grandes maestros humanos ahora utilizan a Stockfish y Leela simplemente como "oráculos" o herramientas de entrenamiento para intentar comprender la profundidad de un juego que las máquinas ya dominan por completo. [1] (https://checkmatex.app/blog/stockfish-vs-leela-vs-komodo-best-chess-engine-2026)

Correcciones al método de medición de ELO cibernético

Para normalizar el rendimiento de distintos motores de ajedrez que corren en plataformas de hardware radicalmente diferentes, no podemos basarnos en variables teóricas aisladas (como la velocidad del reloj en GHz), ya que la arquitectura de un chip moderno o el uso de hilos (threads) no escalan de forma lineal. En ciencias de la computación y ajedrez informático, la métrica reina para medir el rendimiento puro es la velocidad de nodos por segundo (NPS). Los "nodos" representan la cantidad de posiciones que el motor analiza y evalúa cada segundo. Dado que la utilización del ELO es central en la estrategia de éste proyecto se utilizará un algoritmo de normalización matemática llamado *Algoritmo de Elo Corregido por Eficiencia de Silicio (ECES)*.

🍼 El Algoritmo: Elo Corregido por Eficiencia de Silicio (ECES)

El objetivo del algoritmo es calcular un Elo Base de Software (Elo_{Base}). Este indicador representa la fuerza táctica intrínseca del código del motor si todos compitieran exactamente sobre la misma máquina de referencia.

Paso 1: Obtener el Factor de Rendimiento Bruto (NPS)

En lugar de multiplicar GHz por RAM por núcleos, dejamos que el motor corra en su máquina objetivo un set estandarizado de posiciones complejas (por ejemplo, el test suite *Colditz* o *Arasan*) durante 3 minutos y extraemos el NPS promedio reportado por el motor mediante el protocolo UCI. • Para motores tradicionales (CPU): El NPS mide posiciones tradicionales evaluadas por segundo. • Para motores de redes neuronales puras (GPU como Lc0): El NPS suele medir "Nodos de Búsqueda" o evaluaciones de la red por segundo.

Paso 2: Calcular el Factor de Penalización/Bonificación por Hardware (F_H)

Definimos una Máquina de Referencia Estándar (por ejemplo: 1 CPU de 4 núcleos a 3.0 GHz, sin GPU, 8 GB RAM). Esta máquina produce un número de nodos de referencia conocido como NPS_{Ref} para un motor base. Calculamos el coeficiente de ventaja de hardware (F_H mediante la siguiente fórmula logarítmica (el logaritmo es crucial porque en ajedrez, duplicar la velocidad del hardware no duplica la fuerza, sino que ofrece retornos decrecientes): $F_H = \log_2 \left(\frac{NPS_{Motor}}{NPS_{Ref}} \right)$ Si un motor corre en una supercomputadora y genera 16 veces más NPS que la máquina de referencia, su

F_H será $\log_2(16) = 4$

Paso 3: Aplicar la Equivalencia de Elo por Duplicación de Velocidad

ΔElo_{NPS}

Históricamente, en el desarrollo de motores como Stockfish, se ha demostrado una regla empírica constante: Duplicar la velocidad de cálculo (nodos por segundo) equivale a un aumento de aproximadamente entre 50 y 70 puntos de Elo reales (utilizaremos el estándar de la industria de 60 puntos de Elo). El factor de corrección por hardware en puntos de Elo C_{Elo} es: $C_{Elo} = F_H \times 60$

Paso 4: Calcular el Elo Neto de Software (Elo_{Base})

Finalmente, tomamos el Elo observado en las partidas del torneo Elo_{Medido} y le restamos la ventaja otorgada por la potencia de su computadora:

$$Elo_{Base} = Elo_{Medido} - C_{Elo}$$

📝 Ejemplo Práctico de Aplicación

Imaginemos que ponemos a competir a dos motores en un torneo en línea y obtenemos sus Elos medidos tras 100 partidas. Queremos saber cuál software es mejor si corrieran en igualdad de condiciones. • Establecemos el NPS_{Ref} de la comunidad: 1,000,000 nodos por segundo (1 MNPS).

Caso Motor A: Stockfish (Corriendo en una Laptop Modesta)

• Hardware: CPU 4 núcleos, 2.5 GHz, 8 GB RAM. • Rendimiento: Genera 2,000,000 de nodos por segundo (2 MNPS). • Elo Medido en el torneo: 3300.

1. $F_H = \log_2(2,000,000/1,000,000) = \log_2(2) = 1$
2. $C_{Elo} = 1 \times 60 = 60$ puntos de ventaja por hardware.
3. $Elo_{Base} = 3300 - 60 = \mathbf{3240}$

Caso Motor B: Leela Chess Zero (Corriendo en una PC Gamer de Gama Alta)

• Hardware: CPU 16 núcleos, 5.0 GHz, 64 GB RAM, Tarjeta Gráfica dedicada (GPU RTX 4090). • Rendimiento: Gracias a su potente GPU, su equivalencia de procesamiento alcanza los 16,000,000 de nodos por segundo (16 MNPS). • Elo Medido en el torneo: 3450 (Ganó el torneo).

1. $F_H = \log_2(16,000,000/1,000,000) = \log_2(16) = 4$
2. $C_{Elo} = 4 \times 60 = 240$ puntos de ventaja por hardware.
3. $Elo_{Base} = 3450 - 240 = \mathbf{3210}$

🏆 Conclusión del Algoritmo

Aunque el Motor B ganó el torneo en la práctica con un Elo de 3450 frente a los 3300 del Motor A, el algoritmo ECES demuestra que en nivel de igualdad de condiciones (Elo Base), el código del Motor A es superior (3240 vs 3210). Si mudáramos el Motor A a la computadora del Motor B, el Motor A alcanzaría un Elo real estimado de 3240 + 240 = 3480, aplastando al Motor B. ¿Te gustaría que adaptemos este algoritmo para incluir una penalización por la gestión de la Memoria RAM (Tablas de Hash), las cuales reducen drásticamente el NPS necesario en los finales de partida?

Para integrar la gestión de la memoria RAM en el algoritmo, debemos entender cómo la utiliza un motor de ajedrez. Los motores no usan la RAM de forma pasiva; la destinan principalmente a las Tablas de Hash (Transposition Tables). Estas tablas actúan como una memoria caché de cortísimo plazo: si el motor ya calculó una posición en una línea profunda, guarda el resultado en la RAM. Si vuelve a encontrarse con esa misma posición por otro orden de jugadas (transposición), no la vuelve a calcular; lee la respuesta directamente de la memoria. Tener mucha RAM ahorra millones de cálculos, lo que significa que un motor con una tabla de hash gigante tiene una ventaja invisible que el indicador de Nodos por Segundo (NPS) no siempre llega a reflejar en su totalidad. A continuación, actualizamos el algoritmo para transformarlo en el Algoritmo ECES-H (Elo Corregido por Eficiencia de Silicio y Hash).

🧮 El Algoritmo Actualizado: ECES-H

Añadiremos un nuevo factor independiente de penalización por el tamaño de las tablas de Hash asignadas en la memoria RAM.

Paso 1: Definir los Parámetros de la RAM de Referencia

Establecemos un tamaño estándar de Tablas de Hash en RAM para la comunidad informática como base neutra: • $Hash_{Ref} = 1024\ \text{MB}$ (1 GB asignado exclusivamente al motor).

Paso 2: Calcular el Coeficiente de Ventaja por Hash F_{Hash}

De acuerdo con las pruebas empíricas acumuladas en torneos informáticos como el TCEC (Top Chess Engine Championship), duplicar de forma constante el tamaño del Hash ofrece un retorno decreciente que equivale a un aumento de entre 8 y 12 puntos de Elo (fijaremos el estándar de la industria en 10 puntos de Elo por cada duplicación). Calculamos el factor logarítmico de ventaja por memoria:

$$F_{Hash} = \log_2 \left(\frac{Hash_{Motor}}{Hash_{Ref}} \right)$$

El factor de corrección por RAM en puntos de Elo (C_{RAM}) es:

$$C_{RAM} = F_{Hash} \times 10$$

Paso 3: Combinar los Factores en la Fórmula Final de Elo Base

Ahora, el cálculo del Elo Neto de Software Elo_{Base} restará tanto la potencia bruta de procesamiento C_{Elo} calculada previamente, como la ventaja táctica de memoria caché C_{RAM}:

$$Elo_{Base} = Elo_{Medido} - C_{Elo} - C_{RAM}$$

Donde (recordando el paso anterior): $C_{Elo} = \log_2 \left(\frac{NPS_{Motor}}{NPS_{Ref}} \right) \times 60$

📝 Ejemplo Práctico con Corrección de RAM

Ponemos a competir a dos versiones del mismo motor en dos servidores diferentes: • Máquina de Referencia Común: $NPS_{Ref} = 1\ \text{MNPS}$ | $Hash_{Ref} = 1024\ \text{MB}$ (1 GB).

Caso Motor A: (Poco Procesamiento, pero mucha RAM)

• Rendimiento Bruto: Genera 2,000,000 de nodos por segundo (2 MNPS). • Memoria Asignada: 16,384 MB (16 GB de Hash). • Elo Medido en el torneo: 3200.

1. Corrección por CPU C_{Elo}: • $F_H = \log_2(2/1) = 1$ • $C_{Elo} = 1 \times 60 = \mathbf{60}$
2. Corrección por RAM C_{RAM}: • $F_{Hash} = \log_2(16384/1024) = \log_2(16) = 4$ (Ha duplicado 4 veces la RAM de referencia) • $C_{RAM} = 4 \times 10 = \mathbf{40}$
3. Resultado Elo Base: • $Elo_{Base} = 3200 - 60 - 40 = \mathbf{3100}$

Caso Motor B: (Mucho Procesamiento, pero "ahogado" en RAM)

• Rendimiento Bruto: Genera 8,000,000 de nodos por segundo (8 MNPS). • Memoria Asignada: Solo 1,024 MB (1 GB de Hash, igual al de referencia). • Elo Medido en el torneo: 3210.

1. Corrección por CPU C_{Elo}: • $F_H = \log_2(8/1) = 3$ (Ha duplicado 3 veces los nodos por segundo) • $C_{Elo} = 3 \times 60 = \mathbf{180}$
2. Corrección por RAM (C_{RAM}): • $F_{Hash} = \log_2(1024/1024) = \log_2(1) = 0$ • $C_{RAM} = 0 \times 10 = \mathbf{0}$
3. Resultado Elo Base: • $Elo_{Base} = 3210 - 180 - 0 = \mathbf{3030}$

🏆 Conclusión del Análisis de Eficiencia

Al introducir la RAM, vemos un comportamiento sumamente realista: el Motor B tiene mucha potencia bruta de CPU, pero su baja memoria RAM le impidió sacarle el máximo provecho, perdiendo eficiencia. Aun así, al limpiar el ruido del hardware, el Motor A demuestra tener un código más eficiente 3100 vs 3030. Si al Motor A le diéramos la misma velocidad de procesamiento que al B, su ventaja en las Tablas de Hash lo haría destrozar por completo la competencia. Este modelo ya cubre CPU, GPU (mediante el mapeo de nodos equivalentes), hilos y RAM. ¿Te gustaría que incluyéramos la última frontera de optimización de hardware: el uso de las Syzygy Tablebases (discos de estado sólido SSD / NVMe que almacenan finales perfectos resueltos)?

Análisis de una partida

🔎 ¿Qué significa analizar una partida?

En ajedrez, analizar una partida significa desarmar el juego jugada por jugada después de haberlo terminado. Consiste en una autopsia crítica para descubrir la verdad oculta de la posición: qué se hizo bien, qué se hizo mal, qué oportunidades se dejaron pasar y cuáles eran los planes correctos que ninguno de los dos jugadores logró ver mientras corría el reloj. Antiguamente, el análisis se hacía de forma puramente humana (moviendo las piezas sobre el tablero junto al rival o un entrenador). Hoy en día es un proceso híbrido: el jugador intenta comprender sus propios pensamientos y luego contrasta sus conclusiones con el veredicto infalible de un motor de ajedrez (como Stockfish).

📉 Ejemplos prácticos de un análisis

Durante un análisis, las jugadas se evalúan y se clasifican mediante símbolos universales. Aquí tienes cómo se vería un fragmento de análisis:

- Identificar el Error Grave (Blunder - ??):
    - Ejemplo: "En la jugada 14, moví 14. Tfe1??. No me di cuenta de que al quitar la torre de f1, desprotegí el peón de f2. El motor me muestra que mi rival tenía la jugada táctica 14... Cg4!, atacando f2 y ganando la partida de inmediato."
- Descubrir la Jugada Brillante (Brilliant - !!):
    - Ejemplo: "Mi rival sacrificó su dama con 22... Dxf3!!. En el momento pensé que se había equivocado, pero analizando la línea completa descubrí que tras mi captura 23. gxf3, sus alfiles coordinaban un jaque mate inevitable en tres movimientos. Fue un sacrificio profundamente calculado."
- Encontrar "La Jugada Profiláctica" (Prevención):
    - Ejemplo: "Antes de lanzar mi ataque en el flanco de dama, debí jugar 18. h3!. El análisis me demuestra que esa jugada silenciosa evitaba que el caballo enemigo saltara a g4 y arruinara mis planes ofensivos."

🏆 ¿Qué se obtiene al analizar?

Analizar tus propias partidas es el método número uno en el mundo para mejorar tu nivel de ajedrez. De este proceso obtienes:

- Detección de patrones de error: Descubres si tiendes a perder por errores tácticos (colgadas de piezas), por una mala gestión del tiempo o por no saber cómo jugar los finales de partida.
- Ampliación del repertorio: Entiendes en qué jugada exacta te desviaste de la teoría de aperturas y qué debiste haber respondido ante la sorpresa del rival.
- Pensamiento autocrítico: Rompes la ilusión de que "jugaste perfecto y perdiste por mala suerte". El análisis destruye el ego y te muestra la realidad matemática de tus decisiones.

🏷️ ¿Por qué determinadas partidas o jugadas tienen nombres?

En el ajedrez, los nombres no se asignan de forma automática; nacen de la historia, la geografía y el impacto cultural. Existen tres categorías principales de nombres:

Aperturas y Defensas (Los primeros movimientos)

Tienen nombre para que los jugadores no tengan que memorizar secuencias numéricas largas, sino conceptos estratégicos.

- Por origen geográfico: Se nombran según el lugar donde se jugaron por primera vez o donde se popularizó su estudio.
    - Ejemplo: La Apertura Española (1. e4 e5 2. Cf3 Cc6 3. Ab5) o la Defensa Siciliana (1. e4 c5).
- Por el apellido del jugador: En honor al ajedrecista que la descubrió o la llevó a la élite.
    - Ejemplo: La Defensa Alekhine (1. e4 Cf6), nombrada por el campeón mundial Alexander Alekhine.

Partidas Históricas (Bautizos por su belleza o contexto)

Cuando una partida alcanza un nivel de genialidad artística o drama político inigualable, la comunidad internacional le otorga un nombre propio.

- La Partida Inmortal (1851): También conocida como *evergreen* donde Adolf Anderssen sacrificó casi todas sus piezas mayores (sus dos torres y la dama) para dar un jaque mate espectacular con sus tres piezas menores restantes. Se le llamó así porque su belleza vivirá para siempre.

- La Partida del Siglo (1956): Un joven Bobby Fischer de solo 13 años derrotó al maestro Donald Byrne realizando un sacrificio de dama alucinante que revolucionó el ajedrez mundial.

Patrones Tácticos y de Mate

Ciertas configuraciones de piezas que se repiten constantemente reciben nombres visuales o históricos para facilitar su reconocimiento rápido durante el juego.

- El Mate del Pastor: El clásico mate en 4 jugadas que aprovecha la debilidad del peón en f7.
- El Mate del Pasillo: Ocurre cuando un rey queda atrapado detrás de su propia barrera de peones y una torre o dama enemiga lo ataca en la octava fila.
- La Clavada / El Tenedor: Nombres descriptivos de recursos tácticos (un "tenedor" es cuando un caballo ataca a dos piezas importantes a la vez, como si las pinchara juntas).

Métodos de aprendizaje

Aprender ajedrez de forma eficiente es un viaje que va desde el control muscular básico (mover las piezas sin tirarlas) hasta el desarrollo de una mente estratégica capaz de predecir el futuro. Un humano no aprende acumulando datos, sino automatizando patrones. Es útil entender la hoja de ruta descriptiva de los pasos que hay que dar para lograrlo. El desarrollo de métodos automatizados tiene similitudes notables con el mismo proceso hecho por humanos, quizás con otros tiempos. La hoja de ruta puede ser dividida en 4 etapas lógicas para dominar el juego, diseñada para transformar a un principiante absoluto en un jugador competitivo de club.

🗺️ Hoja de Ruta para Aprender Ajedrez

[Etapa 1: Alfabetización] ➔ [Etapa 2: El Despertar Táctico] ➔ [Etapa 3: Pensamiento Estratégico] ➔ [Etapa 4: Competencia]

Etapa 1: La Alfabetización Visual y Reglas (Elo 0 - 600)

El objetivo en esta fase es acostumbrar al ojo a la geometría del tablero y no cometer ilegalidades.

- Paso 1: Mecánica del tablero. Aprender las reglas básicas, los movimientos y los movimientos especiales (enroque, captura al paso y promoción).
- Paso 2: Fluidez de la Notación. Aprender a leer y escribir en notación algebraica de forma fluida. Si no puedes "hablar" el idioma del ajedrez, no podrás leer libros ni entender análisis.
- Paso 3: Evitar la "ceguera de piezas". Jugar partidas amistosas con un único objetivo táctico: no regalar piezas en una sola jugada (comúnmente llamado colgarse una pieza). Antes de mover, el cerebro debe hacerse la "pregunta de seguridad": ¿A qué casillas ataca la pieza que voy a mover y qué casillas deja de defender?

Etapa 2: El Despertar Táctico (Elo 600 - 1200)

En el ajedrez amateur, el 99% de las partidas se deciden por táctica (errores de cálculo inmediatos). Aquí aprendes a usar la fuerza bruta calculada.

- Paso 1: Patrones de Mate Esenciales. Estudiar los mates básicos que terminan la partida rápidamente (Mate del pasillo, mate del beso de la dama, mate con dos torres, rey y torre contra rey).

- Paso 2: Resolución masiva de "Puzzles" (Problemas). Dedicar el 70% del tiempo de estudio a resolver problemas de táctica de 1 o 2 jugadas. El cerebro humano aprende esto por reconocimiento de formas. Debes automatizar los conceptos de:
    - El Tenedor / Ataque doble: Una pieza ataca dos objetivos.
    - La Clavada: Inmovilizar una pieza enemiga porque detrás hay una más valiosa.
    - La Enfilada: Un ataque en línea recta a una pieza valiosa que, al moverse, expone a la que está detrás.
- Paso 3: Construir un "Repertorio de Aperturas" minimalista. No memorizar líneas profundas. Aprender los Principios de la Apertura: controlar el centro con los peones, desarrollar los caballos y alfiles rápidamente, y poner el rey a salvo mediante el enroque. Elegir una sola apertura con Blancas (ej. Apertura Italiana) y una defensa sólida con Negras (ej. Defensa Karo-Kann o e5).

Etapa 3: Pensamiento Estratégico y Posicional (Elo 1200 - 1600)

Aquí el jugador descubre que no siempre hay un golpe táctico disponible. Se aprende a formular planes a largo plazo cuando el tablero está en calma.

- Paso 1: Estructura de peones y casillas. Aprender que los peones dictan los planes. Identificar peones doblados, peones aislados y peones pasados. Aprender a identificar "casillas fuertes" (debilidades en el campo rival donde puedes meter un caballo de forma permanente).
- Paso 2: El Método de Análisis Post-Partida. Aplicar obligatoriamente la regla de oro: revisar cada partida jugada. El humano progresa cuando encuentra el momento exacto donde su plan estratégico falló y busca la alternativa correcta utilizando el motor como asistente.
- Paso 3: Introducción a los Finales. Estudiar la teoría de finales básicos. El ajedrez se aprende al revés: es más fácil entender el poder de tres piezas en un tablero vacío que el de 32 piezas al inicio. Dominar los finales de peones (regla del cuadrado, la oposición del rey) y finales básicos de torres.

Etapa 4: Especialización y Competición (Elo 1600+)

El juego se vuelve un deporte psicológico y de resistencia. El entrenamiento se asemeja al de un atleta.

- Paso 1: Gestión del tiempo y psicología. Aprender a jugar bajo la presión del reloj, gestionar los nervios tras un error y aprender a "recular" o defender posiciones inferiores (resiliencia en el tablero).
- Paso 2: Estudio de Clásicos. Analizar partidas de los Campeones Mundiales (Capablanca para la sencillez posicional, Alekhine o Kasparov para el ataque dinámico, Carlsen para los finales). El cerebro absorbe la "intuición" de los maestros.
- Paso 3: Transición al Ajedrez Presencial (Over the Board). Jugar torneos oficiales con relojes físicos y piezas reales. El salto del tablero bidimensional de la pantalla al tablero tridimensional real requiere una adaptación visual y espacial crucial.

Los algoritmos

Se abordó previamente en forma superficial que para que un algoritmo pueda jugar al ajedrez, dejando de lado cuestiones de conectividad e infraestructura, es necesario que en una determinada posición de piezas pueda evaluar el valor (V_i) de una acción en comparación a otra para luego decidir la jugada en función de

una política π_i que trate de maximizar las posibilidades propias de ganar, o lo que es lo mismo, las posibilidades del oponente de perder.

Algoritmo Minimax con poda Alfa-Beta: fundamentos matemáticos y aplicación al ajedrez

1. Introducción

El algoritmo Minimax con poda Alfa-Beta (Alpha-Beta Pruning) es uno de los fundamentos clásicos de la inteligencia artificial aplicada a juegos de estrategia de dos jugadores, particularmente el ajedrez.

Su objetivo consiste en determinar la mejor decisión posible para un jugador, suponiendo que su adversario también tomará decisiones óptimas. Para ello, construye y explora un árbol de posibles jugadas y respuestas, evaluando las consecuencias de cada alternativa.

El algoritmo combina dos conceptos:

- Minimax: establece cómo seleccionar una jugada considerando que ambos jugadores intentan optimizar resultados opuestos.
- Poda Alfa-Beta: permite descartar partes del árbol de búsqueda que matemáticamente no pueden modificar la decisión final.

La característica fundamental de Alfa-Beta es que produce el mismo resultado que Minimax completo, para la misma profundidad, función de evaluación y orden legal de jugadas, siempre que la búsqueda se implemente correctamente y no se introduzcan aproximaciones adicionales.

La ventaja consiste en reducir considerablemente la cantidad de posiciones que es necesario analizar, permitiendo alcanzar mayores profundidades con los mismos recursos computacionales.

2. Representación del ajedrez como problema de búsqueda

2.1. Espacio de estados

Un juego de ajedrez puede modelarse mediante un conjunto de estados:

\[S=\{s_0,s_1,s_2,\ldots\} \]

Cada estado \(s\) contiene toda la información necesaria para representar una posición y determinar las jugadas legales, incluyendo:

- Distribución de las piezas sobre el tablero.
- Jugador al que corresponde mover.
- Derechos de enroque.
- Posibilidad de captura al paso.
- Contador de movimientos para la regla de los cincuenta movimientos.
- Información histórica necesaria para detectar repeticiones de posiciones.

Una posición de ajedrez no queda necesariamente determinada por la ubicación de las piezas: dos tableros visualmente idénticos pueden permitir distintas jugadas legales o tener diferentes condiciones de tablas.

Definimos:

$$ A(s)=\{a_1,a_2,\ldots,a_n\} $$

como el conjunto de jugadas legales disponibles en el estado s.

Una función de transición:

$$ T:S\times A\rightarrow S $$

representa el resultado de ejecutar una jugada:

$$ s'=T(s,a) $$

Por ejemplo, desde la posición inicial, una de las transiciones posibles es:

$$ s_1=T(s_0,\text{e2e4}) $$

La posición resultante tendrá un peón blanco en e4, corresponderá mover a las negras y se actualizarán los demás componentes del estado.

2.2. Naturaleza adversarial del problema

El ajedrez es un juego:

1. De dos jugadores.
2. De información perfecta: ambos conocen la posición completa.
3. Determinista: una jugada legal tiene un resultado definido.
4. De suma cero, si se modelan los resultados como ganancia de un jugador y pérdida equivalente del otro.

Podemos definir una utilidad terminal desde la perspectiva de las blancas:

$$ U(s)= \begin{cases} +1 & \text{si ganan las blancas}\\ 0 & \text{si la partida termina en tablas}\\ -1 & \text{si ganan las negras} \end{cases} $$

Por lo tanto, si $U_B(s)$ representa la utilidad para las blancas y $U_N(s)$ la de las negras:

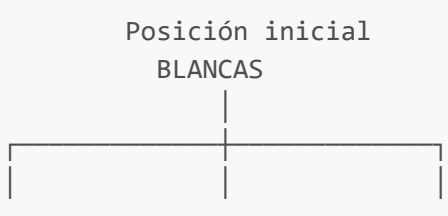
$$ U_N(s)=-U_B(s) $$

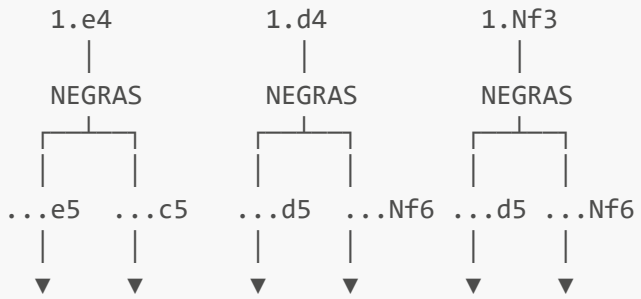
Esto permite representar las decisiones de ambos jugadores mediante una única función de valor.

2.3. El árbol de juego

Desde cada posición se generan sus posibles movimientos, formando un árbol de búsqueda.

Ejemplo simplificado desde la posición inicial





Cada nivel del árbol corresponde a una jugada individual, denominada ply.

Una profundidad de búsqueda de \(d=6\) significa que se examinan seis medias jugadas desde la posición inicial de análisis: tres movimientos de las blancas y tres de las negras, si las jugadas alternan normalmente.

2.4. Complejidad del espacio de búsqueda

Si una posición tiene, en promedio, \(b\) movimientos legales y analizamos una profundidad \(d\), el número aproximado de posiciones terminales del árbol es:

\[N(d)\approx b^d \]

Para un factor de ramificación ilustrativo:

\[b=35 \]

obtenemos:

Profundidad (ply)	Posiciones hoja aproximadas
1	35
2	1.225
3	42.875
4	1.500.625
5	52.521.875
6	1.838.265.625
8	$2{,}25 \times 10^{12}$

Crecimiento exponencial de las posiciones hoja

Modelo teórico con factor de ramificación constante b = 35. Eje vertical en millones de posiciones.

050010001500200012345 6

Este crecimiento exponencial constituye el principal problema computacional de Minimax.

La cantidad exacta de posiciones varía según la posición y las jugadas legales disponibles. Además, en un motor real pueden existir transposiciones, posiciones terminales anticipadas y otras condiciones que hacen

que el árbol no sea uniforme.

3. Fundamento matemático de Minimax

3.1. Jugadores MAX y MIN

El algoritmo utiliza dos tipos de nodos:

MAX: corresponde al jugador que intenta maximizar la utilidad.

MIN: corresponde al adversario que intenta minimizarla.

Si definimos que MAX representa las blancas y MIN las negras, una puntuación positiva representa una posición favorable para las blancas.

Para un estado terminal:

$$ V(s)=U(s) $$

Para un estado no terminal en el que mueve MAX:

$$ \boxed{ V(s)=\max_{a\in A(s)} V(T(s,a)) } $$

Para un estado no terminal en el que mueve MIN:

$$ \boxed{ V(s)=\min_{a\in A(s)} V(T(s,a)) } $$

Estas ecuaciones constituyen el núcleo del algoritmo.

La decisión óptima para MAX se define mediante:

$$ \boxed{ a^*=\arg\max_{a\in A(s)}V(T(s,a)) } $$

Si corresponde jugar a MIN, se utiliza `argmin` en lugar de `argmax`.

La hipótesis esencial es que el adversario no coopera con nuestro plan, sino que selecciona la respuesta más perjudicial para nuestros intereses.

3.2. Ejemplo matemático elemental

Consideremos un árbol con tres jugadas posibles para MAX: A, B y C.

Cada jugada conduce a una posición donde MIN dispone de dos respuestas. Los números representan las evaluaciones obtenidas luego de esas respuestas.

Árbol Minimax de profundidad 2. MAX elige la alternativa C, porque garantiza la mayor evaluación frente a las mejores respuestas de MIN.

Para la alternativa A:

$$ V(A)=\min(3,5)=3 $$

Para la alternativa B:

$$ V(B)=\min(2,9)=2 $$

Para la alternativa C:

$$ V(C)=\min(4,6)=4 $$

En la raíz, MAX selecciona:

$$ V(s)=\max(3,2,4)=4 $$

Por lo tanto:

$$ \boxed{a^*=C} $$

Aunque B ofrece la posibilidad de obtener una puntuación de 9, MAX no puede suponer que su adversario elegirá voluntariamente esa respuesta. MIN escogerá la respuesta que limita el resultado a 2.

Esta es una distinción fundamental entre la evaluación optimista de una jugada y su evaluación Minimax.

4. Minimax con profundidad limitada

En ajedrez real, el árbol completo de la partida resulta demasiado grande para explorarlo exhaustivamente.

Por lo tanto, los motores utilizan una profundidad máxima de búsqueda:

$$ d_{\max} $$

Al alcanzar esa profundidad, en lugar de conocer el resultado definitivo de la partida, se aplica una función heurística de evaluación:

$$ E(s) $$

De este modo:

$$ V(s,d)= \begin{cases} U(s) & \text{si }s\text{ es terminal}\\ E(s) & \text{si }d=0\\ \max\limits_{a\in A(s)}V(T(s,a),d-1) & \text{si mueve MAX}\\ \min\limits_{a\in A(s)}V(T(s,a),d-1) & \text{si mueve MIN} \end{cases} $$

Esto convierte Minimax en una búsqueda heurística de horizonte finito.

La conclusión importante es que el algoritmo ya no determina necesariamente la mejor jugada de toda la partida, sino la mejor jugada según la función de evaluación y la profundidad de búsqueda elegidas.

4.1. Función de evaluación en ajedrez

Una evaluación clásica puede construirse mediante una combinación ponderada de características de la posición:

$$ \boxed{ E(s)=w_m M(s)+w_p P(s)+w_k K(s)+w_c C(s)+w_t T(s) } $$

Donde:

- $M(s)$: diferencia de material.
- $P(s)$: estructura de peones.
- $K(s)$: seguridad de los reyes.
- $C(s)$: control del centro y actividad de las piezas.

- $T(s)$: factores tácticos o posicionales adicionales.

Los coeficientes w_i determinan la importancia relativa de cada característica y deben elegirse de acuerdo con la escala de cada término.

Una aproximación tradicional al valor material, expresada en centipeones, es:

Pieza	Valor aproximado
Peón	100
Caballo	320
Alfil	330
Torre	500
Dama	900
Rey	No tiene valor material intercambiable

Por ejemplo, si las blancas disponen de una torre y las negras de un alfil como únicas piezas no reales, aparte de los peones, la diferencia de material se calcula sumando los valores de todas las piezas de ambos bandos.

En general:

$$ M(s)=\sum_{p\in B}v(p)-\sum_{p\in N}v(p) $$

La evaluación material aislada no contempla situaciones como clavadas, ataques descubiertos, amenazas de mate o piezas atrapadas, por lo que necesita complementarse con otros criterios.

4.2. Funciones de evaluación neuronales

El algoritmo Minimax no exige una función de evaluación diseñada manualmente.

Es posible reemplazarla mediante una red neuronal:

$$ E_\theta(s)=f_\theta(\phi(s)) $$

Donde:

- $\phi(s)$ representa la codificación de la posición.
- θ contiene los parámetros entrenados.
- f_θ produce una valoración de la posición.

En motores que utilizan redes del tipo NNUE (Efficiently Updatable Neural Network), la red suele actuar como evaluador de las hojas del árbol, frecuentemente con mecanismos de actualización incremental de características.

Por lo tanto, búsqueda y evaluación cumplen funciones complementarias:

$$ \text{Minimax / Alfa-Beta} \longrightarrow \text{exploración de consecuencias} $$

$$ \text{Evaluador clásico o NNUE} \longrightarrow \text{estimación del valor de posiciones} $$

Una red neuronal más precisa puede mejorar la decisión sin cambiar la estructura fundamental del algoritmo de búsqueda.

5. La poda Alfa-Beta

5.1. Motivación

Minimax examina todas las alternativas hasta la profundidad establecida, incluso cuando algunas de ellas ya no pueden influir en la selección final.

La observación fundamental de Alfa-Beta es que no siempre necesitamos conocer el valor exacto de una rama.

En determinadas circunstancias basta con saber que su resultado será necesariamente peor que otra alternativa que ya conocemos.

Supongamos que analizamos dos jugadas, A y B.

Ya hemos determinado que A garantiza una evaluación de:

$$ V(A)=+300 $$

es decir, aproximadamente tres peones de ventaja para MAX.

Durante el análisis de B descubrimos que el adversario puede responder de manera que el valor de B no supere:

$$ V(B)\leq +100 $$

Por lo tanto:

$$ V(B)<V(A) $$

No resulta necesario continuar explorando B: independientemente de las demás respuestas posibles, MAX nunca preferirá esa jugada frente a A.

La poda Alfa-Beta formaliza matemáticamente este razonamiento.

5.2. Significado de Alfa y Beta

Durante la búsqueda se mantienen dos parámetros:

$$ \boxed{\alpha} $$

Es el mejor valor que MAX puede garantizar hasta ese momento, considerando las alternativas ya examinadas en el contexto de la búsqueda.

Actúa como una cota inferior.

$$ \boxed{\beta} $$

Es el mejor límite superior impuesto por MIN en el contexto de los nodos ancestros.

Actúa como una cota superior.

Inicialmente se establece:

$$ \alpha=-\infty $$

$$ \beta=+\infty $$

Durante el recorrido del árbol:

- En nodos MAX, el valor de α puede aumentar.
- En nodos MIN, el valor de β puede disminuir.
- Los límites heredados de los ancestros restringen lo que todavía es relevante explorar.

La condición fundamental de poda es:

$$ \boxed{\alpha\geq\beta} $$

Cuando se cumple, los nodos restantes de la rama actual pueden descartarse sin alterar el resultado Minimax de la raíz.

Es importante distinguir las cotas del valor exacto de una posición. Alfa y Beta no representan necesariamente la evaluación definitiva del nodo: describen el intervalo de valores que todavía puede ser relevante para sus ancestros.

5.3. Actualización de Alfa en nodos MAX

En un nodo MAX, cada hijo aporta un valor v_i.

El mejor valor local después de examinar k hijos es:

$$ v_{\text{MAX}}^{(k)}=\max_{1\leq i\leq k}v_i $$

La cota alfa se actualiza:

$$ \boxed{\alpha\leftarrow\max(\alpha,v_i)} $$

Al incorporar más hijos, el máximo parcial nunca disminuye:

$$ v_{\text{MAX}}^{(k+1)}\geq v_{\text{MAX}}^{(k)} $$

Esto permite comprender por qué es seguro interrumpir el análisis cuando el valor ya es demasiado alto para un ancestro MIN.

5.4. Actualización de Beta en nodos MIN

En un nodo MIN, la situación es simétrica.

El valor local después de examinar k hijos es:

$$ v_{\text{MIN}}^{(k)}=\min_{1\leq i\leq k}v_i $$

La cota beta se actualiza:

$$ \boxed{\beta\leftarrow\min(\beta,v_i)} $$

Al examinar más alternativas, el mínimo parcial nunca aumenta:

$$ v_{\text{MIN}}^{(k+1)}\leq v_{\text{MIN}}^{(k)} $$

Por eso, si MIN ya encontró una respuesta suficientemente mala para MAX, las demás respuestas pueden resultar irrelevantes para la decisión de un MAX ancestro.

6. Ejemplo completo de poda Alfa-Beta

Retomemos el árbol anterior, con tres alternativas A, B y C.

Supondremos que las ramas se exploran de izquierda a derecha.

La rama B se interrumpe después de examinar la hoja de valor 2. La hoja de valor 9 se representa únicamente para mostrar lo que contiene el árbol completo.

Paso 1. Examinar la alternativa A

MAX inicia con:

$$ \alpha=-\infty,\qquad\beta=+\infty $$

La alternativa A conduce a un nodo MIN con dos respuestas evaluadas en 3 y 5.

Primera respuesta:

$$ \beta_A=\min(+\infty,3)=3 $$

Segunda respuesta:

$$ \beta_A=\min(3,5)=3 $$

Se obtiene:

$$ V(A)=3 $$

Al regresar a la raíz MAX:

$$ \alpha=\max(-\infty,3)=3 $$

A partir de este instante, MAX sabe que dispone de una jugada capaz de garantizar una puntuación de al menos 3.

Paso 2. Examinar la alternativa B

Entramos en el nodo MIN correspondiente a B.

Este nodo recibe los límites:

$$ \alpha=3,\qquad\beta=+\infty $$

Su primera respuesta tiene evaluación 2.

Por lo tanto:

$$ \beta_B=\min(+\infty,2)=2 $$

Comprobamos:

$$ \alpha\geq\beta_B $$

$$ 3\geq2 $$

La condición de poda se cumple.

Esto significa que MIN ya dispone de una respuesta que limita B a un resultado de 2.

Aunque existieran otras respuestas, no podrían aumentar el mínimo:

$$ V(B)=\min(2,v_2,v_3,\ldots)\leq2 $$

Y, dado que:

$$ 2<3 $$

MAX no seleccionará B mientras conserve A como alternativa.

La segunda hoja de B no necesita examinarse.

Obsérvese que no afirmamos haber calculado exactamente $V(B)$. Sabemos únicamente que:

$$ V(B)\leq2 $$

Esta información resulta suficiente para descartarla.

Paso 3. Examinar la alternativa C

El nodo MIN correspondiente a C recibe:

$$ \alpha=3,\qquad\beta=+\infty $$

Su primera respuesta produce un valor de 4:

$$ \beta_C=\min(+\infty,4)=4 $$

Como:

$$ 3<4 $$

no puede realizarse la poda.

La segunda respuesta tiene valor 6:

$$ \beta_C=\min(4,6)=4 $$

Por lo tanto:

$$ V(C)=4 $$

De regreso en MAX:

$$ \alpha=\max(3,4)=4 $$

Paso 4. Decisión final

La información disponible es:

$$ V(A)=3 $$

$$ V(B)\leq 2 $$

$$ V(C)=4 $$

Por lo tanto:

$$ \boxed{V(s)=4,\qquad a^*=C} $$

El resultado es idéntico al obtenido con Minimax completo, pero una hoja no fue evaluada.

Este ahorro es pequeño porque el árbol es deliberadamente reducido. En árboles con millones de posiciones, la misma propiedad permite evitar la exploración de subárboles enteros.

6.1. Dos tipos de corte

Tradicionalmente se distinguen dos situaciones simétricas:

Corte Alfa (Alpha Cutoff): ocurre en un nodo MIN cuando su valor parcial resulta menor o igual que el alfa heredado. MAX ya dispone de una alternativa mejor en un ancestro.

Corte Beta (Beta Cutoff): ocurre en un nodo MAX cuando su valor parcial alcanza o supera el beta heredado. MIN ya dispone de una alternativa preferible en un ancestro.

En ambas situaciones se aplica la condición general:

$$ \alpha\geq\beta $$

La diferencia consiste en cuál de los límites provoca el corte.

7. Justificación matemática de la corrección de Alfa-Beta

La poda Alfa-Beta puede justificarse mediante el carácter monótono de los operadores mínimo y máximo.

7.1. Propiedad de los nodos MIN

Consideremos un nodo MIN con hijos:

$$ v_1,v_2,\ldots,v_n $$

Su valor exacto es:

$$ V_{\text{MIN}}=\min(v_1,v_2,\ldots,v_n) $$

Después de examinar únicamente los primeros k hijos:

$$ m_k=\min(v_1,\ldots,v_k) $$

Necesariamente:

$$ V_{\text{MIN}}\leq m_k $$

porque incorporar nuevos valores a un mínimo nunca puede aumentarlo.

Si un ancestro MAX ya dispone de un valor garantizado α y:

$$ m_k\leq\alpha $$

entonces:

$$ V_{\text{MIN}}\leq m_k\leq\alpha $$

En consecuencia, ese nodo MIN nunca permitirá mejorar la alternativa que MAX ya conoce.

Los hijos restantes son irrelevantes para la decisión de ese ancestro.

7.2. Propiedad de los nodos MAX

Para un nodo MAX:

$$ V_{\text{MAX}}=\max(v_1,v_2,\ldots,v_n) $$

Después de analizar k hijos:

$$ M_k=\max(v_1,\ldots,v_k) $$

Necesariamente:

$$ V_{\text{MAX}}\geq M_k $$

Si el nodo MAX está subordinado a un ancestro MIN con límite superior β y:

$$ M_k\geq\beta $$

entonces:

$$ V_{\text{MAX}}\geq M_k\geq\beta $$

MIN dispone de otra alternativa que no supera β, de modo que nunca necesitará elegir esta rama para mejorar su decisión.

Los demás hijos del nodo MAX pueden descartarse.

7.3. Invariante de búsqueda

Durante una ejecución correcta de Alfa-Beta se preserva el siguiente principio:

> Toda rama descartada es incapaz de modificar la decisión óptima de alguno de los ancestros que determina el resultado de la búsqueda.

Por inducción sobre la profundidad del árbol, los nodos no podados conservan las operaciones mínimo y máximo correspondientes. Los nodos podados se reemplazan por información de cotas que es suficiente para sus ancestros.

Por ello, al ejecutar una búsqueda exacta dentro del horizonte elegido:

$$ \boxed{ V_{\text{Alfa-Beta}}(s,d)=V_{\text{Minimax}}(s,d) } $$

para la raíz de la búsqueda realizada con una ventana inicial completa.

Esta igualdad supone una función de evaluación determinista, un modelo de estados correcto y las mismas reglas de profundidad y terminalidad.

La poda, por sí sola, no introduce pérdida de calidad en la decisión.

8. Implementación de Minimax

Una implementación recursiva convencional puede escribirse en pseudocódigo.

```
def minimax(position, depth, maximizing):        for move in moves:
child = make_move(position, move)            score = minimax(
child,                depth - 1,                False            )            best
= max(best, score)        return best    else:        best = +INF        for move
in moves:            child = make_move(position, move)            score = minimax(
child,                depth - 1,                True            )            best
= min(best, score)        return best
```

Esta implementación refleja directamente la formulación matemática, aunque presupone que `evaluate()` y `terminal_score()` utilizan una escala fija favorable a MAX.

Sus principales limitaciones son:

- Recorre todas las ramas.
- No aprovecha información de las alternativas ya examinadas.
- Repite el análisis de posiciones que pueden alcanzarse por diferentes secuencias.
- No contempla mecanismos especiales para capturas, amenazas o inestabilidad táctica.

La primera mejora fundamental consiste en incorporar los límites Alfa y Beta.

9. Implementación del algoritmo Alfa-Beta

Una implementación básica, con búsqueda de raíz incluida, es la siguiente:

```
INF = 10**12def alphabeta(position, depth, alpha, beta, maximizing):    if
is_terminal(position):        return terminal_score(position)    if depth == 0:
return evaluate(position)    moves = generate_legal_moves(position)    if
maximizing:        best = -INF        for move in moves:            child =
make_move(position, move)            score = alphabeta(                child,
depth - 1,                alpha,                beta,                False
)            best = max(best, score)            alpha = max(alpha, best)
if alpha >= beta:                break  # Corte Beta        return best    else:
best = +INF        for move in moves:            child = make_move(position, move)
score = alphabeta(                child,                depth - 1,
alpha,                beta,                True            )            best =
min(best, score)            beta = min(beta, best)            if alpha >= beta:
break  # Corte Alfa        return bestdef find_best_move(position, depth):
```

```
maximizing = white_to_move(position)    alpha = -INF    beta = +INF    best_score
= -INF if maximizing else +INF    best_move = None    for move in
generate_legal_moves(position):        child = make_move(position, move)
score = alphabeta(            child,            depth - 1,            alpha,
beta,            not maximizing        )        if maximizing:            if score
> best_score:                best_score = score                best_move = move
alpha = max(alpha, best_score)        else:            if score < best_score:
best_score = score                best_move = move            beta = min(beta,
best_score)    return best_move, best_score
```

El código es una referencia conceptual. Para funcionar como motor de ajedrez necesita implementaciones correctas de generación de jugadas legales, evaluación, transición de estados y detección de resultados terminales.

También debe recordar que los retornos de algunos nodos podados pueden representar cotas y no valores Minimax exactos del subárbol.

9.1. Tratamiento de posiciones terminales

En ajedrez, la evaluación terminal debe distinguir correctamente:

- Jaque mate.
- Ahogado.
- Tablas por material insuficiente.
- Tablas automáticas aplicables.
- Tablas reclamables cuando corresponda, según las reglas y el modelo de búsqueda.

Una escala habitual asigna un valor \(M\), muy superior a cualquier evaluación posicional normal, al jaque mate.

Por ejemplo:

\[M=100000 \]

Desde una perspectiva fija de las blancas:

\[U(s)= \begin{cases} +M & \text{mate favorable a blancas}\\ -M & \text{mate favorable a negras}\\ 0 & \text{tablas} \end{cases} \]

Para priorizar los mates más rápidos y retrasar las derrotas inevitables, puede incorporarse la distancia al mate en medias jugadas:

\[U(s)= \begin{cases} M-p & \text{mate de las blancas}\\ -M+p & \text{mate de las negras}\\ 0 & \text{tablas} \end{cases} \]

donde \(p\) es la distancia desde la raíz hasta el estado terminal.

Así, una victoria por mate en menos jugadas recibe una puntuación mayor, y una derrota más lejana recibe una puntuación menos negativa.

Debe existir una distinción entre los valores devueltos al protocolo UCI como centipeones (`score cp`) y las puntuaciones de mate (`score mate`), ya que no tienen el mismo significado.

10. Aplicación práctica: seleccionar la mejor jugada de ajedrez

10.1. Análisis de una posición concreta

Consideremos la siguiente posición, expresada en notación FEN:

```
6k1/5ppp/8/8/8/5q2/5PPP/6KQ w - - 0 1
```

Posición de ejemplo: juegan las blancas

Las blancas tienen el turno. La dama negra en f3 amenaza al rey blanco, por lo que la búsqueda debe considerar únicamente respuestas legales.

En esta posición, la dama negra ataca al peón de g2 y condiciona severamente la libertad del rey blanco. Una búsqueda Minimax no puede limitarse a comparar material: debe verificar el jaque, las posibles capturas y las consecuencias forzadas.

La secuencia lógica del motor es:

1. Identificar el jugador al que corresponde mover.
2. Generar las jugadas legales, descartando cualquier movimiento que deje al rey en jaque.
3. Aplicar temporalmente cada movimiento y calcular la posición resultante.
4. Generar las mejores respuestas negras, suponiendo un adversario óptimo.
5. Continuar alternando maximización y minimización hasta la profundidad establecida.
6. Evaluar las posiciones hoja o detectar resultados terminales.
7. Propagar hacia la raíz los valores Minimax.
8. Seleccionar la jugada que proporcione la mejor puntuación garantizada.

El ejemplo ilustra por qué la generación de jugadas legales y la detección de amenazas forman parte indispensable del motor. No basta con producir movimientos geométricamente posibles.

10.2. Ejemplo numérico de selección entre jugadas

Supongamos que un motor analiza una posición más equilibrada y considera las siguientes jugadas candidatas:

Jugada	Evaluación Minimax hipotética
1.e4	+35 cp
1.d4	+24 cp
1.Nf3	+18 cp
1.c4	+28 cp

Estos valores son exclusivamente ilustrativos; no corresponden a la salida medida de un motor específico.

Desde la perspectiva de las blancas:

$$ V(\text{e4})=35 $$

$$ V(\text{d4})=24 $$

$$ V(\text{Nf3})=18 $$

$$ V(\text{c4})=28 $$

La jugada seleccionada es:

$$ a^*=\arg\max(35,24,18,28) $$

$$ \boxed{a^*=\text{e4}} $$

Es esencial interpretar correctamente el resultado: el motor no asegura que e4 gane la partida, ni que sea la mejor jugada en términos absolutos. Afirma que, dentro de su horizonte de búsqueda y según su evaluador, e4 resulta preferible a las demás alternativas.

11. Negamax: una formulación más compacta

Debido a que el ajedrez es un juego de suma cero, Minimax puede simplificarse mediante la formulación Negamax.

En lugar de implementar alternativamente nodos MAX y MIN, se define la evaluación desde la perspectiva del jugador al que corresponde mover.

Sea:

$$ V(s,d) $$

la utilidad de la posición para el jugador activo.

Entonces:

$$ \boxed{ V(s,d)=\max_{a\in A(s)} \left[-V(T(s,a),d-1)\right] } $$

La negación aparece porque la evaluación del hijo está expresada desde la perspectiva del adversario.

Una ventaja para el oponente equivale a una desventaja propia.

11.1. Negamax con Alfa-Beta

La implementación recursiva es considerablemente más compacta:

```
def negamax(position, depth, alpha, beta):    if is_terminal(position):
return terminal_score_for_side_to_move(position)    if depth == 0:        return
evaluate_for_side_to_move(position)    best = -INF    moves =
generate_legal_moves(position)    for move in moves:        child =
make_move(position, move)        score = -negamax(            child,
depth - 1,            -beta,            -alpha        )        best = max(best,
score)        alpha = max(alpha, score)        if alpha >= beta:            break
return best
```

Obsérvese el cambio de límites en la llamada recursiva:

$$\boxed{ (\alpha,\beta)\rightarrow(-\beta,-\alpha) }$$

Esta transformación resulta de cambiar la perspectiva del jugador activo.

Si un valor v pertenece al intervalo:

$$\alpha<v<\beta$$

entonces:

$$-\beta<-v<-\alpha$$

Por eso, al negar los valores también deben intercambiarse y negarse las cotas.

Negamax es especialmente conveniente en implementaciones de motores de ajedrez porque evita mantener dos implementaciones distintas para los jugadores MAX y MIN.

12. Eficiencia de Alfa-Beta

12.1. Peor caso

Si las jugadas se examinan en un orden desfavorable, Alfa-Beta puede realizar prácticamente el mismo trabajo que Minimax completo.

Su complejidad temporal en el peor caso sigue siendo:

$$\boxed{O(b^d)}$$

donde:

- b: factor medio de ramificación.
- d: profundidad de búsqueda.

Por lo tanto, Alfa-Beta no elimina el carácter exponencial del problema.

12.2. Mejor caso

En un árbol uniforme, con ordenación perfecta de movimientos y condiciones ideales de búsqueda, Alfa-Beta puede reducir el número de hojas examinadas a un orden de:

$$\boxed{O(b^{d/2})}$$

Una forma habitual de expresar esta mejora es que la búsqueda puede alcanzar aproximadamente el doble de profundidad con un presupuesto comparable de evaluación de hojas, respecto de Minimax sin poda.

La cantidad exacta de hojas depende de la paridad de la profundidad y de las características del árbol, pero el comportamiento asintótico es el indicado.

12.3. Comparación numérica

Si:

$$b=35,\qquad d=6$$

Minimax completo examinaría aproximadamente:

\[N_{\text{Minimax}}=35^6 \]

\[N_{\text{Minimax}}=1.838.265.625 \]

En un caso idealizado de Alfa-Beta:

\[N_{\text{AB}}\sim35^3 \]

\[N_{\text{AB}}\sim42.875 \]

Evaluaciones de hojas: modelo teórico

Comparación en escala de millones para b=35 y d=6; Alfa-Beta representa el orden de magnitud del mejor caso idealizado.

0500100015002000MinimaxAlfa-Beta ideal

Esta diferencia muestra la importancia del algoritmo.

Sin embargo, debe evitarse interpretar el gráfico como una predicción exacta de rendimiento. En un motor real influyen el orden de movimientos, la presencia de jaques, las capturas, las extensiones de profundidad, la reutilización de posiciones y el costo de evaluación de cada nodo.

El número de nodos internos también contribuye al tiempo total, no solamente las hojas.

12.4. Complejidad espacial

En una búsqueda recursiva en profundidad, la memoria necesaria para mantener la pila de llamadas es proporcional a:

\[O(d) \]

si la generación y exploración de movimientos se realiza de forma incremental y el almacenamiento por nivel es acotado.

Si se materializan y almacenan explícitamente hasta \(b\) movimientos en cada nivel, la memoria auxiliar puede aproximarse por:

\[O(bd) \]

Esto es muy inferior a almacenar el árbol completo, que requeriría memoria exponencial.

13. La importancia del ordenamiento de jugadas

Una propiedad extraordinariamente importante de Alfa-Beta es que su eficiencia depende del orden en que se examinan los movimientos.

Considérese un nodo MIN con valores:

\[[2,9,6,5] \]

y un alfa heredado de:

$$ \alpha=3 $$

Si examinamos primero el valor 2:

$$ \beta=2 $$

Como:

$$ \beta\leq\alpha $$

podamos inmediatamente las otras tres ramas.

En cambio, si examinamos:

$$ [9,6,5,2] $$

necesitamos recorrer las cuatro respuestas para descubrir que el nodo puede reducirse a 2.

El resultado Minimax es idéntico, pero el esfuerzo de búsqueda es diferente.

13.1. Heurísticas de ordenamiento

Los motores modernos utilizan diversas estrategias para intentar examinar primero las jugadas más prometedoras:

Heurística	Principio
PV Move	Priorizar la jugada de la variante principal anterior.
TT Move	Priorizar la jugada registrada en una tabla de transposiciones.
MVV-LVA	Priorizar determinadas capturas según víctima y atacante.
Killer Moves	Recordar movimientos no capturantes que produjeron cortes anteriores.
History Heuristic	Utilizar estadísticas de efectividad de jugadas en búsquedas previas.
SEE	Estimar el balance de intercambios en una casilla.

Una ordenación frecuente coloca primero el movimiento sugerido por la tabla de transposiciones, luego capturas favorables y otras jugadas tácticamente importantes, y finalmente los movimientos restantes según heurísticas históricas.

La mejor ordenación no puede conocerse de antemano sin resolver previamente el problema de búsqueda, por lo que estas técnicas son aproximaciones prácticas.

14. Búsqueda iterativa en profundidad

Una optimización central de los motores de ajedrez consiste en ejecutar búsquedas sucesivas con profundidad creciente.

En lugar de comenzar directamente con profundidad 8:

$$ d=8 $$

se realizan búsquedas:

$$ d=1,2,3,\ldots,8 $$

Este procedimiento se denomina Iterative Deepening.



A primera vista, repetir las búsquedas parece desperdiciar tiempo. Sin embargo, existen beneficios significativos:

1. La mejor jugada encontrada en profundidad $d-1$ sirve para ordenar la búsqueda de profundidad d.
2. Las tablas de transposiciones conservan resultados aprovechables.
3. El motor puede finalizar al alcanzar un límite de tiempo.
4. Siempre dispone de una mejor jugada provisional procedente de una iteración completada.

En particular, el ordenamiento mejorado puede producir suficientes podas adicionales para compensar una parte sustancial del costo de las búsquedas anteriores.

14.1. Ventanas de aspiración

Si en profundidad $d-1$ se obtuvo una evaluación v, puede intentarse la siguiente búsqueda con una ventana estrecha:

$$ \alpha=v-\delta $$

$$ \beta=v+\delta $$

Por ejemplo:

$$ v=+45,\qquad\delta=30 $$

resulta:

$$ \alpha=15,\qquad\beta=75 $$

Una ventana estrecha puede aumentar las posibilidades de poda.

Sin embargo, si el valor verdadero queda fuera de la ventana, la búsqueda produce un resultado de tipo fail-low o fail-high y normalmente debe repetirse con límites más amplios.

Por lo tanto, las ventanas de aspiración intercambian un mayor potencial de poda por el riesgo de repetir parte del trabajo.

15. Tablas de transposiciones

15.1. Problema de los estados repetidos

Distintas secuencias de jugadas pueden conducir a una misma posición.

Por ejemplo, determinados órdenes de desarrollo de piezas pueden producir el mismo estado final.

Si el motor no conserva información de búsquedas anteriores, examinará nuevamente esa posición aunque ya la haya analizado.

Las tablas de transposiciones (TT) permiten registrar resultados asociados a posiciones previamente exploradas.

Generalmente se utiliza una clave hash, habitualmente de tipo Zobrist, para representar el estado.

Una entrada puede contener:

```
Hash de posición
Profundidad de búsqueda
Valor obtenido
Tipo de cota
Mejor jugada
Generación o antigüedad
```

15.2. Tipos de resultados almacenados

Una tabla de transposiciones debe distinguir entre valores exactos y cotas.

Tipo	Interpretación
EXACT	Valor Minimax calculado exactamente al horizonte correspondiente.
LOWERBOUND	El valor verdadero es mayor o igual que el valor almacenado.
UPPERBOUND	El valor verdadero es menor o igual que el valor almacenado.

Esta distinción resulta necesaria porque los cortes Alfa-Beta pueden devolver información parcial.

Por ejemplo, un resultado de tipo LOWERBOUND:

$$ V(s)\geq120 $$

no significa necesariamente:

$$ V(s)=120 $$

Usarlo como valor exacto podría producir decisiones incorrectas.

15.3. Consulta de una tabla de transposiciones

Conceptualmente:

```
entry = transposition_table.lookup(position)if entry is not None and entry.depth
>= depth:    if entry.type == EXACT:        return entry.score    if entry.type ==
LOWERBOUND:        alpha = max(alpha, entry.score)    if entry.type == UPPERBOUND:
beta = min(beta, entry.score)    if alpha >= beta:        return entry.score
```

Esta lógica representa una consulta simplificada. Una implementación de producción también debe administrar correctamente las puntuaciones de mate relativas a la distancia, las colisiones de hash, la validez del estado y el almacenamiento de nuevas entradas.

Además, la historia de repeticiones y las reglas de tablas requieren atención especial: dos posiciones con las mismas piezas no siempre son equivalentes respecto de todos los resultados posibles de la partida.

16. El problema del horizonte

Una limitación importante de Minimax con profundidad fija es el denominado Horizon Effect.

Supongamos que un motor evalúa una posición en la que una dama puede ser capturada en la siguiente jugada.

Si la búsqueda finaliza justo antes de la captura, el evaluador puede asignar una puntuación artificialmente favorable porque la pérdida material todavía no se produjo.

El problema no es estrictamente un error de Minimax: es una consecuencia de sustituir la resolución completa de la partida por una evaluación heurística a profundidad limitada.

16.1. Quiescence Search

Para reducir este problema se utiliza la búsqueda de quiescencia, o Quiescence Search.

Al alcanzar la profundidad nominal, en lugar de aplicar inmediatamente la evaluación estática, el motor continúa explorando una selección de jugadas tácticamente relevantes.

Habitualmente se consideran capturas, ciertas promociones y, según la implementación, otras amenazas forzantes.

La formulación básica desde Negamax puede expresarse mediante:

$$ q(s,\alpha,\beta) $$

En una posición tranquila, donde no es obligatorio responder a un jaque, se calcula primero la evaluación estática:

$$ v_0=E(s) $$

Si:

$$ v_0\geq\beta $$

puede producirse un corte.

En caso contrario:

$$ \alpha\leftarrow\max(\alpha,v_0) $$

y se exploran los movimientos tácticos seleccionados:

$$ v_a=-q(T(s,a),-\beta,-\alpha) $$

actualizando:

$$ \alpha\leftarrow\max(\alpha,v_a) $$

La búsqueda continúa hasta alcanzar una posición suficientemente estable o algún límite de seguridad establecido por el motor.

Cuando el rey se encuentra en jaque, no es válido utilizar simplemente la evaluación estática como alternativa a responder: deben considerarse las evasiones legales, ya que no existe la opción de permanecer en jaque.

La búsqueda de quiescencia es especialmente relevante cuando se usan evaluadores basados en material o redes NNUE, porque permite que la evaluación se aplique a posiciones menos inestables tácticamente.

17. Alfa-Beta en motores modernos de ajedrez

Un motor competitivo raramente utiliza Alfa-Beta aislado.

Normalmente implementa una arquitectura de búsqueda con varios componentes complementarios.

Arquitectura conceptual simplificada de un motor de ajedrez basado en Alfa-Beta. La interacción real entre los componentes es recursiva, y no estrictamente secuencial.

17.1. Otras optimizaciones frecuentes

Además de las técnicas ya presentadas, pueden utilizarse:

Principal Variation Search (PVS). Parte de la hipótesis de que el primer movimiento, después de una buena ordenación, probablemente sea el mejor. Los movimientos posteriores se examinan inicialmente con ventanas estrechas, y se reexaminan cuando es necesario.

Null Move Pruning. Permite estimar si una posición parece suficientemente favorable incluso concediendo ficticiamente un turno al adversario. Puede producir cortes eficientes, pero requiere precauciones importantes, especialmente en posiciones de zugzwang.

Late Move Reductions (LMR). Reduce inicialmente la profundidad para algunos movimientos tardíos y considerados menos prometedores. Cuando una búsqueda reducida indica que podrían ser importantes, puede requerirse una búsqueda de verificación más profunda.

Futility Pruning. Descarta ciertas ramas próximas al horizonte cuando una cota basada en evaluación estática y márgenes de seguridad indica que probablemente no mejorarán alfa.

Extensiones selectivas. Incrementan la profundidad ante situaciones especialmente relevantes, como determinados jaques o secuencias forzadas.

Debe destacarse una diferencia conceptual: mientras que la poda Alfa-Beta clásica conserva el resultado Minimax exacto al horizonte elegido, algunas de estas optimizaciones introducen aproximaciones heurísticas.

Por tanto, al incorporar LMR, Null Move Pruning o Futility Pruning, ya no puede garantizarse en general la igualdad exacta con una búsqueda Minimax exhaustiva de profundidad fija.

18. Diferencia entre encontrar la mejor jugada y encontrar una jugada ganadora

Es importante diferenciar tres conceptos.

Mejor jugada según el evaluador

Es aquella que maximiza el valor calculado por el motor a una determinada profundidad:

\[a^*=\arg\max_a V(T(s,a),d-1) \]

No implica necesariamente que la posición sea ganadora.

Mejor jugada según el resultado teórico

Es la jugada que maximiza el resultado de la partida bajo juego perfecto, considerando la búsqueda hasta estados terminales.

Si la posición es teóricamente tablas, una jugada óptima puede simplemente preservar esas tablas.

Mejor jugada práctica

En determinadas circunstancias competitivas, un jugador puede preferir una jugada objetivamente inferior porque complica la posición y aumenta las posibilidades de error del adversario.

Esa decisión incorpora factores probabilísticos o psicológicos que no están presentes en el Minimax tradicional de adversario perfecto.

Así, Minimax responde fundamentalmente a la pregunta:

> ¿Qué jugada ofrece el mejor resultado bajo el supuesto de que el adversario siempre seleccionará la mejor respuesta disponible, según el modelo de valoración utilizado?

No responde directamente:

> ¿Qué jugada tiene mayor probabilidad de hacer cometer un error a este adversario en particular?

Para abordar la segunda pregunta se necesitaría un modelo adicional del comportamiento del oponente.

19. Relación entre profundidad, evaluación y calidad de juego

La calidad de un motor basado en Alfa-Beta depende de varias dimensiones:

$$ Q\approx F(d,E,O,R) $$

donde:

- d: profundidad de búsqueda alcanzada.
- E: calidad de la función de evaluación.
- O: eficacia del ordenamiento de movimientos.
- R: calidad y corrección de las reglas, reducciones y extensiones.

Esta expresión es conceptual, no una fórmula empírica universal de fuerza ajedrecística.

Dos motores con idéntico evaluador pueden ofrecer resultados distintos si uno alcanza mayor profundidad o consigue ordenar mejor sus movimientos.

También es posible que un evaluador más preciso compense parcialmente una búsqueda menos profunda.

Sin embargo, no existe una relación simple que garantice que duplicar la profundidad o reducir a la mitad el error de evaluación produzca una determinada mejora de Elo.

19.1. Consideraciones sobre redes NNUE pequeñas

En sistemas con recursos computacionales limitados, el costo de evaluar una posición puede resultar decisivo.

Supongamos dos evaluadores:

Característica	**Evaluador A**	**Evaluador B**
Tiempo por evaluación	10 µs	100 µs
Precisión heurística	Moderada	Mayor
Evaluaciones por segundo, sin otros costos	100.000	10.000

El evaluador B es hipotéticamente más preciso, pero diez veces más costoso.

Al usar Alfa-Beta, la decisión no puede tomarse solamente comparando la calidad estática de las evaluaciones.

Debe medirse el rendimiento completo:

$$ NPS=\frac{N_{\text{nodos}}}{t_{\text{segundos}}} $$

y también:

$$ D(t)=\text{profundidad alcanzada en tiempo }t $$

Una red pequeña y eficiente puede permitir búsquedas más profundas; una red más grande puede proporcionar evaluaciones más precisas pero reducir la profundidad alcanzable.

La combinación óptima depende de la arquitectura del procesador, el costo de generación de movimientos, el ordenamiento, la posibilidad de actualizar incrementalmente la red y la calidad efectiva del evaluador.

20. Ejemplo de telemetría de un motor UCI

Un motor que implementa búsqueda Alfa-Beta puede reportar su progreso mediante el protocolo UCI.

Un ejemplo ilustrativo es:

```
info depth 4 score cp 18 nodes 12500 time 120 nps 104166 pv e2e4 e7e5 g1f3 b8c6

info depth 5 score cp 32 nodes 58000 time 510 nps 113725 pv e2e4 c7c5 g1f3 d7d6

info depth 6 score cp 25 nodes 210000 time 1780 nps 117977 pv e2e4 e7e5 g1f3 b8c6
f1b5 a7a6

bestmove e2e4
```

Los valores son ficticios y se incluyen únicamente para explicar los campos del protocolo.

La telemetría contiene:

Campo	Significado
`depth`	Profundidad nominal alcanzada, expresada en medias jugadas.
`score cp`	Evaluación en centipeones, desde la perspectiva definida por UCI.
`nodes`	Nodos visitados durante la búsqueda según el contador del motor.
`time`	Tiempo acumulado de búsqueda en milisegundos.
`nps`	Nodos procesados por segundo.
`pv`	Principal Variation: secuencia principal de jugadas encontrada.
`bestmove`	Mejor movimiento seleccionado.

La puntuación puede cambiar entre profundidades porque el motor descubre nuevas consecuencias.

Una posición inicialmente evaluada en +32 cp puede pasar a +25 cp cuando se detecta una defensa que no era visible en la búsqueda anterior.

La variante principal también puede cambiar completamente al aumentar la profundidad.

Esto no significa necesariamente que el motor esté funcionando mal; refleja el carácter aproximado de una búsqueda de horizonte limitado.

21. Limitaciones fundamentales

Aunque Minimax con Alfa-Beta es muy eficiente en comparación con Minimax completo, conserva varias limitaciones estructurales.

21.1. Dependencia de la evaluación

El algoritmo utiliza los valores que recibe del evaluador.

Si la función de evaluación considera favorable una posición que en realidad contiene una amenaza táctica decisiva, Alfa-Beta puede propagar correctamente una valoración equivocada.

La corrección matemática del algoritmo de búsqueda no garantiza la corrección estratégica del evaluador.

21.2. Dependencia de la profundidad

Una búsqueda de profundidad limitada puede no detectar combinaciones cuyo desenlace aparece más allá del horizonte.

Extensiones, quiescencia y búsqueda selectiva reducen ciertos problemas, pero no eliminan esta limitación general.

21.3. Complejidad exponencial

Incluso con poda eficaz, el crecimiento del árbol continúa siendo un desafío.

El mejor caso idealizado:

$$ O(b^{d/2}) $$

sigue siendo exponencial respecto de la profundidad.

21.4. Calidad del ordenamiento

Un ordenamiento deficiente reduce las oportunidades de poda y puede producir diferencias importantes en el número de nodos examinados.

21.5. Corrección de la implementación

Los errores en generación de movimientos, detección de jaques, tratamiento del enroque, captura al paso, promociones o tablas pueden invalidar todo el análisis.

Por ello, en un motor real, la verificación de la legalidad y de la consistencia del estado debe considerarse anterior a las optimizaciones de rendimiento.

22. Conclusiones

Minimax con poda Alfa-Beta constituye un ejemplo fundamental de cómo el razonamiento matemático puede transformar un problema de búsqueda computacionalmente costoso en uno considerablemente más eficiente.

Minimax modela el comportamiento adversarial mediante operaciones alternadas de máximo y mínimo:

$$ V_{\text{MAX}}(s)=\max_{a\in A(s)}V(T(s,a)) $$

$$ V_{\text{MIN}}(s)=\min_{a\in A(s)}V(T(s,a)) $$

La poda Alfa-Beta incorpora información sobre los límites de utilidad relevantes:

$$ \alpha=\text{cota inferior relevante para MAX} $$

$$ \beta=\text{cota superior relevante impuesta por MIN} $$

y descarta ramas cuando:

$$ \boxed{\alpha\geq\beta} $$

La importancia de este mecanismo reside en que evita cálculos cuyo resultado no puede alterar la decisión de los ancestros.

Bajo las condiciones de una búsqueda exacta a profundidad fija, Alfa-Beta no modifica la respuesta de Minimax: solamente reduce el esfuerzo necesario para obtenerla.

En el ajedrez, esta técnica se combina con evaluación heurística o neuronal, búsqueda iterativa, tablas de transposiciones, ordenamiento de movimientos y búsqueda de quiescencia.

El resultado es una arquitectura capaz de explorar millones de posiciones y seleccionar jugadas mediante el análisis de sus consecuencias, incluso cuando no resulta posible resolver completamente el árbol de la partida.

La idea central es que un motor de ajedrez no necesita examinar todas las continuaciones posibles para descartar una jugada: necesita demostrar que esa jugada no puede superar a otra alternativa ya conocida.

Esta distinción entre conocer el valor exacto de una alternativa y conocer lo suficiente para descartarla es el fundamento matemático y computacional de la poda Alfa-Beta.

Referencias bibliográficas recomendadas

1. Knuth, D. E., & Moore, R. W. (1975). An Analysis of Alpha-Beta Pruning. Artificial Intelligence, 6(4), 293–326.
2. Pearl, J. (1984). Heuristics: Intelligent Search Strategies for Computer Problem Solving. Addison-Wesley.
3. Russell, S., & Norvig, P. Artificial Intelligence: A Modern Approach. Secciones de búsqueda adversarial y juegos.
4. Shannon, C. E. (1950). Programming a Computer for Playing Chess. Philosophical Magazine, 41(314), 256–275.
5. Marsland, T. A. (1986). A Review of Game-Tree Pruning. ICCA Journal, 9(1), 3–19.
6. Chessprogramming Wiki — Alpha-Beta. Recurso técnico sobre los fundamentos, las variantes y las técnicas de implementación de Alfa-Beta.
7. Chessprogramming Wiki — Negamax. Formulación Negamax y relación con la búsqueda Alfa-Beta.

La arquitectura

En la arquitectura se resume todo lo que tiene que tener una plataforma, y también lo que no va a tener nunca. Es en cierta medida el contrato intelectual o técnico de lo que se quiere conseguir con un desarrollo.

Es posible que los objetivos que se persiguen difieran, incluso substancialmente, de los objetivos de otros desarrolladores.

A sus ojos ésta plataforma puede lucir muy poco atractiva, de bajo poder o incluso no totalmente necesaria.

Sin embargo hay algunos desafíos técnicos, que de alcanzarse, permitirán considerar el resultado un aporte original.

La plataforma combina motores de ajedrez, con sistemas embebidos, con técnicas avanzadas de optimización. En muchos casos será *un perro que canta tangos* , no es maravilloso como lo hace, sino el mero hecho que lo haga.

Aplicación de la ecuación de Bellman al aprendizaje profundo del valor y la política de juego en ajedrez

1. Introducción

La ecuación de Bellman constituye uno de los fundamentos matemáticos de la programación dinámica, la teoría de decisiones secuenciales y el aprendizaje por refuerzo (Reinforcement Learning, RL).

Su importancia reside en una idea aparentemente sencilla:

> El valor de una decisión puede expresarse mediante la recompensa inmediata que produce y el valor esperado de las decisiones futuras que quedan disponibles después de ejecutarla.

Esta propiedad, conocida como principio de optimalidad de Bellman, permite descomponer problemas de decisión de largo plazo en relaciones recursivas entre estados sucesivos.

En el ajedrez, el objetivo consiste en determinar qué movimientos conducen al mejor resultado posible de una partida, considerando que ambos jugadores persiguen objetivos opuestos.

En principio, si fuera posible asignar correctamente un valor a todas las posiciones de ajedrez, elegir una jugada se reduciría a comparar los valores de las posiciones que produce.

El problema es que esos valores son desconocidos para prácticamente todas las posiciones y el espacio de estados es demasiado grande para calcularlos de manera exhaustiva.

El aprendizaje profundo ofrece una alternativa: aproximar la función de valor mediante una red neuronal y aprender simultáneamente una política que asigne probabilidades o preferencias a las jugadas disponibles.

En este enfoque, la ecuación de Bellman no es una arquitectura neuronal ni un algoritmo de entrenamiento por sí misma. Es una relación matemática que caracteriza cómo deberían ser los valores coherentes de estados sucesivos. El aprendizaje profundo proporciona un mecanismo aproximado para intentar satisfacer esa relación a partir de la experiencia.

Examinaremos una arquitectura en la cual un agente aprende a jugar ajedrez mediante partidas contra sí mismo, utilizando:

- Una función de valor $V_\theta(s)$, que estima el resultado esperado de una posición.
- Una función de acción-valor $Q_\theta(s,a)$, que estima el mérito de ejecutar una jugada.
- Una política $\pi_\phi(a\mid s)$, que determina la selección de movimientos.
- La ecuación de Bellman, que relaciona las valoraciones de posiciones sucesivas.
- Un procedimiento de aprendizaje por diferencias temporales o por resultados finales.

- Opcionalmente, un mecanismo de búsqueda que mejore las decisiones y produzca objetivos de entrenamiento más informativos.

El objetivo es estudiar qué podría aprender un sistema de estas características, cómo lo aprendería y qué dificultades aparecen al aplicarlo al ajedrez.

2. Formulación matemática del ajedrez como problema de decisión

2.1. Definición del estado

Sea:

$$ s_t\in\mathcal S $$

el estado del juego en el instante t.

El estado debe incluir suficiente información para que las transiciones futuras dependan únicamente de él y de las acciones que se ejecuten.

Para el ajedrez, una representación completa debe contemplar:

$$ s_t=(B_t,c_t,C_t,e_t,h_t,H_t) $$

donde:

- B_t: distribución de las piezas.
- c_t: jugador al que corresponde mover.
- C_t: derechos de enroque.
- e_t: situación de captura al paso.
- h_t: contador relevante para las reglas de movimientos sin capturas ni movimientos de peones.
- H_t: información histórica necesaria para determinar repeticiones y otras condiciones dependientes de la historia.

La inclusión de información histórica es importante. Una representación limitada únicamente al tablero y al turno puede no satisfacer completamente la propiedad de Markov, porque la posibilidad de reclamar tablas por repetición depende de acontecimientos anteriores.

En términos de un proceso de decisión de Markov:

$$ P(s_{t+1}\mid s_0,a_0,\ldots,s_t,a_t) = P(s_{t+1}\mid s_t,a_t) $$

La igualdad significa que, conocido el estado actual completo y la acción seleccionada, la historia anterior ya no aporta información adicional para determinar la transición siguiente.

2.2. Espacio de acciones

Definimos:

$$ \mathcal A(s)=\{a_1,a_2,\ldots,a_n\} $$

como el conjunto de movimientos legales en la posición s.

La cantidad de movimientos disponibles cambia de una posición a otra.

Desde la posición inicial existen 20 jugadas legales. En posiciones posteriores pueden existir muchas más, y en un estado terminal no quedan jugadas que permitan continuar la partida.

El agente debe elegir exclusivamente entre acciones legales:

$$ a_t\in\mathcal A(s_t) $$

Esto constituye una restricción fundamental para las redes neuronales de política, porque una red puede producir una salida para un conjunto fijo de movimientos posibles, pero no todos serán legales en una posición determinada.

2.3. Transición de estados

El ajedrez es determinista respecto de las jugadas:

$$ s_{t+1}=T(s_t,a_t) $$

Una vez conocido el estado y la jugada legal, el estado siguiente está determinado.

No existe incertidumbre física sobre el resultado de ejecutar una acción, a diferencia de otros problemas de aprendizaje por refuerzo, como el control de robots sometidos a perturbaciones o el comportamiento de sistemas con transiciones probabilísticas.

La incertidumbre del ajedrez aparece principalmente porque no conocemos qué jugadas elegirá el adversario y porque no es posible evaluar exhaustivamente todas las consecuencias futuras.

2.4. Recompensa

Una formulación particularmente sencilla utiliza recompensas únicamente al finalizar la partida.

Desde la perspectiva de las blancas:

$$ z= \begin{cases} +1 & \text{victoria de las blancas}\\ 0 & \text{tablas}\\ -1 & \text{victoria de las negras} \end{cases} $$

Para las negras, el resultado tiene signo opuesto:

$$ z_N=-z_B $$

La recompensa inmediata puede definirse como:

$$ r_t= \begin{cases} 0 & \text{si la jugada no finaliza la partida}\\ z & \text{si la jugada finaliza la partida} \end{cases} $$

La escala anterior es un ejemplo de recompensa escasa (sparse reward).

Durante la mayor parte de la partida, el sistema no recibe ninguna señal inmediata de éxito o fracaso. Solamente al finalizar obtiene una indicación del resultado alcanzado.

Esta característica tendrá consecuencias importantes sobre la velocidad y la estabilidad del aprendizaje.

3. La ecuación de Bellman en su formulación tradicional

3.1. Retorno acumulado

En un problema de aprendizaje por refuerzo, el retorno desde un instante t se define como:

$$\boxed{ G_t=\sum_{k=0}^{T-t-1}\gamma^k r_{t+k+1} }$$

donde:

- T representa el instante terminal.
- r_{t+k+1} es la recompensa recibida después de una acción.
- $\gamma\in[0,1]$ es el factor de descuento.

El factor de descuento permite dar menor peso a recompensas alejadas temporalmente.

En un problema de control económico, por ejemplo, puede ser razonable que una recompensa obtenida hoy tenga mayor valor que una obtenida dentro de varios años.

En el ajedrez, la interpretación es distinta.

Si el objetivo consiste exclusivamente en ganar, empatar o perder, puede elegirse:

$$\boxed{\gamma=1}$$

De ese modo, una victoria no pierde valor solamente por requerir más movimientos.

Si se selecciona $\gamma<1$, se introduce una preferencia temporal: bajo una formulación apropiada, las victorias cercanas resultan más valiosas y las derrotas lejanas pueden resultar menos perjudiciales.

Esto modifica el objetivo de optimización, por lo que no es una decisión puramente numérica.

3.2. Función de valor de una política

Sea:

$$\pi(a\mid s)$$

la probabilidad de seleccionar la acción a cuando el agente se encuentra en s.

La función de valor de una política se define como:

$$\boxed{ V^\pi(s)=\mathbb E_\pi[G_t\mid s_t=s] }$$

Representa el retorno esperado si el agente comienza en el estado s y posteriormente sigue la política π.

La ecuación de Bellman de evaluación de una política es:

$$\boxed{ V^\pi(s)= \sum_{a\in\mathcal A(s)} \pi(a\mid s) \sum_{s'} P(s'\mid s,a) \left[r(s,a,s')+\gamma V^\pi(s') \right] }$$

Esta relación se obtiene separando la primera recompensa del retorno restante:

$$G_t=r_{t+1}+\gamma G_{t+1}$$

Tomando esperanza condicional:

$$V^\pi(s_t) = \mathbb E_\pi \left[r_{t+1}+\gamma V^\pi(s_{t+1}) \mid s_t \right]$$

La ecuación es recursiva: el valor de una posición está determinado por las recompensas inmediatas y los valores de las posiciones que pueden alcanzarse desde ella.

3.3. Simplificación en un entorno determinista

Como el ajedrez tiene transiciones deterministas, para cada acción legal existe un único sucesor:

$$ s'=T(s,a) $$

Por lo tanto:

$$ P(s'\mid s,a)= \begin{cases} 1 & s'=T(s,a)\\ 0 & \text{en otro caso} \end{cases} $$

La ecuación se simplifica a:

$$ \boxed{ V^\pi(s)= \sum_{a\in\mathcal A(s)} \pi(a\mid s) \left[r(s,a)+\gamma V^\pi(T(s,a)) \right] } $$

Pero aquí aparece una particularidad esencial: en el ajedrez los jugadores tienen intereses opuestos.

No puede aplicarse ingenuamente la ecuación de optimalidad de un único agente que maximiza el retorno en todos los turnos, porque eso equivaldría a suponer que ambos jugadores colaboran para mejorar la misma utilidad.

4. Bellman en juegos adversariales de suma cero

4.1. Formulación con una perspectiva fija

Definamos:

$$ V(s) $$

como el valor de una posición desde la perspectiva de las blancas.

Cuando mueven las blancas, buscan maximizar ese valor:

$$ \boxed{ V^*(s)= \max_{a\in\mathcal A(s)} \left[r_B(s,a)+\gamma V^*(T(s,a)) \right] } $$

Cuando mueven las negras, buscan minimizar el valor de las blancas:

$$ \boxed{ V^*(s)= \min_{a\in\mathcal A(s)} \left[r_B(s,a)+\gamma V^*(T(s,a)) \right] } $$

Por lo tanto:

$$ \boxed{ V^*(s)= \begin{cases} \max\limits_{a\in\mathcal A(s)} [r_B(s,a)+\gamma V^*(T(s,a))] & \text{turno blanco}\\[6pt] \min\limits_{a\in\mathcal A(s)} [r_B(s,a)+\gamma V^*(T(s,a))] & \text{turno negro} \end{cases} } $$

Esta es la versión adversarial de la ecuación de optimalidad.

Cuando $\gamma=1$ y las recompensas no terminales son nulas, la formulación coincide con la recurrencia básica de Minimax:

$$ V^*(s)= \begin{cases} \max_a V^*(T(s,a)) & \text{blancas}\\ \min_a V^*(T(s,a)) & \text{negras} \end{cases} $$

Así, Minimax y Bellman no son principios incompatibles.

Minimax representa precisamente una forma de programación dinámica adversarial sobre el árbol de juego.

La diferencia está en cómo se obtiene el valor de las posiciones. Minimax con búsqueda lo calcula retrospectivamente a partir de hojas evaluadas, mientras que el aprendizaje por refuerzo intenta aprender una aproximación reutilizable del valor a partir de numerosas experiencias.

4.2. Formulación desde el jugador activo

Para trabajar con una sola red neuronal es conveniente definir:

$$ V(s) $$

como el valor de la posición desde la perspectiva del jugador al que corresponde mover.

En ese caso, después de ejecutar una jugada, el turno cambia y la evaluación debe cambiar de signo.

Si todas las recompensas intermedias son cero:

$$ \boxed{ V^*(s)= \max_{a\in\mathcal A(s)} \left[-\gamma V^*(T(s,a)) \right] } $$

para transiciones no terminales, con:

$$ V^*(s_T)=z_T $$

en estados terminales evaluados desde la perspectiva que se haya definido para ellos.

Una formulación especialmente clara utiliza:

$$ Q^*(s,a)=r(s,a)-\gamma V^*(s') $$

para una transición no terminal, y la recompensa final directamente cuando la acción termina la partida.

Entonces:

$$ \boxed{ V^*(s)=\max_{a\in\mathcal A(s)}Q^*(s,a) } $$

Esta expresión muestra que el agente no necesita distinguir explícitamente entre blancas y negras en su regla de selección: ambos jugadores maximizan su propio valor, cambiando el signo del valor adversario.

4.3. Política óptima

Si se conoce $Q^*(s,a)$, la política óptima determinista puede establecerse mediante:

$$ \boxed{ \pi^*(s)=\arg\max_{a\in\mathcal A(s)}Q^*(s,a) } $$

La función Q^* proporciona una valoración de las acciones.

La política determina cuál de ellas ejecutar.

Esta distinción será fundamental cuando incorporemos redes neuronales.

5. Ejemplo numérico de aplicación de Bellman

Supongamos una posición en la cual juegan las blancas y existen tres movimientos relevantes:

$$ \mathcal A(s)=\{a_1,a_2,a_3\} $$

Los movimientos conducen a posiciones donde corresponde jugar a las negras.

Una red neuronal devuelve las siguientes estimaciones desde la perspectiva del jugador activo en cada posición sucesora:

Movimiento blanco	**Valor de la posición para las negras**
a_1	$-0,40$
a_2	$+0,25$
a_3	$-0,10$

Tomemos:

$$ \gamma=1 $$

y recompensas intermedias nulas.

Aplicando Bellman con cambio de perspectiva:

$$ Q(s,a_1)=-(-0,40)=+0,40 $$

$$ Q(s,a_2)=-(+0,25)=-0,25 $$

$$ Q(s,a_3)=-(-0,10)=+0,10 $$

La política codiciosa seleccionará:

$$ a^*=\arg\max(0,40,-0,25,0,10) $$

$$ \boxed{a^*=a_1} $$

Valor de las acciones desde la perspectiva blanca

Valores hipotéticos obtenidos cambiando el signo de las evaluaciones de los estados sucesores.

-0.3-0.10.10.30.5$a_1a_2a_3$

La interpretación importante es que a_1 no se selecciona por ser visualmente atractiva ni por obtener una ventaja material inmediata, sino porque conduce a una posición que, según el evaluador, resulta desfavorable para el adversario.

No obstante, los valores son estimaciones. Si la red se equivoca al asignar $-0,40$ a la primera posición sucesora, la selección también puede ser equivocada.

El mecanismo de aprendizaje intentará corregir precisamente esas estimaciones.

6. Aproximación de la función de valor mediante aprendizaje profundo

La formulación anterior presupone que conocemos $V^*(s)$, lo que en realidad constituye el problema que deseamos resolver.

El espacio de posiciones de ajedrez es demasiado grande para almacenar explícitamente el valor de cada estado.

Por lo tanto, se propone utilizar una red neuronal parametrizada:

$$\boxed{ V_\theta(s)\approx V^*(s) }$$

donde:

$$\theta=\{W_1,b_1,W_2,b_2,\ldots,W_L,b_L\}$$

representa los pesos y sesgos de una red de L capas.

La red recibe una representación de la posición y produce una estimación escalar de su valor.

6.1. Representación de entrada

Una posibilidad consiste en representar el tablero mediante planos binarios.

Para las seis clases de piezas de cada jugador:

$$6\times2=12$$

se utilizan doce matrices de dimensión 8×8.

Cada elemento indica la presencia o ausencia de una determinada pieza en una casilla.

Por ejemplo, el plano correspondiente a los peones blancos puede representarse como:

$$X_{\text{peones blancos}}\in\{0,1\}^{8\times8}$$

donde:

$$X_{ij}= \begin{cases} 1 & \text{si existe un peón blanco en }(i,j)\\ 0 & \text{en otro caso} \end{cases}$$

La representación básica tiene:

$$12\times8\times8=768$$

características binarias.

Sin embargo, esto no es suficiente para representar completamente el estado del juego.

Se necesitan características adicionales para identificar, entre otras cosas, el turno, los derechos de enroque, las posibilidades de captura al paso y la información relevante para las reglas de tablas.

Puede construirse un tensor más amplio:

$$X(s)\in\mathbb R^{C\times8\times8}$$

donde C representa la cantidad total de canales.

También es posible representar las piezas desde la perspectiva del jugador activo, intercambiando los colores y orientando el tablero de manera canónica.

Esa decisión reduce ciertas simetrías que la red tendría que aprender, pero exige transformar coherentemente la codificación de jugadas y las características históricas.

6.2. Arquitectura de la red de valor

Una red simple completamente conectada puede expresarse mediante:

$$ h_1=\sigma(W_1x+b_1) $$

$$ h_2=\sigma(W_2h_1+b_2) $$

$$ \vdots $$

$$ h_L=\sigma(W_Lh_{L-1}+b_L) $$

$$ \boxed{ V_\theta(s)=\tanh(w_v^\top h_L+b_v) } $$

La función tangente hiperbólica limita la salida al intervalo:

$$ V_\theta(s)\in[-1,1] $$

Esto resulta coherente con el objetivo de aprender resultados de victoria, tablas o derrota.

Por ejemplo:

$$ V_\theta(s)=+0{,}70 $$

significa que la red estima una posición favorable para el jugador activo, de acuerdo con la escala de entrenamiento.

No debe interpretarse automáticamente como un 70 % de probabilidad de victoria.

Si se utiliza la codificación $(+1,0,-1)$, la esperanza del resultado es:

$$ V(s)=P(W\mid s)-P(L\mid s) $$

porque las tablas tienen contribución nula.

Y como:

$$ P(W\mid s)+P(D\mid s)+P(L\mid s)=1 $$

un único valor escalar no permite reconstruir por separado las tres probabilidades.

Para obtener probabilidades de victoria, tablas y derrota resulta más apropiado utilizar una cabeza de salida WDL (Win–Draw–Loss):

$$ \mathbf p_\theta(s)= \begin{bmatrix} P_\theta(W\mid s)\\ P_\theta(D\mid s)\\ P_\theta(L\mid s) \end{bmatrix} $$

mediante una función Softmax.

Entonces, el valor esperado puede obtenerse como:

$$\boxed{ V_\theta(s)=P_\theta(W\mid s)-P_\theta(L\mid s) }$$

Esta representación conserva más información que un único escalar.

7. Cómo se aprende la función de valor

El problema fundamental es determinar los parámetros:

$$\theta^*$$

que permitan aproximar correctamente el valor de las posiciones.

Existen varias formas de generar los objetivos de entrenamiento.

7.1. Aprendizaje a partir del resultado final

Consideremos una partida completa:

$$s_0,a_0,s_1,a_1,\ldots,s_T$$

Al terminar, conocemos el resultado:

$$z\in\{-1,0,+1\}$$

Si z se expresa desde la perspectiva de las blancas y la red evalúa desde el jugador activo, el objetivo de cada posición será:

$$z_t= \begin{cases} z & \text{si mueven las blancas en }s_t\\ -z & \text{si mueven las negras en }s_t \end{cases}$$

Se intenta minimizar:

$$\boxed{ L_V(\theta)= \frac{1}{N} \sum_{i=1}^{N} \left[V_\theta(s_i)-z_i \right]^2 }$$

La función de pérdida mide el error cuadrático entre la predicción neuronal y el resultado observado.

El entrenamiento ajusta los pesos mediante descenso por gradiente:

$$\boxed{ \theta_{k+1} = \theta_k-\eta\nabla_\theta L_V(\theta_k) }$$

donde η es la tasa de aprendizaje.

Este mecanismo es una forma de aprendizaje Monte Carlo a partir de retornos completos.

Tiene una ventaja importante: los objetivos se obtienen de resultados efectivamente observados, sin depender de una estimación neuronal del estado siguiente.

Pero también presenta una dificultad: una posición muy favorable puede terminar en derrota porque el jugador cometió errores posteriores.

Por consiguiente, el resultado final es una muestra del retorno de la política que produjo la partida, no necesariamente del valor óptimo de la posición.

7.2. Aprendizaje por diferencias temporales

La ecuación de Bellman permite aprender sin esperar necesariamente el final de la partida.

Este mecanismo se denomina Temporal Difference Learning (TD).

Para una transición no terminal:

\[s_t\xrightarrow{a_t}s_{t+1} \]

desde la perspectiva alternada de los jugadores, se define el objetivo:

\[\boxed{ y_t=r_t-\gamma V_{\bar\theta}(s_{t+1}) } \]

donde \(\bar\theta\) son los parámetros de una red de referencia, que puede mantenerse temporalmente fija.

El signo negativo aparece porque después del movimiento corresponde jugar al adversario.

Si la acción termina la partida, se utiliza simplemente la recompensa terminal desde la perspectiva del jugador que ejecutó la acción:

\[y_t=r_t \]

Así, una expresión completa es:

\[\boxed{ y_t= \begin{cases} r_t & \text{si }s_{t+1}\text{ es terminal}\\ r_t-\gamma V_{\bar\theta}(s_{t+1}) & \text{en otro caso} \end{cases} } \]

El error TD es:

\[\boxed{ \delta_t=y_t-V_\theta(s_t) } \]

El objetivo de aprendizaje consiste en reducir ese error:

\[L_{\text{TD}}(\theta)= \frac12 \left[V_\theta(s_t)-y_t \right]^2 \]

Para una actualización TD semigradiente, manteniendo fijo el objetivo \(y_t\) durante el paso de entrenamiento:

\[\boxed{ \theta_{t+1} = \theta_t+ \eta\delta_t\nabla_\theta V_\theta(s_t) } \]

Esta ecuación es fundamental.

El agente modifica los parámetros de la red para hacer que el valor predicho en una posición se aproxime al valor respaldado por la posición sucesora y la recompensa observada.

7.3. Ejemplo de una actualización TD

Supongamos una posición en la que juegan las blancas.

La red estima:

\[V_\theta(s_t)=0,20 \]

Las blancas realizan una jugada no terminal, y la red de referencia evalúa la nueva posición desde la perspectiva negra:

$$ V_{\bar\theta}(s_{t+1})=-0,50 $$

Con:

$$ r_t=0,\qquad\gamma=1 $$

el objetivo TD es:

$$ y_t=0-(-0,50)=+0,50 $$

El error:

$$ \delta_t=0,50-0,20 $$

$$ \boxed{\delta_t=+0,30} $$

La red había asignado a la posición inicial un valor de $0,20$, pero la evaluación posterior sugiere que debería acercarse a $0,50$.

El entrenamiento ajusta sus parámetros para aumentar la predicción, localmente, en la dirección del gradiente.

La actualización no garantiza que el nuevo valor sea correcto. Solamente reduce la discrepancia con el objetivo generado.

Esta observación conduce a una dificultad central del aprendizaje mediante Bellman: el objetivo también puede estar equivocado.

8. Propagación retrospectiva del resultado de una partida

La importancia del aprendizaje temporal se comprende mejor observando cómo una recompensa final puede propagarse hacia posiciones anteriores.

Supongamos que un agente comienza sin conocimiento y asigna valor cero a todas las posiciones:

$$ V_\theta(s)=0 $$

Luego juega una partida que termina en victoria de las blancas.

Esquema conceptual: la recompensa observada al terminar la partida puede modificar las estimaciones de posiciones anteriores a través de aprendizaje retrospectivo o actualizaciones TD repetidas.

Para simplificar la explicación, utilizaremos inicialmente un valor definido desde una perspectiva fija, sin alternar signos.

Supongamos que:

$$ s_0\rightarrow s_1\rightarrow s_2\rightarrow s_3\rightarrow s_T $$

y que la última transición produce:

$$ r=+1 $$

Con una actualización tabular TD(0), tasa de aprendizaje:

$$\eta=0,5$$

y valor inicial cero, la transición final produce:

$$V_{\text{nuevo}}(s_3) = V(s_3)+\eta[1-V(s_3)]$$

$$V_{\text{nuevo}}(s_3)=0,5$$

En una revisión posterior de la transición anterior:

$$V_{\text{nuevo}}(s_2) = 0+0,5[0,5-0]$$

$$V_{\text{nuevo}}(s_2)=0,25$$

Continuando retrospectivamente:

$$V_{\text{nuevo}}(s_1)=0,125$$

$$V_{\text{nuevo}}(s_0)=0,0625$$

Los valores obtenidos son:

Estado	Valor después del ejemplo de actualización
s_0	0,0625
s_1	0,125
s_2	0,2500
s_3	0,5000
Final	1,0000

Esta demostración supone actualizaciones tabulares retrospectivas, con $\gamma=1$, y no representa directamente el resultado de una sola pasada TD(0) en orden cronológico.

En una red neuronal, modificar los pesos para mejorar un estado también puede alterar los valores de otros estados, debido a la generalización entre posiciones.

Si se utiliza una evaluación desde el jugador activo, los signos deben alternarse según el turno.

8.1. Problema de propagación lenta

El mecanismo TD(0) propaga información mediante transiciones individuales.

Si una recompensa aparece después de 80 medias jugadas, pueden necesitarse múltiples actualizaciones para que su influencia alcance las primeras posiciones.

Una forma de acelerar ese proceso consiste en utilizar retornos de varios pasos.

Para una transición de n medias jugadas, y suponiendo que ninguna finaliza la partida antes del último estado, un objetivo consistente con la perspectiva alternada es:

$$\boxed{ G_t^{(n)} = \sum_{k=0}^{n-1} (-\gamma)^k r_{t+k} + (-\gamma)^n V_{\bar\theta}(s_{t+n}) }$$

Aquí cada recompensa r_{t+k} está expresada desde la perspectiva del jugador que actúa en ese paso.

El factor $(-1)^k$ permite trasladar las recompensas sucesivas a la perspectiva del jugador inicial.

En una implementación que utilice una perspectiva fija para un jugador, resulta más sencillo emplear las ecuaciones habituales de retornos descontados sin alternancia de signos, siempre que todas las recompensas y valores se expresen en esa misma perspectiva.

También pueden utilizarse métodos TD(λ) o trazas de elegibilidad, que combinan información de varios horizontes.

9. Aprender el valor de las acciones: Deep Q-Learning

Hasta aquí hemos considerado una red que estima:

$$V_\theta(s)$$

Pero también puede entrenarse una red que evalúe directamente cada par estado-acción:

$$\boxed{ Q_\theta(s,a) }$$

En este caso, el agente intenta aprender el valor de ejecutar la jugada a en la posición s, seguido de decisiones posteriores.

La principal ventaja es que una política puede derivarse directamente de:

$$\boxed{ \pi_\theta(s)= \arg\max_{a\in\mathcal A(s)}Q_\theta(s,a) }$$

sin necesitar evaluar por separado todas las posiciones sucesoras con una red de valor de estados.

9.1. Ecuación de Bellman óptima para Q

En ajedrez, usando valores desde la perspectiva del jugador activo, definimos:

$$Q^*(s,a)= \begin{cases} r(s,a) & \text{si la transición es terminal}\\[4pt] r(s,a)-\gamma\max\limits_{a'\in\mathcal A(s')} Q^*(s',a') & \text{si no es terminal} \end{cases}$$

donde:

$$s'=T(s,a)$$

Por lo tanto:

$$\boxed{ Q^*(s,a)= r(s,a)-\gamma\max_{a'}Q^*(s',a') }$$

para una transición no terminal.

Obsérvese que esta expresión difiere del Q-Learning convencional de un único agente por el signo negativo.

En un agente convencional se maximiza el valor del estado siguiente porque se supone que el mismo agente continúa controlando las decisiones.

En ajedrez, después de la jugada corresponde decidir al oponente.

Si se utilizara incorrectamente:

$$ r+\gamma\max Q(s',a') $$

sin controlar la perspectiva, el agente podría aprender a seleccionar posiciones favorables para su adversario.

9.2. Objetivo de entrenamiento de Deep Q-Learning

Definimos:

$$ y_t= \begin{cases} r_t & \text{si termina la partida}\\[4pt] r_t-\gamma\max\limits_{a'\in\mathcal A(s_{t+1})} Q_{\bar\theta}(s_{t+1},a') & \text{en otro caso} \end{cases} $$

La pérdida puede calcularse como:

$$ \boxed{ L_Q(\theta)= \mathbb E_{(s,a,r,s')\sim D} \left[\left(Q_\theta(s,a)-y \right)^2 \right] } $$

donde D es un conjunto de experiencias anteriores.

En la práctica, puede preferirse una pérdida Huber para reducir la sensibilidad a errores elevados.

9.3. Experience Replay

Las experiencias pueden almacenarse en una memoria:

$$ D=\{(s_t,a_t,r_t,s_{t+1},d_t)\} $$

donde d_t indica si el episodio terminó.

Durante el entrenamiento se seleccionan lotes aleatorios de experiencias almacenadas.

Esto permite:

- Reutilizar experiencias anteriores.
- Reducir la correlación entre ejemplos consecutivos.
- Mejorar la eficiencia de datos.
- Evitar entrenar exclusivamente con las posiciones más recientes.

Sin embargo, almacenar experiencias generadas por políticas anteriores introduce un problema: las distribuciones de datos pueden diferir de las producidas por la política actual.

9.4. Target Network

Otra técnica consiste en utilizar dos redes:

$$ Q_\theta $$

y:

$$ Q_{\bar\theta} $$

La primera es la red que se entrena.

La segunda se mantiene relativamente estable y se utiliza para calcular los objetivos Bellman.

Cada cierto número de pasos, sus parámetros se actualizan mediante:

$$ \bar\theta\leftarrow\theta $$

O bien mediante una actualización gradual:

$$ \boxed{ \bar\theta\leftarrow \tau\theta+(1-\tau)\bar\theta } $$

con:

$$ 0<\tau\ll1 $$

Esto reduce parcialmente el problema de perseguir un objetivo que cambia simultáneamente con los parámetros que se están optimizando.

10. Aprendizaje de la política mediante una red neuronal

La función de valor permite estimar qué tan favorable es una posición. La política representa algo distinto: cómo seleccionar las jugadas.

Una política estocástica se define mediante:

$$ \boxed{ \pi_\phi(a\mid s) = P(a_t=a\mid s_t=s;\phi) } $$

donde ϕ representa los parámetros de una red neuronal.

La política debe satisfacer:

$$ \sum_{a\in\mathcal A(s)}\pi_\phi(a\mid s)=1 $$

y:

$$ \pi_\phi(a\mid s)\geq0 $$

En ajedrez, la red puede producir preferencias para todas las acciones representables, aunque solamente las legales deberán participar en la normalización.

10.1. Obtención de probabilidades mediante Softmax

Supongamos que la red produce una puntuación o logit para cada acción:

$$ \ell_\phi(s,a) $$

Entonces:

$$ \boxed{ \pi_\phi(a\mid s)= \frac{ \exp(\ell_\phi(s,a)/\tau) }{ \sum_{b\in\mathcal A(s)} \exp(\ell_\phi(s,b)/\tau) } } $$

donde $\tau>0$ es un parámetro de temperatura.

Si τ es grande, la distribución tiende a ser más uniforme.

Si τ es pequeña, se concentra alrededor de las jugadas con mayor puntuación.

Para una política determinista de evaluación, puede seleccionarse:

$$ a^*=\arg\max_a\pi_\phi(a\mid s) $$

En entrenamiento, elegir movimientos mediante muestreo favorece la exploración.

10.2. Ejemplo de política neuronal

Supongamos que la red produce las siguientes probabilidades entre cuatro jugadas legales:

Movimiento	Probabilidad
e2e4	0,55
d2d4	0,25
g1f3	0,15
c2c4	0,05

Distribución de una política neuronal

Ejemplo hipotético de probabilidades de selección de movimientos.

00.150.30.450.6e2e4d2d4g1f3c2c4

La política considera que e2e4 es la alternativa preferida, pero no elimina completamente las otras opciones.

Durante el autojuego, esta característica permite experimentar con jugadas distintas de las consideradas mejores hasta ese momento.

El resultado posterior de las partidas permitirá modificar las probabilidades.

Una política neuronal no es necesariamente una representación exacta de los valores de las jugadas. Puede aprender preferencias de acción mediante sus propios objetivos, que no tienen por qué coincidir perfectamente con el orden inducido por la red de valor.

11. Cómo aprender simultáneamente valor y política

Existen distintas posibilidades para combinar ambos componentes.

Una arquitectura especialmente interesante utiliza una red neuronal compartida con dos salidas.

Matemáticamente:

$$ h=f_{\theta_s}(s) $$

$$ \boxed{ (\mathbf p,v)=f_{\theta_s,\phi,\theta_v}(s) } $$

donde:

- θ_s: parámetros compartidos.
- ϕ: parámetros de la cabeza de política.
- θ_v: parámetros de la cabeza de valor.

La ventaja es que la red puede compartir características útiles para ambas tareas.

Por ejemplo, reconocer una estructura de peones favorable puede contribuir tanto a determinar el valor de la posición como a priorizar jugadas que preserven esa estructura.

Pero el aprendizaje simultáneo también puede producir interferencias entre objetivos, porque los gradientes de valor y política no siempre apuntan hacia modificaciones compatibles de la representación interna.

12. Aprendizaje Actor-Critic

Una posibilidad es utilizar una arquitectura Actor-Critic.

El componente Actor aprende la política:

$$ \pi_\phi(a\mid s) $$

El componente Critic estima el valor:

$$ V_\theta(s) $$

La función de valor permite evaluar si las decisiones tomadas por el actor producen resultados mejores o peores que los esperados.

12.1. Ventaja de una acción

Definimos:

$$ \boxed{ A^\pi(s,a)=Q^\pi(s,a)-V^\pi(s) } $$

Esta magnitud se denomina Advantage.

Si:

$$ A^\pi(s,a)>0 $$

la acción tiene un valor superior al promedio esperado bajo la política considerada.

Si:

$$ A^\pi(s,a)<0 $$

su resultado esperado es inferior al valor de referencia.

El Actor puede aprender a aumentar la probabilidad de acciones con ventaja positiva y reducirla para acciones con ventaja negativa.

12.2. Aproximación de la ventaja mediante TD

Utilizando la perspectiva del jugador activo:

$$ \boxed{ \hat A_t= r_t-\gamma V_{\bar\theta}(s_{t+1})-V_\theta(s_t) } $$

para transiciones no terminales.

En estados terminales:

$$ \hat A_t=r_t-V_\theta(s_t) $$

Esto coincide con el error TD definido anteriormente.

La coincidencia no es accidental: el error de predicción de valor puede utilizarse como una señal de mejora relativa para la política.

12.3. Función de pérdida del Actor

El gradiente de política tiene la forma:

$$\boxed{ \nabla_\phi J(\phi) = \mathbb E \left[\nabla_\phi\log\pi_\phi(a_t\mid s_t) A^\pi(s_t,a_t) \right] }$$

Esta expresión corresponde al marco de gradientes de política, bajo los supuestos apropiados sobre la distribución de trayectorias y el retorno.

Para minimizar una pérdida:

$$\boxed{ L_{\text{Actor}}(\phi)= -\mathbb E \left[\log\pi_\phi(a_t\mid s_t) \hat A_t \right] }$$

Cuando la ventaja es positiva, el entrenamiento tiende a aumentar la probabilidad de la acción ejecutada.

Cuando es negativa, tiende a disminuirla.

12.4. Función de pérdida del Critic

El evaluador intenta reducir el error de predicción:

$$\boxed{ L_{\text{Critic}}(\theta)= \frac12 \mathbb E \left[V_\theta(s_t)-y_t \right]^2 }$$

Una función objetivo combinada puede ser:

$$\boxed{ L= L_{\text{Actor}} + \lambda_v L_{\text{Critic}} - \lambda_H H(\pi) }$$

donde:

$$H(\pi)= -\sum_{a\in\mathcal A(s)} \pi(a\mid s)\log\pi(a\mid s)$$

es la entropía de la política.

El término de entropía favorece que la política no se concentre demasiado pronto en unas pocas acciones.

Los coeficientes:

$$\lambda_v,\lambda_H$$

controlan la influencia relativa de los distintos objetivos.

12.5. Ejemplo de aprendizaje de una jugada

Supongamos que el Actor asigna:

$$\pi_\phi(a_1\mid s)=0,20$$

El Critic estima:

$$V_\theta(s)=+0,10$$

Después de ejecutar la jugada, la estimación respaldada por la transición es:

$$ y=+0,45 $$

La ventaja estimada es:

$$ \hat A=0,45-0,10 $$

$$ \boxed{\hat A=+0,35} $$

La jugada fue mejor de lo esperado por el Critic.

El entrenamiento del Actor tenderá a aumentar su probabilidad en situaciones similares.

Simultáneamente, el Critic ajustará su estimación del valor de la posición.

Pero existe una advertencia importante: el objetivo $+0,45$ es una estimación, no una verdad demostrada. Si el Critic sobreestima la posición sucesora, ambos componentes pueden reforzar una jugada objetivamente mala.

13. Iteración de políticas y ecuación de Bellman

La relación entre valor y política puede entenderse mediante Policy Iteration.

El procedimiento ideal contiene dos etapas.

13.1. Evaluación de la política

Dada una política:

$$ \pi_k $$

se calcula su función de valor:

$$ V^{\pi_k}(s) $$

mediante la ecuación de Bellman correspondiente.

En un juego de suma cero, esta evaluación debe considerar las políticas de ambos jugadores.

Si ambos utilizan la misma política expresada desde la perspectiva del jugador activo, el valor describe el resultado esperado del enfrentamiento entre esas políticas, no necesariamente el resultado contra un adversario óptimo.

13.2. Mejora de la política

Una vez conocida la función de valor, puede construirse una política mejorada.

Idealmente:

$$ \boxed{ \pi_{k+1}(s) = \arg\max_{a\in\mathcal A(s)} \left[r(s,a)-\gamma V^{\pi_k}(T(s,a)) \right] } $$

con la convención terminal correspondiente.

El procedimiento conceptual es:



En programación dinámica tabular exacta, existen resultados de convergencia bajo determinados supuestos.

Pero trasladar esas garantías directamente al ajedrez con redes profundas sería incorrecto.

En aprendizaje profundo, la función de valor es aproximada, el muestreo es parcial, los adversarios cambian y las actualizaciones pueden ser inestables.

Por lo tanto, la iteración de políticas proporciona una estructura conceptual, no una garantía automática de convergencia.

14. Autojuego como mecanismo de generación de experiencia

Una característica especialmente atractiva del ajedrez es que el entorno no requiere un simulador probabilístico complejo.

Las reglas son conocidas y permiten generar una cantidad ilimitada de partidas sintéticas, condicionada solamente por los recursos computacionales disponibles.

El agente puede jugar contra sí mismo:

\[\pi_{\phi_k}\quad\text{versus}\quad\pi_{\phi_k} \]

Cada partida produce una trayectoria:

\[\tau_i= (s_0,a_0,s_1,a_1,\ldots,s_T,z) \]

Las trayectorias se almacenan y se utilizan para entrenar la función de valor y la política.

14.1. Ciclo completo de aprendizaje

El entrenamiento puede comenzar con una política inicial completamente aleatoria sobre las jugadas legales.

Sin embargo, ese comienzo presenta dificultades importantes.

Un agente aleatorio no tiene conocimientos básicos de desarrollo, material, seguridad del rey ni coordinación de piezas.

Por lo tanto, las primeras partidas pueden contener secuencias prácticamente irrelevantes para aprender juego estratégico.

Una alternativa es comenzar con una política entrenada mediante partidas existentes, un evaluador convencional o un agente anterior.

Esto no contradice el uso de Bellman: modifica la inicialización y la distribución inicial de experiencias.

14.2. Política exploratoria

Para evitar que el sistema repita exclusivamente las jugadas consideradas mejores, puede utilizarse una política ε-greedy:

$$ \pi_\varepsilon(a\mid s)= \begin{cases} \text{acción óptima estimada} & \text{con probabilidad }1-\varepsilon\\ \text{acción legal aleatoria} & \text{con probabilidad }\varepsilon \end{cases} $$

En caso de empates entre acciones, puede establecerse una regla de desempate aleatoria.

También pueden utilizarse políticas Softmax, muestreo con temperatura o ruido introducido en las preferencias.

La exploración es necesaria porque una política que nunca prueba alternativas puede quedar atrapada en comportamientos mediocres.

Pero explorar durante una partida competitiva también implica ejecutar jugadas que el propio sistema considera inferiores.

Por eso conviene separar el comportamiento de entrenamiento del comportamiento de evaluación.

15. Ejemplo completo de aprendizaje de una política

Consideremos una posición hipotética donde el agente dispone de tres acciones:

$$ a_1,a_2,a_3 $$

Inicialmente, la política asigna:

$$ \pi(a_1\mid s)=0{,}40 $$

$$ \pi(a_2\mid s)=0{,}35 $$

$$ \pi(a_3\mid s)=0{,}25 $$

La función de valor estima:

$$ V_\theta(s)=0{,}10 $$

El agente ejecuta a_2.

Después de esa jugada, el valor de la posición sucesora desde la perspectiva adversaria es:

$$ V_{\bar\theta}(s')=-0,50 $$

Con recompensa inmediata nula y $\gamma=1$:

$$ y=0-(-0,50)=0,50 $$

La ventaja estimada es:

$$ \hat A=0,50-0,10=0,40 $$

Como la ventaja es positiva, el Actor aumenta la preferencia por a_2.

Supongamos que después de numerosas actualizaciones se obtiene:

$$ \pi_{\text{nueva}}(a_1\mid s)=0,25 $$

$$ \pi_{\text{nueva}}(a_2\mid s)=0,60 $$

$$ \pi_{\text{nueva}}(a_3\mid s)=0,15 $$

La política ha modificado su comportamiento:

Antes del aprendizaje

a_1

40%

a_2

35%

a_3

25%

Después del aprendizaje

a_1

25%

a_2

60%

a_3

15%

Los números son ilustrativos y no implican una regla fija de actualización entre ventaja y probabilidades.

El ejemplo muestra la lógica del entrenamiento: una experiencia evaluada favorablemente modifica la distribución de acciones para aumentar la frecuencia de movimientos similares.

Pero el sistema todavía necesita resolver un problema crítico.

¿La jugada a_2 es realmente buena o simplemente el evaluador se equivoca al considerar mala la posición del adversario?

Para responder, necesitamos utilizar información más sólida que una única predicción de la red.

16. Combinación de Bellman con búsqueda de movimientos

Una posibilidad particularmente interesante consiste en combinar la red neuronal con un procedimiento de búsqueda.

La red produce estimaciones de valor y probabilidades iniciales de jugadas, mientras que la búsqueda examina posibles consecuencias y corrige parcialmente esas estimaciones.

Este es el principio general de los sistemas que combinan redes de política y valor con Monte Carlo Tree Search (MCTS), como AlphaZero. Conviene precisar que AlphaZero no es simplemente un sistema de Deep Q-Learning ni utiliza como objetivo principal el residuo Bellman TD(0): su entrenamiento publicado utiliza resultados finales de partidas y distribuciones de acciones obtenidas de la búsqueda.

image

Science+1

16.1. Valor y política como aproximaciones complementarias

La red calcula:

$$\boxed{ (\mathbf p,v)=f_\theta(s) }$$

donde:

$$p_a=\pi_\theta(a\mid s)$$

es la probabilidad previa asignada a cada movimiento y:

$$v=V_\theta(s)$$

es la estimación de valor.

La búsqueda utiliza estas dos salidas para decidir qué ramas explorar.

En una implementación de MCTS puede mantenerse, para cada acción:

- $N(s,a)$: cantidad de visitas.
- $W(s,a)$: suma de valores acumulados.
- $Q(s,a)=W(s,a)/N(s,a)$: valor medio estimado.
- $P(s,a)$: probabilidad previa neuronal.

Una regla frecuente, denominada PUCT, selecciona movimientos utilizando:

$$\boxed{ a^*=\arg\max_{a\in\mathcal A(s)} \left[Q(s,a)+U(s,a) \right] }$$

donde un término de exploración puede ser:

$$\boxed{ U(s,a)= c_{\text{puct}} P(s,a) \frac{ \sqrt{\sum_b N(s,b)} }{ 1+N(s,a) } }$$

El término Q favorece movimientos que han obtenido evaluaciones elevadas.

El término U promueve movimientos con probabilidad previa significativa y pocas visitas.

Esta expresión presupone que Q está representado desde la perspectiva del jugador que selecciona la acción en s.

16.2. Actualización de valores durante la búsqueda

Supongamos que una simulación alcanza una posición hoja:

$$ s_L $$

La red devuelve:

$$ v_L=V_\theta(s_L) $$

Ese valor se propaga hacia los nodos anteriores.

Como los jugadores alternan, debe cambiarse el signo al atravesar cada media jugada.

Para una trayectoria:

$$ s_0\rightarrow s_1\rightarrow \cdots\rightarrow s_L $$

con recompensas intermedias cero y $\gamma=1$, el valor de la hoja desde la perspectiva del jugador activo en s_t es:

$$ \boxed{ v_t=(-1)^{L-t}v_L } $$

La búsqueda puede acumular esos valores en las aristas de su árbol.

Cada vez que una arista (s,a) recibe una nueva observación v:

$$ N(s,a)\leftarrow N(s,a)+1 $$

$$ W(s,a)\leftarrow W(s,a)+v $$

$$ Q(s,a)=\frac{W(s,a)}{N(s,a)} $$

De esta manera, los valores iniciales de la red se combinan con información obtenida mediante exploración de continuaciones.

Esto no garantiza eliminar los errores de evaluación, pero permite contrastar las preferencias iniciales con consecuencias de varios movimientos.

16.3. Política mejorada por búsqueda

Después de realizar K simulaciones, puede obtenerse una nueva distribución de política a partir de los conteos de visitas:

$$ \boxed{ \pi_{\text{MCTS}}(a\mid s)= \frac{ N(s,a)^{1/\tau} }{ \sum_b N(s,b)^{1/\tau} } } $$

Esta distribución puede servir para seleccionar la jugada ejecutada y también como objetivo para entrenar la cabeza de política de la red.

Una función de pérdida inspirada en el procedimiento publicado para AlphaZero es:

$$\boxed{ L(\theta)= (z-V_\theta(s))^2 - \sum_a \pi_{\text{MCTS}}(a\mid s) \log p_\theta(a\mid s) + \lambda\|\theta\|_2^2 }$$

El primer término entrena el valor frente al resultado final.

El segundo entrena la política para aproximar la distribución de visitas producida por MCTS.

El tercero introduce regularización.

De esta forma, la red intenta aprender a reproducir parte del comportamiento de una búsqueda mucho más costosa, permitiendo orientar mejor búsquedas futuras.

image

davidsilver.uk+1

16.4. Diferencia respecto del aprendizaje directo por Bellman

Existen tres estrategias que conviene distinguir:

Estrategia	Objetivo de valor	Obtención de la política
TD / Bellman	Valor del estado sucesor y recompensa	Greedy, Actor-Critic o derivada de Q
Monte Carlo	Resultado final observado	Gradiente de política o supervisión
Red + MCTS	Resultado final, valor de búsqueda o combinación	Distribución de visitas de la búsqueda

Las tres pueden considerarse dentro de una familia más amplia de aprendizaje por refuerzo.

Pero no son equivalentes desde el punto de vista de las funciones de pérdida, la estabilidad o el costo computacional.

17. Una propuesta de aprendizaje profundo guiado explícitamente por Bellman

Podemos construir ahora una arquitectura que utilice directamente la ecuación de Bellman para mejorar tanto el valor como la política.

La idea consiste en utilizar una red de doble salida y entrenarla mediante un procedimiento de autojuego, con objetivos Bellman respaldados por una red de referencia.

17.1. Arquitectura propuesta

Definimos:

$$f_\theta(s)= \left(V_\theta(s),\pi_\theta(\cdot\mid s) \right)$$

Para cada estado s, se generan sus movimientos legales:

$$\mathcal A(s)$$

Después de aplicar cada jugada:

$$ s'_a=T(s,a) $$

calculamos un valor aproximado de acción:

$$ \boxed{ \hat Q(s,a)= \begin{cases} r(s,a) & \text{si la jugada termina la partida}\\[4pt] r(s,a)-\gamma V_{\bar\theta}(s'_a) & \text{en otro caso} \end{cases} } $$

Esta expresión genera una estimación del valor de cada jugada sin realizar una búsqueda extensa.

17.2. Objetivo de valor

Podemos construir un objetivo de optimalidad Bellman:

$$ \boxed{ y_V(s)= \max_{a\in\mathcal A(s)}\hat Q(s,a) } $$

La pérdida de valor será:

$$ \boxed{ L_V= \left[V_\theta(s)-\operatorname{stopgrad}(y_V(s)) \right]^2 } $$

La operación $\operatorname{stopgrad}$ indica que el objetivo se considera constante durante la actualización.

Con ello se evita propagar el gradiente a través del propio cálculo del objetivo Bellman.

Esta estrategia constituye una forma de aproximación de valor óptimo de un paso, en la que se evalúan todos los sucesores legales con la red de referencia.

No debe confundirse con TD sobre la jugada realmente ejecutada: en el objetivo anterior se realiza una maximización sobre las jugadas disponibles.

17.3. Objetivo de política

Para la política puede utilizarse una distribución derivada de los valores de acción:

$$ \boxed{ \pi_B(a\mid s)= \frac{ \exp(\hat Q(s,a)/\tau) }{ \sum_{b\in\mathcal A(s)} \exp(\hat Q(s,b)/\tau) } } $$

La distribución π_B constituye una política mejorada aproximadamente por Bellman.

A temperaturas bajas, se concentra en los movimientos con mayor valor estimado.

A temperaturas altas, conserva mayor diversidad.

La red de política se entrena para aproximar esa distribución mediante entropía cruzada:

$$ \boxed{ L_\pi= -\sum_{a\in\mathcal A(s)} \pi_B(a\mid s) \log\pi_\theta(a\mid s) } $$

La función de pérdida combinada puede escribirse:

$$ \boxed{ L(\theta)= \lambda_V L_V+ \lambda_\pi L_\pi+ \lambda_R\|\theta\|_2^2 } $$

El objetivo es que la cabeza de valor aprenda a estimar el valor Bellman de las posiciones y que la cabeza de política aprenda a seleccionar las acciones que maximizan esos valores.

17.4. Interpretación matemática

La arquitectura intenta aproximar simultáneamente dos relaciones.

Consistencia de valor:

$$ \boxed{ V_\theta(s)\approx \max_{a\in\mathcal A(s)} \left[r(s,a)-\gamma V_{\bar\theta}(T(s,a)) \right] } $$

con tratamiento terminal especial.

Consistencia de política:

$$ \boxed{ \pi_\theta(a\mid s)\approx \operatorname{Softmax} \left(\hat Q(s,a)/\tau \right) } $$

La primera relación busca que el valor de una posición sea coherente con las consecuencias de sus jugadas.

La segunda busca que la probabilidad de seleccionar una jugada sea coherente con su valor estimado.

En términos conceptuales, ambas cabezas aprenden aproximaciones diferentes de una misma estructura de decisión.

17.5. Procedimiento de entrenamiento

Una implementación conceptual puede utilizar el siguiente esquema:

```
initialize network(theta)initialize target_network(theta_target)initialize
replay_buffer()for iteration in range(max_iterations):    # 1. Generar partidas
games = self_play(        network=theta,        exploration=True    )
replay_buffer.add(games)    # 2. Seleccionar posiciones    batch =
replay_buffer.sample_batch()    value_targets = []    policy_targets = []    for
position in batch.positions:        legal_moves = generate_legal_moves(position)
action_values = []        # 3. Evaluar cada sucesor legal        for move in
legal_moves:            child = make_move(position, move)            if
is_terminal(child):                q = terminal_reward_for_mover(child)
else:               q = -gamma * target_network.value(child)
```

El pseudocódigo describe la estructura del método y omite detalles como el tratamiento de posiciones terminales ya presentes en el lote, las máscaras legales, los límites de partidas, la gestión de gradientes y la evaluación de nuevos modelos.

17.6. ¿Qué se está intentando aprender?

Si el entrenamiento funcionara idealmente, la red aprendería una función:

$$ V_\theta(s) $$

que satisficiera aproximadamente:

$$ V_\theta(s)\approx\mathcal T V_\theta(s) $$

donde $\mathcal T$ es el operador de optimalidad Bellman adversarial.

En el caso de recompensas finales y transiciones no terminales:

$$ (\mathcal T V)(s)= \max_{a\in\mathcal A(s)} \left[-\gamma V(T(s,a))\right] $$

con las condiciones terminales correspondientes.

Un punto fijo del operador satisface:

$$ \boxed{ V=\mathcal T V } $$

En un juego finito adecuadamente formulado, la función óptima cumple esa relación.

La red intenta aproximar ese punto fijo mediante gradientes y experiencia parcial, sin enumerar todo el espacio de estados.

Este es el aspecto matemáticamente más atractivo de la propuesta.

También es el origen de varias de sus dificultades.

18. ¿Qué resultado puede obtenerse?

Es importante distinguir entre el objetivo matemático ideal y el resultado que razonablemente puede esperarse de una implementación real.

18.1. Resultado ideal

En el límite ideal, si se dispusiera de:

- Representación completa de los estados.
- Exploración suficiente.
- Aproximación exacta de la función de valor.
- Actualizaciones estables.
- Condiciones terminales correctas.
- Recursos computacionales suficientes.

podría obtenerse una aproximación arbitrariamente precisa al valor óptimo de las posiciones relevantes, bajo los supuestos matemáticos correspondientes.

Para una formulación tabular exacta de un juego finito de suma cero, los valores óptimos determinan estrategias que preservan el resultado de juego perfecto.

En un juego como el ajedrez, si el valor exacto de todas las posiciones fuera conocido, una política podría seleccionar movimientos que preservaran el mejor resultado teórico disponible.

Con utilidad terminal:

$$ z\in\{-1,0,+1\} $$

la función de valor óptimo tendría, para estados deterministas completamente resueltos, valores correspondientes a victoria, tablas o derrota bajo juego perfecto.

En tal escenario:

$$ V^*(s)\in\{-1,0,+1\} $$

cuando el estado incluye toda la información relevante y el juego se resuelve con reglas terminales bien definidas.

Esto no debe confundirse con el valor de una política estocástica aprendida, que puede tomar valores intermedios porque representa expectativas sobre múltiples continuaciones.

18.2. Resultado de una implementación neuronal real

En la práctica, la red podría producir:

1. Una estimación continua del resultado esperado de una posición.
2. Una distribución de probabilidades sobre jugadas legales.
3. Una política capaz de seleccionar movimientos sin búsqueda extensa.
4. Una representación interna de patrones posicionales y tácticos aprendidos.
5. Un evaluador reutilizable dentro de un motor con Minimax, Alfa-Beta o MCTS.

Sin embargo, el aprendizaje no garantiza obtener juego óptimo, ni siquiera una mejora monotónica de la fuerza del agente.

La calidad final dependerá de la arquitectura, la cantidad y diversidad de experiencias, los objetivos de entrenamiento, los adversarios utilizados y el presupuesto computacional.

19. Principales problemas del enfoque

La utilización directa de la ecuación de Bellman con redes neuronales para aprender a jugar ajedrez resulta matemáticamente razonable, pero enfrenta dificultades importantes.

Algunas son problemas generales del aprendizaje por refuerzo profundo; otras aparecen específicamente por la naturaleza adversarial del ajedrez.

19.1. Recompensas extremadamente escasas

El primer problema es que el resultado relevante de una partida aparece normalmente después de numerosas decisiones.

Una partida puede contener decenas o cientos de medias jugadas, pero solamente proporciona una recompensa final:

$$ r_T\in\{-1,0,+1\} $$

Durante el resto de la partida:

$$ r_t=0 $$

Esto dificulta determinar qué decisiones fueron responsables del resultado.

Por ejemplo, una derrota puede deberse a un error cometido 30 movimientos antes del final.

Un mecanismo TD de un paso no identifica directamente esa responsabilidad. La información debe propagarse mediante sucesivas actualizaciones.

El problema recibe el nombre de credit assignment.

Consecuencia: el aprendizaje puede requerir grandes cantidades de partidas para descubrir relaciones que una búsqueda táctica relativamente superficial podría identificar directamente.

Una alternativa sería introducir recompensas intermedias por capturar material, mejorar la seguridad del rey o controlar el centro.

Sin embargo, esto altera el objetivo original. Un agente podría aprender a maximizar capturas de peones aun cuando ello perjudicara sus posibilidades de ganar.

Si se incorpora reward shaping, debe diseñarse cuidadosamente para evitar que el agente optimice una medida auxiliar en lugar del resultado real de la partida.

19.2. El problema del adversario cambiante

En el aprendizaje por refuerzo convencional, suele suponerse que las propiedades del entorno permanecen estables.

En el autojuego, el adversario también está aprendiendo.

Por lo tanto:

\[\pi_{\text{oponente}}^{(k)} \neq \pi_{\text{oponente}}^{(k+1)} \]

La distribución de experiencias cambia a lo largo del entrenamiento.

Una estrategia eficaz contra una versión anterior del agente puede ser completamente ineficaz contra una versión posterior.

Esto produce un problema de no estacionariedad.

Además, una política puede aprender a explotar debilidades específicas de sí misma sin adquirir necesariamente una capacidad general para enfrentar adversarios diferentes.

Por ejemplo, si ambos agentes desarrollan una tendencia a no proteger determinadas piezas, pueden aprender patrones que funcionan dentro de su población de autojuego pero fracasan frente a un adversario externo.

Una solución parcial consiste en entrenar contra una población de versiones anteriores, políticas de distinta fuerza o adversarios de diferentes estilos.

19.3. Errores de autoevaluación que se refuerzan

Este es uno de los problemas más importantes de un enfoque Bellman directo.

Supongamos que la red asigna incorrectamente a una posición:

\[V_\theta(s')=-0,80 \]

El agente que puede conducir a esa posición calcula:

\[Q(s,a)=-V_\theta(s')=+0,80 \]

y considera que la jugada es excelente.

La cabeza de política incrementa su probabilidad.

El agente comienza a ejecutar esa jugada con mayor frecuencia.

Si los ejemplos posteriores tampoco revelan adecuadamente el error, el sistema puede reforzar una creencia equivocada.

Se produce un ciclo:

```
Evaluación equivocada
        ↓
Preferencia por una mala jugada
        ↓
Más experiencias asociadas
        ↓
Objetivos Bellman contaminados
        ↓
Refuerzo de la evaluación equivocada
```

El problema no consiste en que Bellman sea matemáticamente incorrecto.

Consiste en que las aproximaciones neuronales utilizadas en ambos lados de la ecuación pueden contener errores correlacionados.

Las redes de referencia, la búsqueda adicional y los resultados finales ayudan a mitigar este problema, pero no lo eliminan de manera general.

19.4. Inestabilidad por aproximación de funciones y bootstrapping

En Deep Q-Learning aparece una dificultad conocida como la tríada mortal del aprendizaje por refuerzo:

1. Aproximación de funciones.
2. Aprendizaje fuera de política (off-policy).
3. Objetivos construidos mediante bootstrapping.

Estas tres características pueden generar divergencia o inestabilidad.

El problema surge porque los valores objetivo dependen de estimaciones realizadas por una red cuyos parámetros también están cambiando.

En particular:

$$ y=r-\gamma\max_{a'}Q_{\bar\theta}(s',a') $$

puede variar considerablemente cuando se modifican los pesos de la red.

Además, el operador de maximización puede amplificar errores de estimación.

Es importante distinguir que la presencia de bootstrapping y aproximación neuronal no implica automáticamente divergencia; el riesgo depende del algoritmo y de las condiciones de entrenamiento.

Las redes objetivo, el replay buffer y las variantes de Double Q-Learning pueden mejorar la estabilidad, pero no proporcionan una garantía general de convergencia en el ajedrez profundo.

19.5. Gran espacio de estados y escasa cobertura

El espacio de posiciones de ajedrez es extraordinariamente grande.

La red solamente observa una fracción diminuta de ese espacio durante el entrenamiento.

Por lo tanto, debe generalizar hacia posiciones que nunca vio.

Esto puede funcionar para patrones frecuentes, como el desarrollo de piezas o determinadas estructuras de peones.

Sin embargo, en ajedrez existen posiciones muy poco frecuentes donde pequeños detalles cambian radicalmente el resultado.

Un peón desplazado una sola casilla, una posibilidad de captura al paso o la pérdida de un derecho de enroque pueden modificar completamente el valor estratégico.

La generalización neuronal no garantiza conservar esas diferencias críticas.

19.6. La evaluación estratégica no sustituye necesariamente a la búsqueda táctica

Una red puede aprender que una posición parece favorable, pero eso no significa que haya identificado todas las combinaciones forzadas.

Por ejemplo, una combinación de sacrificio puede requerir diez medias jugadas para mostrar sus consecuencias.

Una evaluación directa:

\[V_\theta(s) \]

debe inferir esas consecuencias sin explorarlas explícitamente.

Esto puede exigir una capacidad de representación y un volumen de entrenamiento considerablemente mayores que los necesarios para reconocer patrones posicionales generales.

La búsqueda, en cambio, puede verificar concretamente una secuencia de movimientos.

Por ello, una arquitectura puramente neuronal sin búsqueda tiene una limitación importante cuando se enfrenta a problemas tácticos que requieren razonamiento combinatorio preciso.

La existencia de sistemas eficaces que combinan redes de valor y política con búsqueda ofrece evidencia de la utilidad de esta complementariedad, pero no demuestra que toda arquitectura híbrida supere a cualquier sistema neuronal directo.

image

Nature+1

19.7. El problema de las tablas

El ajedrez presenta una dificultad adicional: muchas posiciones permiten obtener tablas mediante juego correcto.

La recompensa:

$$ z=0 $$

es idéntica tanto para una posición completamente equilibrada como para una posición materialmente desfavorable en la que existe una defensa forzada que preserva las tablas.

Desde el punto de vista del resultado teórico, ambas pueden tener el mismo valor.

Pero desde una perspectiva práctica, las posiciones pueden ser muy diferentes.

Además, si el autojuego contiene muchas partidas que terminan en tablas, la red puede recibir una gran cantidad de objetivos cercanos a cero.

Esto puede dificultar la discriminación de ventajas pequeñas y reducir la presión de aprendizaje para encontrar estrategias ganadoras.

Una cabeza WDL permite distinguir probabilidades de victoria, tablas y derrota, pero no resuelve automáticamente la falta de información sobre la calidad táctica de una posición.

19.8. El problema del factor de descuento

En muchas aplicaciones de aprendizaje por refuerzo se utiliza:

$$ 0<\gamma<1 $$

porque ese descuento proporciona propiedades matemáticas convenientes.

Para operadores Bellman estándar, el descuento puede hacer que el operador sea contractivo en la norma supremo:

$$ \|\mathcal TV-\mathcal TW\|_\infty \leq \gamma\|V-W\|_\infty $$

En particular, si:

$$ \gamma<1 $$

la contracción facilita ciertos resultados de existencia, unicidad y convergencia en formulaciones tabulares exactas.

La propiedad puede extenderse al operador adversarial de máximo y mínimo bajo supuestos adecuados.

Pero en ajedrez suele interesar:

$$ \gamma=1 $$

porque el objetivo es ganar, independientemente del tiempo que requiera conseguirlo.

Con $\gamma=1$, ya no se dispone de la contracción estricta basada únicamente en el descuento.

En juegos episódicos finitos o adecuadamente terminantes pueden existir otros argumentos que garanticen una solución, pero no deben confundirse con la contracción descontada.

En particular, si se permiten ciclos sin condiciones terminales adecuadas, la formulación puede resultar incompleta.

Este es un compromiso matemático real: elegir un descuento puede facilitar el tratamiento numérico, pero introduce preferencias temporales que no estaban presentes en el objetivo original.

19.9. Diferencia entre consistencia Bellman y corrección estratégica

Este problema merece especial atención.

Supongamos que una red obtiene un residuo Bellman pequeño:

$$ \epsilon_B(s)= V_\theta(s)-(\mathcal TV_\theta)(s) $$

y que:

$$ |\epsilon_B(s)|\approx0 $$

para las posiciones observadas durante el entrenamiento.

Esto indica consistencia local con las predicciones utilizadas.

Pero no demuestra que:

$$ V_\theta(s)\approx V^*(s) $$

para todas las posiciones relevantes.

Con $\gamma<1$, en un modelo tabular exacto y cuando el residuo está acotado uniformemente sobre todo el espacio de estados, puede obtenerse una relación del tipo:

$$ \boxed{ \|V-V^*\|_\infty \leq \frac{ \|V-\mathcal TV\|_\infty }{ 1-\gamma } } $$

Esta cota requiere las hipótesis de contracción y control uniforme del residuo.

No puede aplicarse directamente al error medio observado en un pequeño conjunto de posiciones de entrenamiento.

Cuando $\gamma=1$, la cota anterior deja de ser útil, porque el denominador se anula.

Por lo tanto, un error Bellman pequeño medido sobre partidas de autojuego no certifica por sí mismo que la política aprendida juegue bien al ajedrez.

19.10. Coste computacional de evaluar todas las acciones

En la propuesta de Bellman de un paso, para obtener:

$$ y_V(s)=\max_a\hat Q(s,a) $$

es necesario evaluar los estados sucesores de todas las jugadas legales.

Si una posición dispone de b movimientos:

$$ N_{\text{eval}}\approx b $$

por cada posición de entrenamiento.

Si se procesan N posiciones:

$$ N_{\text{eval,total}}\approx Nb $$

Para:

$$ N=10^7,\qquad b=35 $$

se necesitarían aproximadamente:

$$ 3,5\times10^8 $$

evaluaciones de posiciones sucesoras.

Esto no incluye el costo de generar partidas, calcular gradientes, almacenar experiencias ni evaluar nuevos modelos.

Por lo tanto, aun sin búsqueda profunda, el entrenamiento puede ser costoso.

Una política que evalúa directamente todos los movimientos en una única pasada puede reducir parte de ese costo, pero también cambia las características del modelo y de sus objetivos de aprendizaje.

20. Comparación con otros enfoques de aprendizaje y búsqueda

Enfoque	**Principal ventaja**	**Principal dificultad**
Bellman + red de valor	Aprende una estimación reutilizable de posiciones	Objetivos autoestimados y propagación de errores
Deep Q-Learning	Aprende valores directamente para acciones	Estabilidad, espacio de acciones y cambio de perspectiva
Actor-Critic	Aprende conjuntamente política y valor	Varianza, errores del Critic y no estacionariedad
Monte Carlo desde partidas	Utiliza resultados finales observados	Recompensas escasas y alta varianza
Red + MCTS	Combina aprendizaje y exploración de consecuencias	Alto costo de búsqueda y entrenamiento
Alfa-Beta + evaluador neuronal	Verifica tácticamente consecuencias y utiliza un evaluador aprendido	Necesita una búsqueda eficiente y una evaluación adecuada

No existe una superioridad universal entre estas alternativas.

El resultado depende de los recursos disponibles, la arquitectura de la red, las características del hardware, el costo de evaluación y los objetivos de investigación.

Un aspecto especialmente interesante es que la red Bellman puede utilizarse como evaluador de otro algoritmo de búsqueda.

No resulta obligatorio elegir entre aprender una función de valor y ejecutar Minimax.

Por ejemplo:

\[V_\theta(s) \]

puede actuar como evaluación de hoja de un árbol Alfa-Beta.

La búsqueda puede corregir algunas decisiones tácticamente equivocadas que la red tomaría de forma aislada.

También es posible emplear los valores Bellman para entrenar una política que ordene las jugadas antes de la búsqueda.

Esta combinación permite separar dos problemas:

- Aprender patrones generales de valoración.
- Verificar consecuencias concretas de movimientos.

21. Propuesta de experimento para evaluar el enfoque

Para investigar seriamente la viabilidad de utilizar Bellman en ajedrez, convendría diseñar un experimento con controles que permitan distinguir la mejora real del aprendizaje de los efectos provocados por una búsqueda adicional.

21.1. Modelos comparados

Una comparación razonable utilizaría cuatro agentes:

Agente	Arquitectura
A	Red de valor entrenada mediante Bellman TD
B	Red de política y valor entrenada mediante Actor-Critic
C	Red de política y valor entrenada con resultados finales y búsqueda
D	Alfa-Beta con evaluador clásico de referencia

También convendría incluir un agente aleatorio legal como control inicial.

Para una comparación equilibrada, los agentes neuronales deberían utilizar capacidades semejantes y, cuando sea posible, presupuestos de entrenamiento comparables.

21.2. Métricas

No sería suficiente medir solamente la pérdida de entrenamiento.

Se deberían registrar:

Error de valor:

$$ MSE= \frac1N \sum_{i=1}^N (V_\theta(s_i)-z_i)^2 $$

Error Bellman:

$$ BE= \frac1N \sum_{i=1}^N \left[V_\theta(s_i)-(\mathcal TV_{\bar\theta})(s_i) \right]^2 $$

Calidad de política: porcentaje de coincidencia con jugadas de referencia, cuando existan, o medidas de concordancia entre distribuciones de política.

Rendimiento competitivo: resultados de partidas contra adversarios fijos y variados, con colores alternados y controles de tiempo definidos.

Eficiencia computacional: tiempo de entrenamiento, inferencias por segundo, memoria utilizada y costo por jugada.

Generalización: desempeño sobre posiciones no observadas durante el entrenamiento y problemas tácticos seleccionados.

Los resultados de autojuego contra la misma versión del agente no serían suficientes para demostrar una mejora general de fuerza.

21.3. Un experimento decisivo

Una comparación particularmente informativa consistiría en evaluar el mismo modelo neuronal bajo tres condiciones:

1. Selección directa de la jugada con la política aprendida.
2. Evaluación Bellman de un paso sobre todos los sucesores legales.
3. Búsqueda Alfa-Beta de profundidad fija utilizando la misma red de valor.

Así podría determinarse hasta qué punto la capacidad adquirida reside en la red y hasta qué punto depende de la búsqueda.

También permitiría medir la relación entre el costo de realizar más inferencias neuronales y el beneficio obtenido por explorar más profundamente.

22. Conclusión

La ecuación de Bellman ofrece una formulación rigurosa para describir cómo debería relacionarse el valor de una posición de ajedrez con los valores de las posiciones que pueden alcanzarse mediante sus jugadas legales.

Desde la perspectiva del jugador activo, su formulación adversarial fundamental es:

$$ \boxed{ V^*(s)= \max_{a\in\mathcal A(s)} \left[-\gamma V^*(T(s,a)) \right] } $$

para movimientos no terminales con recompensa inmediata nula, utilizando las condiciones de frontera apropiadas para los resultados finales.

El aprendizaje profundo propone aproximar esa función mediante:

$$ \boxed{ V_\theta(s)\approx V^*(s) } $$

y aprender una política:

$$\boxed{ \pi_\phi(a\mid s) }$$

capaz de seleccionar las jugadas con mayor valor estimado.

El entrenamiento ajusta los parámetros neuronales utilizando errores de predicción, relaciones Bellman, resultados finales de partidas y eventualmente información producida por una búsqueda.

En términos conceptuales, el aprendizaje intenta trasladar parte del trabajo de exploración hacia una función de evaluación generalizable: en lugar de resolver nuevamente desde cero cada posición, la red aprende regularidades a partir de experiencias anteriores.

Sin embargo, esta transferencia plantea una dificultad fundamental.

La ecuación de Bellman proporciona una condición de consistencia entre valores, pero no proporciona por sí misma valores correctos ni garantiza que una red neuronal entrenada con experiencias parciales converja al juego óptimo.

En ajedrez, los principales obstáculos son la escasez de recompensas, los enormes espacios de estados y acciones, la no estacionariedad del autojuego, los errores que se retroalimentan mediante bootstrapping y la necesidad de detectar secuencias tácticas precisas.

Por estas razones, una arquitectura neuronal que aplique Bellman de manera directa puede constituir una línea de investigación valiosa, aunque no deba suponerse que necesariamente superará a los métodos basados en búsqueda.

La recomendación metodológica más sólida es separar experimentalmente tres capacidades: aprender el valor de las posiciones, aprender una política de movimientos y mejorar las decisiones mediante búsqueda.

Esa separación permitiría determinar qué conocimiento adquirió realmente la red neuronal, cuáles son las limitaciones de la ecuación de Bellman aproximada y cuánto contribuye cada componente a la fuerza ajedrecística del sistema.

Referencias bibliográficas

1. Bellman, R. (1957). Dynamic Programming. Princeton University Press.
2. Sutton, R. S., & Barto, A. G. (2018). Reinforcement Learning: An Introduction. MIT Press. Especialmente los capítulos sobre programación dinámica, aprendizaje por diferencias temporales, aproximación de funciones y métodos de gradiente de política.
3. Watkins, C. J. C. H., & Dayan, P. (1992). Q-learning. Machine Learning, 8, 279–292.
4. Mnih, V., et al. (2015). Human-level control through deep reinforcement learning. Nature, 518, 529–533.
5. Silver, D., et al. (2018). A general reinforcement learning algorithm that masters chess, shogi, and Go through self-play. Science, 362, 1140–1144.
6. Schrittwieser, J., et al. (2020). Mastering Atari, Go, Chess and Shogi by Planning with a Learned Model. Nature, 588, 604–609.
7. van Hasselt, H., Guez, A., & Silver, D. (2016). Deep Reinforcement Learning with Double Q-learning. Proceedings of AAAI.
8. Schulman, J., et al. (2017). Proximal Policy Optimization Algorithms. arXiv:1707.06347.

9. Acquisition of chess knowledge in AlphaZero. Estudio sobre el conocimiento ajedrecístico adquirido mediante redes de valor y política entrenadas con autojuego.

Microchess (1976): arquitectura, algoritmo de búsqueda y evaluación estratégica

Autor: Peter Jennings
Año: 1976
Plataforma original: MOS Technology KIM-1
Procesador: MOS 6502, de 8 bits
Lenguaje: Ensamblador 6502
Distribución original: Micro-Ware Ltd., Toronto, Canadá
Categoría: Motor de ajedrez basado en heurísticas, generación secuencial de movimientos y búsqueda táctica selectiva.

---

1. Introducción y contexto histórico

Microchess fue desarrollado por Peter Jennings en 1976 y es considerado uno de los primeros programas comerciales de ajedrez para microcomputadoras personales.

Su primera versión funcionaba en el computador KIM-1, basado en el microprocesador MOS Technology 6502, con aproximadamente 1 KB de memoria disponible.

El código original ocupaba **924 bytes**, mientras que el programa y sus datos utilizaban aproximadamente **1.118 de los 1.152 bytes de RAM disponibles**, según la documentación comercial de la época.

El programa podía distribuirse como un listado hexadecimal que el usuario introducía manualmente mediante el teclado del KIM-1. Posteriormente se ofreció su distribución en casete y se desarrollaron adaptaciones para otras plataformas.

La importancia histórica de Microchess no reside en su fuerza ajedrecística, considerablemente inferior a la de los motores modernos, sino en haber demostrado que un computador extremadamente limitado podía generar movimientos, reconocer amenazas, analizar intercambios y seleccionar jugadas mediante una evaluación estratégica.

Su arquitectura constituye un ejemplo temprano de optimización algorítmica bajo restricciones severas de memoria.

2. Arquitectura del computador original

El KIM-1 era una computadora de placa única introducida en 1976.

Sus principales características eran:

Característica	Descripción
Procesador	MOS Technology 6502

Característica	Descripción
Arquitectura	8 bits
Frecuencia típica	Aproximadamente 1 MHz
Memoria RAM	1.152 bytes disponibles en la configuración original
Interfaz	Teclado hexadecimal
Visualización	Seis dígitos LED
Almacenamiento	Casete, opcional
Sistema operativo	Monitor residente en ROM

La limitación fundamental era la memoria.

No resultaba práctico almacenar múltiples tableros, árboles completos de búsqueda ni estructuras auxiliares grandes.

Jennings resolvió el problema mediante una representación compacta de las piezas, generación secuencial de movimientos y un mecanismo de aplicación y restauración de jugadas.

El programa privilegiaba ahorrar memoria, incluso cuando ello significaba repetir cálculos.

3. Representación del tablero

3.1. Codificación de casillas

Microchess utilizaba una representación numérica compacta de las 64 casillas, compatible con una numeración octal.

Una casilla podía expresarse mediante:

$$s = 8r + f$$

donde:

- r es la fila, entre 0 y 7.
- f es la columna, entre 0 y 7.
- s es la posición codificada, entre 0 y 63.

La representación permitía utilizar desplazamientos numéricos para calcular movimientos.

Pieza o desplazamiento	Incrementos
Horizontal	± 1
Vertical	± 8
Diagonal	$\pm 7, \pm 9$
Caballo	$\pm 6, \pm 10, \pm 15, \pm 17$

Estos desplazamientos necesitaban comprobaciones de los límites del tablero para impedir movimientos que atravesaran incorrectamente sus bordes.

3.2. Representación centrada en piezas

Una decisión fundamental fue mantener información sobre las posiciones de las piezas en lugar de conservar múltiples matrices completas de 64 casillas.

Conceptualmente, puede representarse mediante:

$$L = (s_0, s_1, \ldots, s_{31})$$

Cada entrada indica la casilla ocupada por una pieza determinada.

Esta descripción es una abstracción del diseño histórico, no una reconstrucción exacta de todas sus tablas de memoria.

La identificación de las piezas podía depender de su posición en las tablas, mientras que la ocupación de una casilla se determinaba examinando las posiciones registradas.

Esto reducía el almacenamiento necesario, a costa de aumentar el tiempo de consulta.

3.3. Aplicación y restauración de movimientos

Microchess utilizaba una pila de movimientos que permitía explorar posiciones hipotéticas y regresar a la posición original.

Para un movimiento:

$$m = (p, a, b, c)$$

donde p identifica la pieza, a su origen, b su destino y c una eventual captura, el procedimiento conceptual era:

1. Registrar la información necesaria para restaurar el estado.
2. Desplazar la pieza.
3. Registrar temporalmente la captura, si existía.
4. Analizar la nueva posición.
5. Deshacer el movimiento y recuperar las piezas afectadas.

El mecanismo evitaba copiar el tablero completo para cada variante examinada.

4. Organización general del programa

La documentación original divide Microchess en tres grandes unidades funcionales.

4.1. Control y entrada/salida

Responsable de inicializar el juego, interpretar comandos, recibir las jugadas del adversario y presentar las respuestas mediante los dispositivos del KIM-1.

4.2. Generación de movimientos y recopilación de información

Responsable de determinar los movimientos disponibles y obtener información acerca de:

- Movilidad de las piezas.
- Amenazas propias y adversarias.
- Posibles capturas.
- Continuaciones de una jugada.
- Respuestas del oponente.
- Intercambios sucesivos de material.

4.3. Análisis estratégico

Responsable de combinar las mediciones anteriores mediante una función matemática y asignar una puntuación a cada jugada candidata.

El programa seleccionaba el movimiento con la mayor valoración estratégica.

5. Rutinas fundamentales: GNM, JANUS y STATE

5.1. GNM: generación de movimientos

La rutina **GNM** (*Generate Moves*) era uno de los elementos centrales del programa.

Su funcionamiento consistía en generar movimientos mediante tres niveles de iteración.

NEWP seleccionaba sucesivamente las piezas, desde los peones hasta el rey.

NEX recorría las direcciones potencialmente disponibles para cada pieza, utilizando tablas como `MOVEX` y el puntero de dirección `MOVEN`.

El tercer nivel determinaba las distancias posibles en cada dirección, hasta encontrar un obstáculo, una captura o un límite del tablero.

GNM no necesitaba conservar una lista completa de jugadas. Podía entregar cada movimiento al mecanismo de análisis y continuar posteriormente con el siguiente.

La generación también se reutilizaba para reconocer ataques, calcular movilidad y estudiar intercambios.

5.2. JANUS: coordinador del análisis

JANUS era la rutina encargada de interpretar el estado de la búsqueda y decidir qué operación ejecutar a continuación.

Después de generar un movimiento, GNM transfería el control a JANUS.

Según el estado vigente, JANUS podía:

- Evaluar un movimiento candidato.
- Examinar continuaciones.
- Calcular respuestas del adversario.
- Analizar capturas sucesivas.
- Comprobar amenazas al rey.
- Regresar a una etapa anterior.

JANUS podía modificar el estado y volver a invocar GNM, proporcionando una forma compacta de exploración recursiva.

5.3. STATE: máquina de estados

La variable `STATE` determinaba qué información debía recopilarse y cómo debía interpretarse cada movimiento generado.

Entre los estados documentados se encuentran:

Estado	Función
Inicial (`00`)	Recopilar estadísticas de referencia de la posición
`04`	Examinar movimientos candidatos
Continuación	Examinar oportunidades de movimientos propios posteriores
Respuesta	Analizar las respuestas del adversario
`FF`, `FE`, `FD`, `FC`	Etapas del análisis de intercambios
`F9`	Comprobación de ilegalidad relacionada con amenazas al rey

Los valores simbólicos y numéricos reflejan la documentación histórica; la correspondencia completa de cada transición requiere el análisis del listado ensamblador original.

La combinación GNM/JANUS/STATE permitía reutilizar una parte importante del código para operaciones distintas.

6. Función de evaluación estratégica

Microchess no utilizaba redes neuronales ni una función convencional basada exclusivamente en la diferencia de material.

Su evaluación combinaba movilidad, amenazas, capturas, intercambios y algunas consideraciones posicionales.

El manual original presenta una función denominada `STRATGY`.

6.1. Variables de evaluación

Las variables se organizaban alrededor de tres perspectivas:

Prefijo	Significado
`P`	Mediciones de la posición anterior al movimiento
`W`	Mediciones de las oportunidades propias
`B`	Mediciones de las amenazas y respuestas adversarias

Las principales magnitudes eran:

MOB — movilidad

Representaba el número de movimientos disponibles.

La documentación indicaba que los movimientos de dama se contaban dos veces.

Conceptualmente:

$$M(P,c) = \sum_{m \in \mathcal{M}(P,c)} w(m)$$

con:

$$w(m) = \begin{cases} 2 & \text{si mueve una dama} \\ 1 & \text{en otro caso} \end{cases}$$

MAXC — máxima captura

Representaba el valor de la pieza más importante que podía capturarse.

$$MAXC(P,c) = \max_{m \in C(P,c)} v(m)$$

donde $v(m)$ es el valor de la pieza capturable.

CC — cuenta de capturas o amenazas

Representaba la suma del valor de las piezas adversarias amenazadas.

$$CC(P,c) = \sum_{p \in A(P,c)} v(p)$$

Estas variables permitían diferenciar una amenaza individual importante del conjunto de oportunidades ofensivas.

6.2. Valoración de las piezas

Microchess utilizaba valores relativos del material para reconocer la importancia de las capturas y los intercambios.

El material histórico consultado no permite fijar con suficiente certeza una tabla completa y exacta de los valores internos utilizados por todas las versiones del programa.

Para explicar el mecanismo puede utilizarse una escala convencional ilustrativa:

Pieza	Valor relativo ilustrativo
Peón	1
Caballo	3
Alfil	3
Torre	5
Dama	9

Estos valores son ilustrativos y no deben atribuirse como constantes exactas del Microchess original.

La característica central de su heurística era que el valor de las piezas intervenía en las oportunidades de captura y en los riesgos de intercambio, no solamente en el material presente en el tablero.

6.3. WCAP y BCAP: intercambios tácticos

Las variables `WCAP0`, `WCAP1`, `WCAP2` y `BCAP0`, `BCAP1`, `BCAP2` reflejaban información obtenida de distintas etapas del análisis de capturas.

El programa examinaba intercambios sucesivos con límites definidos, que podían alcanzar hasta tres capturas por bando.

El objetivo era distinguir una ganancia material aparente de una captura que podía provocar pérdidas posteriores.

Por ejemplo, si una torre captura un alfil, pero después resulta capturada por un peón:

$$G = 3 - 5 = -2$$

La secuencia resulta desfavorable pese a la captura inicial.

Esta expresión es un ejemplo de intercambio neto moderno; el Microchess histórico utilizaba sus mediciones de captura en la función estratégica publicada.

6.4. Ecuación histórica STRATGY

La función documentada puede expresarse como:

$$\begin{aligned} V = {} & 4WCAP0 + 1.25WCAP1 \\ & + 0.75(WMAXC + WCC) \\ & + 0.25(WMOB + WCAP2) \\ & - 2.50BMAXC - 2BCC \\ & - 1.25BCAP1 - 0.50BMAXC \\ & - 0.25(PMAXC + PCC + PMOB \\ & \quad + BCAP0 + BCAP2 + BMOB) \end{aligned}$$

En la transcripción del manual utilizada como referencia, `BMAXC` figura dos veces, con coeficientes diferentes. Se conserva esa particularidad sin atribuirla automáticamente a un error tipográfico.

Reuniendo ambos términos y multiplicando por cuatro se obtiene:

$$\begin{aligned} 4V = {} & 16WCAP0 + 5WCAP1 \\ & + 3(WMAXC + WCC) \\ & + WMOB + WCAP2 \\ & - 12BMAXC - 8BCC - 5BCAP1 \\ & - (PMAXC + PCC + PMOB \\ & \quad + BCAP0 + BCAP2 + BMOB) \end{aligned}$$

Esta transformación permite expresar los coeficientes mediante operaciones enteras.

Es consistente con las restricciones del microprocesador 6502, aunque no demuestra por sí sola que el programa ensamblador implementara literalmente esta transformación.

6.5. Heurísticas adicionales

Además de la ecuación, la documentación describe bonificaciones por:

- Avance hacia casillas centrales.
- Desarrollo de piezas desde sus posiciones iniciales.
- Situaciones estratégicamente favorables.

Una bonificación documentada era:

$$V \leftarrow V + 2$$

También se utilizaban valores especiales para penalizar movimientos ilegales y premiar posiciones de jaque mate.

Estos valores estaban vinculados con la codificación compacta del sistema original y no deben interpretarse directamente como centipeones modernos.

7. Política de selección de jugadas

Microchess utilizaba una política de selección basada en la evaluación de candidatos.

Para una posición P, la decisión puede representarse como:

$$m^* = \underset{m \in \mathcal{M}(P)}{\operatorname{argmax}} S(P, m)$$

donde:

- P: posición actual.
- $\mathcal{M}(P)$: conjunto de movimientos considerados.
- $S(P, m)$: valoración estratégica de la jugada.
- m^*: movimiento seleccionado.

El procesamiento conceptual era:

1. Obtener estadísticas de referencia de la posición.
2. Generar un movimiento candidato mediante GNM.
3. Aplicarlo temporalmente.
4. Calcular movilidad y oportunidades propias.
5. Examinar amenazas y respuestas del adversario.
6. Investigar intercambios sucesivos de piezas.
7. Evaluar la jugada mediante STRATGY.
8. Restaurar la posición anterior.
9. Conservar el mejor movimiento encontrado.
10. Continuar con el siguiente candidato.

Este procedimiento permitía seleccionar jugadas sin mantener un árbol completo en memoria.

A diferencia de un motor minimax convencional, Microchess utilizaba las continuaciones principalmente para reunir información estratégica y táctica con la que valorar los movimientos.

No corresponde describirlo simplemente como un minimax moderno de profundidad fija.

8. Profundidad de análisis y búsqueda selectiva

El programa original dedicaba esfuerzo especial al examen de capturas y amenazas.

Su exploración no era uniforme: determinados intercambios podían examinarse con mayor profundidad que otros movimientos.

Esta política anticipa conceptualmente la búsqueda de quiescence utilizada en motores posteriores, aunque no constituye una implementación histórica idéntica de dicho algoritmo.

Una formulación moderna de quiescence sería:

$$Q(P, d) = \max\left(E(P), \max_{m \in C(P)} [-Q(T(P, m), d - 1)]\right)$$

donde:

- $E(P)$ es una evaluación estática.
- $C(P)$ reúne capturas tácticamente relevantes.
- $T(P, m)$ es la posición resultante.
- d limita el análisis.

La implementación original de Jennings recopilaba magnitudes de intercambio para incorporarlas a STRATGY; no se ha demostrado que utilizara esta recurrencia.

9. Reglas del ajedrez y limitaciones históricas

Para ahorrar memoria, la primera versión básica de Microchess omitía algunas reglas del ajedrez:

- Enroque.
- Captura al paso.
- Promoción de peones.

El manual también reconocía limitaciones en el tratamiento de determinadas situaciones de jaque y en la evaluación estratégica de estructuras posicionales.

Por lo tanto, Microchess no constituía una implementación completa de las reglas modernas.

Es importante distinguir esta limitación histórica de cualquier adaptación actual que utilice bibliotecas ajedrecísticas completas para validar los movimientos.

10. Características principales

Característica	**Microchess original**
Autor	Peter Jennings
Año	1976

Característica	Microchess original
Computador	MOS KIM-1
CPU	MOS 6502
Memoria	Aproximadamente 1 KB
Representación	Compacta, centrada en piezas
Generación	Secuencial mediante GNM
Control	Máquina de estados JANUS/STATE
Evaluación	Heurística lineal STRATGY
Criterios	Movilidad, amenazas, capturas, desarrollo
Análisis táctico	Intercambios selectivos
Selección	Mejor puntuación entre candidatos
Aprendizaje automático	No
Red neuronal	No
Limitaciones	Reglas incompletas y horizonte táctico reducido

11. Ventajas

Eficiencia de memoria. Su principal innovación fue realizar análisis ajedrecístico con una cantidad extraordinariamente pequeña de memoria.

Reutilización algorítmica. La rutina GNM servía para generar movimientos y recopilar diferentes tipos de información estratégica.

Arquitectura compacta. JANUS/STATE permitía compartir código entre distintas etapas del análisis.

Evaluación interpretable. La función STRATGY utilizaba características explícitas y coeficientes conocidos, lo que facilitaba comprender y modificar su comportamiento.

Análisis selectivo. La exploración de capturas permitía dedicar trabajo adicional a situaciones tácticamente importantes sin recorrer uniformemente todo el árbol.

Adaptabilidad. El programa estaba documentado para permitir modificaciones y experimentos con nuevas heurísticas.

12. Desventajas

Fuerza de juego limitada. Su capacidad táctica y estratégica era reducida frente a motores que podían explorar árboles de búsqueda más profundos.

Horizonte de análisis restringido. Las secuencias tácticas largas podían escapar a su evaluación.

Dependencia de coeficientes manuales. Los pesos no eran producto de entrenamiento estadístico ni de optimización exhaustiva.

Evaluación posicional incompleta. No incorporaba modelos sofisticados de estructura de peones, seguridad del rey, finales o coordinación de piezas.

Reglas incompletas. La primera versión no implementaba todas las reglas del ajedrez.

Costo de generación repetida. Ahorrar memoria obligaba a repetir operaciones que otros motores podían acelerar almacenando estructuras auxiliares.

Arquitectura altamente especializada. Su diseño estaba estrechamente condicionado por el hardware original, lo que dificulta trasladarlo literalmente a arquitecturas modernas sin modificar sus compromisos.

13. Aplicación al proyecto Akkad

La incorporación de Microchess al proyecto Akkad tiene dos objetivos distintos:

1. Recuperar los principios de su arquitectura algorítmica.
2. Permitir comparaciones experimentales con motores y evaluadores más modernos.

En una adaptación Python resulta razonable conservar:

- La generación secuencial conceptual de GNM.
- La organización del análisis mediante JANUS/STATE.
- Las características MOB, MAXC y CC.
- Las variables de intercambio WCAP y BCAP.
- La función estratégica STRATGY.
- La selección de jugadas por valoración de candidatos.

Al mismo tiempo, pueden modernizarse aspectos que respondían exclusivamente a la escasez de memoria del KIM-1.

En particular, la implementación de Akkad puede utilizar `python-chess` para validar legalidad, enroques, capturas al paso, promociones y posiciones terminales.

Los parámetros `depth` y `qdepth` permiten controlar la profundidad de búsqueda y la exploración adicional de posiciones tácticas. Estos parámetros son extensiones de la adaptación moderna, no características equivalentes de la interfaz original de 1976.

La separación entre player y evaluador permite que el evaluador Microchess sea utilizado por otros motores Python, como SARGON y SARGON-PLUS, respetando el contrato de evaluación compartido.

Existe una diferencia conceptual importante: STRATGY fue concebida para valorar movimientos utilizando información anterior y posterior a cada candidato, mientras que un evaluador intercambiable puede recibir únicamente una posición.

Por tanto, la adaptación debe distinguir la valoración histórica de jugadas de la evaluación posicional derivada, sin atribuirles equivalencia matemática completa.

La implementación en Akkad debe considerarse una **reconstrucción moderna inspirada en el Microchess original**, no una emulación exacta del programa ensamblador de Jennings.

14. Conclusión

Microchess representa un hito temprano de la ingeniería de software para sistemas con restricciones extremas.

Su innovación fundamental consistió en reemplazar estructuras de datos grandes y exploraciones exhaustivas por generación secuencial, reutilización de rutinas, máquinas de estados y evaluación heurística.

Su arquitectura demuestra que un sistema puede producir decisiones relativamente complejas utilizando representaciones compactas e información calculada bajo demanda.

Desde el punto de vista del ajedrez computacional, su contribución principal no fue una fuerza de juego excepcional, sino una solución ingeniosa al problema de decidir bajo recursos extraordinariamente limitados.

La incorporación de sus ideas a Akkad permite estudiar experimentalmente las diferencias entre heurísticas históricas, búsqueda tradicional y evaluación mediante redes neuronales.

---

15. Referencias bibliográficas

[1] Jennings, Peter (1976). *Microchess: A Chess Playing Program for the 6502 Microcomputer.* Micro-Ware Ltd., Toronto, Canadá. Manual del programa, documentación técnica, explicación de rutinas y listado de código fuente. Conservado por Computer History Museum.
https://www.computerhistory.org/chess/doc-431614f6d8478/

[2] Jennings, Peter (1976). *Microchess Programmer's Manual.* Documentación original de GNM, JANUS, STATE y STRATGY. Reproducida en la colección de documentación histórica 1976–1978 del Computer History Museum.
https://archive.computerhistory.org/resources/access/text/2018/02/102775100-05-01-acc.pdf

[3] Jennings, Peter. *History of Microchess.* Relato del autor acerca del desarrollo, representación del tablero, limitaciones de memoria y comercialización del programa.
https://www.benlo.com/microchess/

[4] Computer History Museum. *Microchess Source Code, Version 1.0 (1976).* Registro del programa original para KIM-1 y su listado de 924 bytes.
https://www.computerhistory.org/chess/sft-431e19dae0914/

[5] Computer History Museum. *Early Microcomputer Chess.* Panorama histórico de los primeros programas comerciales de ajedrez para microcomputadoras.
https://www.computerhistory.org/chess/early-microcomputer-chess/

[6] Computer History Museum (2005). *Oral History of Peter Jennings.* Entrevista sobre el desarrollo de Microchess, las limitaciones del KIM-1 y las decisiones de diseño.
https://archive.computerhistory.org/projects/chess/related_materials/oral-history/jennings.oral_history.2005.102630656/jennings.oral_history_transcrit.2005.102630656.pdf

---

Nota de alcance documental: los datos históricos y la descripción de GNM/JANUS/STATE proceden de los documentos originales y del relato del autor. Las ecuaciones conceptuales de representación, los ejemplos de intercambio y las formulaciones de búsqueda moderna son explicaciones técnicas. No deben confundirse con una traducción exacta del ensamblador original de 1976.

SARGON (1978): arquitectura, representación del tablero, búsqueda alfa-beta y evaluación estratégica

Autores: Dan Spracklen y Kathe Spracklen
Año de creación: 1978
Plataforma original: Wave-Mate Jupiter III
Procesador: Zilog Z80, arquitectura de 8 bits
Lenguaje: Ensamblador Z80, utilizando TDL Macro Assembler
Memoria requerida: Aproximadamente 8 KB
Publicación: *Sargon: A Computer Chess Program*, Hayden Book Company, 1978
Categoría: Motor de ajedrez basado en búsqueda minimax, poda alfa-beta, evaluación heurística y análisis especializado de intercambios.

---

1. Introducción y contexto histórico

SARGON fue desarrollado por Dan y Kathe Spracklen en 1978 y se convirtió en uno de los programas fundamentales de la historia del ajedrez para microcomputadoras.

Su presentación pública más importante tuvo lugar durante la segunda West Coast Computer Faire, celebrada en marzo de 1978. Allí ganó el primer torneo de ajedrez destinado exclusivamente a microcomputadoras, obteniendo cinco victorias en cinco partidas.

Este resultado fue especialmente significativo porque los microprocesadores de aquella época disponían de recursos computacionales muy inferiores a los grandes computadores utilizados en los programas de ajedrez más avanzados.

SARGON demostró que una combinación eficiente de búsqueda en árbol, evaluación posicional y conocimiento táctico podía producir un nivel de juego competitivo dentro de restricciones severas de memoria y capacidad de procesamiento.

En 1978, sus autores publicaron el libro *Sargon: A Computer Chess Program*, que incluía una descripción de la arquitectura, instrucciones de funcionamiento y el código fuente completo en ensamblador Z80.

Esta publicación convirtió a SARGON en uno de los motores históricos mejor documentados, permitiendo estudiar directamente sus estructuras de datos y rutinas.

SARGON dio origen a una familia de programas que continuó evolucionando durante las décadas siguientes, con versiones como SARGON II, SARGON III y SARGON V.

Es importante distinguir estas versiones: sus algoritmos, evaluadores, capacidades tácticas y plataformas no eran necesariamente idénticos.

El presente documento se concentra en **SARGON I, versión original de 1978**.

2. Arquitectura del computador original

La versión documentada fue desarrollada para el Wave-Mate Jupiter III, un computador basado en el microprocesador Zilog Z80.

El Z80 era un microprocesador de 8 bits con registros de propósito general de 8 y 16 bits, instrucciones aritméticas, operaciones lógicas, acceso directo a memoria y soporte para operaciones de bloques.

La memoria disponible era limitada, de manera que el programa debía utilizar estructuras relativamente compactas y controlar cuidadosamente el almacenamiento asociado a la búsqueda.

Los autores describieron una distribución aproximada de 8 KB:

Componente	Memoria
Estructuras y datos	2 KB
Interfaz y representación gráfica	2 KB
Lógica de movimientos y búsqueda	4 KB
Total	**8 KB**

Esta distribución no debe confundirse con el tamaño de todos los ejecutables comerciales posteriores.

La versión original presentaba un tablero gráfico y una lista de movimientos, además de permitir elegir el color y la profundidad de análisis.

La interfaz dependía parcialmente del sistema operativo JOVE del Jupiter III, pero los autores procuraron aislar las operaciones dependientes del hardware.

Esta separación facilitó posteriormente la adaptación del programa a computadoras como TRS-80, Apple II y Commodore PET.

3. Representación interna del tablero

3.1. Arquitectura de 120 casillas

Una de las decisiones fundamentales de SARGON fue utilizar una representación del tablero mediante una matriz de **120 posiciones**, organizada conceptualmente como 12 filas por 10 columnas.

Solamente 64 posiciones correspondían a las casillas reales del ajedrez.

Las restantes formaban un borde artificial que permitía detectar movimientos fuera del tablero.

Esta representación se conoce actualmente como *mailbox 10×12*.

Conceptualmente:

$$B[0], B[1], \ldots, B[119]$$

La codificación de una casilla válida puede expresarse mediante:

$$s = 21 + 10r + f$$

donde:

- $r \in \{0, \ldots, 7\}$ representa una fila interna.
- $f \in \{0, \ldots, 7\}$ representa una columna.

- s es el índice correspondiente dentro de la matriz.

El origen exacto de las coordenadas depende de la orientación interna del tablero.

Los elementos exteriores se marcaban con el valor hexadecimal `FF`.

3.2. Ventajas de la representación mailbox

La principal ventaja consistía en simplificar la generación de movimientos.

Por ejemplo:

Desplazamiento	Incremento
Horizontal	± 1
Vertical	± 10
Diagonal	$\pm 9, \pm 11$
Caballo	$\pm 8, \pm 12, \pm 19, \pm 21$

Para examinar un desplazamiento:

$$s' = s + \Delta$$

bastaba consultar el contenido de la casilla de destino.

Si:

$$B[s'] = FF_{16}$$

el movimiento había alcanzado el borde artificial y debía descartarse.

Esto eliminaba numerosas comprobaciones explícitas de filas y columnas.

El costo era utilizar 120 bytes para representar un tablero de 64 casillas, pero esa decisión simplificaba considerablemente el código de generación.

3.3. Codificación de las piezas

SARGON representaba las piezas mediante valores compactos de un byte.

Los tres bits inferiores identificaban el tipo de pieza:

Pieza	Código
Casilla vacía	0
Peón	1
Caballo	2
Alfil	3
Torre	4

Pieza	Código
Dama	5
Rey	6

El bit 7 representaba el color:

$$c = \begin{cases} 0 & \text{blancas} \\ 128 & \text{negras} \end{cases}$$

Por tanto, una pieza podía codificarse como:

$$p = c + t$$

donde t es el tipo de pieza.

Los bits restantes podían representar información adicional, como si la pieza ya había sido movida o determinadas condiciones asociadas al enroque.

Esta representación permitía realizar operaciones sobre tipo y color mediante máscaras y operaciones binarias.

4. Arquitectura general del programa

El programa estaba organizado alrededor de rutinas especializadas y estructuras compartidas.

La rutina de control general era `DRIVER`.

La selección de movimientos estaba centralizada en `FNDMOV`, que coordinaba la búsqueda.

Las restantes rutinas podían clasificarse en cuatro grupos funcionales:

1. Representación y modificación del tablero.
2. Generación y validación de movimientos.
3. Evaluación de posiciones e intercambios.
4. Búsqueda, ordenación y selección de jugadas.

La separación era considerablemente más estructurada que en muchos programas contemporáneos de tamaño similar.

5. Rutinas principales

5.1. DRIVER: control general

`DRIVER` coordinaba el funcionamiento del programa.

Sus responsabilidades incluían inicializar el tablero, solicitar el color y la profundidad de búsqueda, recibir movimientos del usuario, ejecutar movimientos propios y actualizar la pantalla.

El control de entrada y salida estaba relativamente separado de la lógica ajedrecística.

5.2. GENMOV: generación de movimientos

`GENMOV` generaba los movimientos correspondientes a un bando.

Utilizaba rutinas subordinadas:

- `MPIECE`: generación para una pieza determinada.
- `PATH`: exploración de una dirección de movimiento.
- `ADMOVE`: incorporación de un movimiento a la lista.
- `CASTLE`: tratamiento del enroque.
- `ENPSNT`: tratamiento de la captura al paso.
- `INCHK`: comprobación de jaque.

A diferencia del Microchess original, SARGON implementaba explícitamente reglas especiales del ajedrez, incluyendo enroque y captura al paso.

El programa también incluía tratamiento de promoción de peones.

5.3. MOVE y UNMOVE

`MOVE` aplicaba temporalmente una jugada sobre el tablero.

`UNMOVE` restauraba el estado anterior.

La combinación permitía explorar sucesivamente ramas del árbol sin almacenar una copia completa del tablero en cada nodo.

Conceptualmente:

$$P' = T(P, m)$$

y, después de analizar la posición:

$$P = T^{-1}(P', m, u)$$

donde u representa la información necesaria para deshacer exactamente la jugada.

5.4. PINFND: identificación de clavadas

`PINFND` identificaba piezas clavadas contra el rey o la dama.

Esta información era importante para evitar atribuir una capacidad de ataque efectiva a piezas que no podían desplazarse sin comprometer una pieza de mayor valor.

Las clavadas también influían en el análisis de intercambios.

5.5. ATTACK: análisis de ataques

`ATTACK` determinaba las piezas que atacaban o defendían una casilla.

Construía información sobre atacantes y defensores que podía ser utilizada por las rutinas de evaluación.

5.6. XCHNG: evaluación de intercambios

`XCHNG` era una de las rutinas tácticas más importantes.

Su objetivo era determinar el resultado potencial de intercambios de material en una casilla.

El análisis consideraba las piezas atacantes y defensoras, sus valores relativos y el orden de posibles capturas.

La rutina `NEXTAD` ayudaba a seleccionar sucesivos atacantes y defensores.

Esta técnica permitía evaluar amenazas materiales sin tener que desarrollar sistemáticamente todos los movimientos posibles en un árbol completo.

5.7. POINTS: evaluación de posiciones

`POINTS` combinaba los criterios estratégicos empleados por SARGON.

Sus elementos principales eran:

- Diferencia de material.
- Control del tablero.
- Desarrollo de piezas.
- Seguridad táctica del material.
- Oportunidades y riesgos de intercambio.

La evaluación de material era predominante.

5.8. LIMIT: limitación de factores posicionales

`LIMIT` impedía que las bonificaciones de desarrollo y control del tablero dominaran excesivamente las decisiones.

Esto resultaba fundamental para evitar que una ventaja posicional modesta compensara arbitrariamente una pérdida material significativa.

5.9. SORTM: ordenación de movimientos

`SORTM` ordenaba las jugadas candidatas mediante puntuaciones obtenidas a través de `EVAL`.

La ordenación era un componente esencial de la eficiencia de alfa-beta.

5.10. FNDMOV: búsqueda y selección

`FNDMOV` era la rutina central del motor.

Controlaba:

- La profundidad de búsqueda.
- La generación de candidatos.
- La exploración de variantes.
- La ordenación de movimientos.
- Las evaluaciones.
- Los límites alfa-beta.
- La propagación de resultados.
- La selección final.

5.11. ASCEND

ASCEND administraba el retorno desde un nivel del árbol hacia el nivel anterior.

Incluía las operaciones necesarias para restaurar el estado y actualizar la información de búsqueda.

5.12. BOOK

BOOK implementaba un repertorio de apertura extremadamente reducido.

La primera jugada de blancas podía seleccionarse entre los avances centrales de peón de rey o dama, utilizando un elemento de aleatoriedad.

Cuando jugaba con negras, elegía una respuesta central apropiada.

No era una biblioteca moderna de aperturas, sino un mecanismo para evitar la búsqueda completa durante la primera jugada.

6. Valores heurísticos de las piezas

El listado original incluye una tabla denominada PVALUE.

Los valores publicados son:

Pieza	Valor
Peón	1
Caballo	3
Alfil	3
Torre	5
Dama	9
Rey	10

La tabla tiene la forma:

```
PVALUE = {1, 3, 3, 5, 9, 10}
```

Estos valores son los del código original y no una reconstrucción aproximada.

El valor asignado al rey debe interpretarse en el contexto de las rutinas internas de captura y evaluación, no como el valor real de sacrificar un rey. Su pérdida sigue estando sujeta a las reglas especiales del juego.

6.1. Evaluación material

Sea $n_{c,p}(P)$ el número de piezas de tipo p pertenecientes al color c.

La diferencia material puede expresarse como:

$$M(P) = \sum_{p} v_p \left[n_{W,p}(P) - n_{B,p}(P) \right]$$

donde v_p corresponde a la tabla histórica.

En unidades de peón, una posición en la que las blancas disponen de una torre adicional tendría una ventaja material nominal de cinco unidades.

Sin embargo, SARGON no se limitaba a sumar material.

El programa intentaba determinar si las piezas presentes podían conservarse después de resolver los intercambios pendientes.

7. Evaluación de intercambios

7.1. Motivación

Una captura puede representar una ganancia inmediata y, simultáneamente, conducir a una pérdida material posterior.

Supongamos que una torre captura un alfil y después es capturada por un peón:

$$G_0 = 3$$

La recaptura representa una pérdida de cinco unidades:

$$G_1 = 3 - 5 = -2$$

Por tanto, una valoración que considere solamente la primera captura resulta engañosa.

7.2. Concepto de intercambio estático

SARGON buscaba estimar los resultados de los intercambios examinando atacantes y defensores.

Una formulación moderna útil para expresar la idea es:

$$SEE(P, s) = \max\left(0,\ v_0 - SEE(P', s)\right)$$

donde:

- s es la casilla objeto del intercambio.
- v_0 es el valor de la pieza capturada.
- P' representa la posición después de la captura.
- El siguiente término estima las recapturas posibles.

Esta ecuación describe el concepto de evaluación estática de intercambios, no una transcripción literal del código de `XCHNG`.

El algoritmo histórico utilizaba listas de atacantes y defensores, evaluaba secuencias de capturas y registraba ganancias o pérdidas potenciales.

7.3. Influencia de las clavadas

Una pieza clavada puede aparecer geométricamente como atacante, pero no necesariamente puede ejecutar su captura.

Por ello, el análisis táctico necesitaba información adicional sobre clavadas.

Esta integración entre `PINFND`, `ATTACK` y `XCHNG` constituía una fortaleza importante.

7.4. Limitaciones

El evaluador de intercambios no equivalía a una exploración táctica arbitrariamente profunda.

Podía fallar cuando la secuencia incluía:

- Jugadas intermedias.
- Jaques inesperados.
- Sacrificios posicionales.
- Desviaciones de defensores.
- Amenazas en otras casillas.
- Capturas cuya conveniencia depende de una continuación no local.

El sistema analizaba principalmente la dinámica material de los intercambios, no todas las consecuencias estratégicas de la posición.

8. Evaluación posicional

8.1. Componentes principales

El evaluador de SARGON puede describirse mediante el esquema:

$$E(P) = M(P) + C(P) + D(P) + X(P)$$

donde:

- $M(P)$: diferencia material.
- $C(P)$: control del tablero.
- $D(P)$: desarrollo.
- $X(P)$: ajustes asociados a intercambios.

Esta ecuación es una representación conceptual de las responsabilidades del evaluador. No es una fórmula literal publicada en el listado de SARGON.

El código real combina mediciones, comparaciones, restricciones y condiciones específicas.

8.2. Control del tablero

Para cada casilla, SARGON calculaba información de atacantes blancos y negros.

Una medida conceptual de control es:

$$C(P) = \sum_{s \in B} [A_W(P, s) - A_B(P, s)]$$

donde A_W y A_B representan los números de atacantes de cada bando.

El listado muestra explícitamente que se acumulaba la diferencia entre cantidades de atacantes.

Esta medida favorecía posiciones donde las piezas propias controlaban más casillas.

8.3. Desarrollo

SARGON contemplaba bonificaciones y penalizaciones asociadas a la actividad de las piezas.

El código incluía, entre otras condiciones:

- Penalizaciones asociadas al desarrollo de determinadas piezas mayores durante la apertura.
- Premios relacionados con el enroque.
- Consideraciones sobre si una pieza había sido movida.

En el listado de `POINTS` aparece una bonificación de seis unidades por determinadas condiciones de enroque y penalizaciones de dos unidades relacionadas con el desarrollo.

Estos coeficientes deben interpretarse dentro de las escalas internas del programa, no como centipeones modernos.

8.4. Límite de la evaluación posicional

SARGON aplicaba la rutina `LIMIT` para restringir la influencia de las ventajas de control y desarrollo.

Conceptualmente:

$$E(P) = M(P) + \text{clip}\left(C(P) + D(P), -L, L\right) + X(P)$$

La expresión anterior ilustra el principio de limitar contribuciones posicionales; no reproduce exactamente el orden de operaciones del ensamblador.

Su propósito era asegurar que la posición no recibiera una valoración excesiva por aspectos estratégicos menores mientras existieran diferencias materiales importantes.

Esta decisión introducía una jerarquía heurística: el material y los intercambios debían tener un peso decisivo.

9. Algoritmo de búsqueda minimax

A diferencia del diseño de Microchess, SARGON utilizaba explícitamente un árbol de búsqueda alternando las decisiones de ambos jugadores.

9.1. Formulación matemática

Sea:

- P: posición.
- d: profundidad restante.
- $E(P)$: evaluación estática.
- $T(P, m)$: posición resultante de ejecutar m.
- $\mathcal{M}(P)$: movimientos legales.

Desde la perspectiva de blancas:

$$V(P,d) = \begin{cases} E(P), & d = 0 \\ \max\limits_{m \in \mathcal{M}(P)} V(T(P,m), d-1), & \text{juegan blancas} \\ \min\limits_{m \in \mathcal{M}(P)} V(T(P,m), d-1), & \text{juegan negras} \end{cases}$$

Los estados terminales requieren valores especiales para mate y tablas.

Una formulación simétrica moderna es negamax:

$$N(P,d) = \max_{m \in \mathcal{M}(P)} \left[-N(T(P,m), d-1) \right]$$

con la evaluación expresada desde el punto de vista del bando que mueve.

Negamax es una reformulación matemática conveniente de minimax. No debe interpretarse como una identificación exacta de la estructura del código ensamblador.

9.2. Profundidad

La interfaz documentada permitía seleccionar profundidades de uno a seis *plies*.

Un ply corresponde a un movimiento individual de uno de los jugadores.

Por ejemplo, tres plies representan:

1. Una jugada del motor.
2. Una respuesta del adversario.
3. Una segunda jugada del motor.

La cantidad de nodos crece aproximadamente como:

$$N(d) \approx b^d$$

donde b representa el factor de ramificación medio.

Si suponemos $b = 30$:

$$N(3) \approx 27\,000$$

$$N(4) \approx 810\,000$$

$$N(5) \approx 24\,300\,000$$

Estos valores son ilustrativos y corresponden a un árbol completo uniforme, no a mediciones del SARGON original.

10. Poda alfa-beta

10.1. Objetivo

La poda alfa-beta evita explorar ramas que no pueden mejorar la decisión del jugador.

Mantiene dos límites:

$$\alpha = \text{mejor valor garantizado para MAX}$$

$$\beta = \text{mejor valor garantizado para MIN}$$

Cuando:

$$\alpha \geq \beta$$

la rama restante puede descartarse.

10.2. Recurrencia

Una formulación moderna del algoritmo es:

```
ALPHABETA(P, d, alpha, beta):

    si P es terminal:
        devolver resultado terminal

    si d == 0:
        devolver E(P)

    si juega MAX:

        valor = -infinito

        para cada movimiento legal m:

            aplicar(m)

            valor = max(
                valor,
                ALPHABETA(P', d-1, alpha, beta)
            )

            deshacer(m)

            alpha = max(alpha, valor)

            si alpha >= beta:
                interrumpir

        devolver valor

    si juega MIN:

        valor = +infinito

        para cada movimiento legal m:

            aplicar(m)

            valor = min(
                valor,
                ALPHABETA(P', d-1, alpha, beta)
```

```
            )

            deshacer(m)

            beta = min(beta, valor)

            si alpha >= beta:
                interrumpir

        devolver valor
```

El listado original de SARGON implementaba la administración de límites y el recorrido en profundidad fundamentalmente dentro de `FNDMOV`.

10.3. Complejidad

Sin poda:

$$O(b^d)$$

Con ordenación ideal de movimientos, alfa-beta puede reducir considerablemente el número de nodos examinados, aproximándose en el caso favorable a:

$$O(b^{d/2})$$

Esta es una propiedad teórica de alfa-beta y no una garantía del rendimiento obtenido por SARGON.

En una máquina de 8 bits, reducir el número de nodos resultaba especialmente importante.

11. Ordenación de movimientos

La eficiencia de alfa-beta depende fuertemente del orden en que se examinan los movimientos.

Si las mejores jugadas se examinan primero, los límites alfa y beta se ajustan antes y se producen más cortes.

SARGON utilizaba `SORTM` para ordenar candidatos.

Para obtener una estimación preliminar, `SORTM` recurría a `EVAL`, que:

1. Aplicaba temporalmente la jugada.
2. Verificaba su legalidad.
3. Actualizaba la información de clavadas.
4. Evaluaba la posición resultante mediante `POINTS`.
5. Restauraba la posición anterior.

Conceptualmente, la ordenación puede representarse como:

$$m_1, m_2, \ldots, m_n = \mathrm{sort}_{\downarrow}\left(\widehat{V}(P, m)\right)$$

donde $\widehat{V}$ representa una valoración preliminar.

La evaluación previa tenía un costo computacional propio, pero podía compensarlo reduciendo considerablemente la exploración posterior.

En el contexto histórico, esta combinación de evaluación y ordenación fue una decisión particularmente importante.

12. Política de elección de jugada

La política de SARGON no consistía simplemente en seleccionar el movimiento con mejor evaluación inmediata.

Buscaba maximizar el resultado esperado bajo la hipótesis de que el adversario también elegiría respuestas favorables para sí mismo.

Su criterio conceptual era:

$$m^* = \underset{m \in \mathcal{M}(P)}{\operatorname{argmax}} \left[-N(T(P, m), d-1) \right]$$

La selección comprendía:

1. Generar candidatos.
2. Ordenarlos mediante una evaluación preliminar.
3. Explorar las respuestas del adversario.
4. Aplicar poda alfa-beta.
5. Evaluar las posiciones alcanzadas.
6. Incorporar las consecuencias de los intercambios materiales.
7. Propagar resultados hacia la raíz.
8. Seleccionar la jugada de mayor valor minimax.

Esta arquitectura tenía una ventaja significativa frente a una política basada únicamente en la valoración estratégica inmediata: permitía detectar secuencias tácticas que dependían de varias jugadas alternadas.

Sin embargo, la fuerza final dependía de la profundidad alcanzada y de la calidad de la evaluación utilizada en las hojas.

13. Un ejemplo de selección de jugadas

Supongamos que SARGON considera tres movimientos:

Movimiento	Evaluación inmediata	Mejor respuesta adversaria	Resultado a profundidad 2
A	+4	−6	−2
B	+2	−1	+1
C	+1	0	+1

Los valores son hipotéticos y se expresan desde la perspectiva del motor.

Una política de evaluación inmediata elegiría A porque presenta la mayor ganancia aparente.

Minimax considera la respuesta del adversario.

Suponiendo que las cifras finales representan los valores minimax de cada rama:

$$V(A) = -2, \quad V(B) = +1, \quad V(C) = +1$$

SARGON preferiría B o C frente a A.

La selección definitiva entre B y C dependería de la ordenación, los criterios de desempate y la información adicional disponible.

Este ejemplo ilustra por qué el análisis de respuestas proporciona una capacidad táctica que no se obtiene con la evaluación inmediata de movimientos.

14. Diferencias entre SARGON y Microchess

Característica	**Microchess (1976)**	**SARGON (1978)**
Autores	Peter Jennings	Dan y Kathe Spracklen
Procesador original	MOS 6502	Zilog Z80
Memoria	Aproximadamente 1 KB	Aproximadamente 8 KB
Representación	Compacta, centrada en piezas	Mailbox de 120 casillas
Generación	Secuencial	Generación de listas
Control	GNM/JANUS/STATE	FNDMOV y rutinas especializadas
Búsqueda principal	Valoración selectiva de candidatos	Minimax con alfa-beta
Profundidad	Selectiva y limitada	Configurable por plies
Evaluación	STRATGY	POINTS
Intercambios	WCAP/BCAP	XCHNG
Clavadas	Tratamiento limitado	PINFND
Ordenación	Selección de candidatos	SORTM
Desarrollo	Heurístico	Heurístico
Aperturas	Sin biblioteca extensa	BOOK de una jugada
Reglas especiales	Incompletas en versión básica	Enroque, al paso y promoción
Principal ventaja	Economía extrema de memoria	Mayor profundidad y capacidad táctica

La diferencia conceptual más importante es que SARGON asignaba mayor responsabilidad a la exploración del árbol de variantes.

Microchess obtenía una parte considerable de su conocimiento de la posición mediante mediciones estratégicas y análisis selectivos relativamente compactos.

SARGON combinaba evaluación heurística con una búsqueda adversarial explícita.

15. Fortalezas

15.1. Arquitectura modular

La separación entre generación, evaluación, búsqueda, análisis de intercambios e interfaz facilitaba comprender y modificar el programa.

15.2. Búsqueda adversarial

Minimax permitía considerar que el oponente seleccionaría respuestas desfavorables para el motor.

Esta característica mejoraba la capacidad de reconocer amenazas tácticas.

15.3. Poda alfa-beta

La poda permitía dedicar la capacidad limitada del Z80 a variantes relevantes, evitando examinar ramas que no podían alterar el resultado.

15.4. Evaluación material relativamente sofisticada

El análisis mediante `XCHNG` permitía reconocer una parte de los intercambios desfavorables sin tener que explorar todas las continuaciones.

15.5. Detección de clavadas

La existencia de `PINFND` mejoraba la interpretación de ataques y defensas.

15.6. Ordenación de movimientos

La clasificación previa de movimientos aumentaba las posibilidades de poda y mejoraba el rendimiento de la búsqueda.

15.7. Interpretabilidad

Los componentes de evaluación eran explícitos y estaban documentados.

El algoritmo no dependía de parámetros aprendidos difíciles de interpretar.

15.8. Portabilidad conceptual

La separación relativa entre lógica ajedrecística e interfaz permitió adaptar el programa a distintas computadoras.

16. Debilidades

16.1. Profundidad limitada

La cantidad de variantes aumentaba exponencialmente con la profundidad.

El procesador Z80 restringía considerablemente la profundidad práctica de análisis.

16.2. Efecto horizonte

SARGON podía valorar incorrectamente una posición cuando una consecuencia táctica importante ocurría más allá del límite de búsqueda.

16.3. Evaluación posicional relativamente simple

La evaluación estaba dominada por material, control y desarrollo.

No disponía de modelos estratégicos comparables a los de los motores modernos.

16.4. Dependencia de valores manuales

Los coeficientes heurísticos eran establecidos por los programadores.

No existía un mecanismo de aprendizaje automático para optimizarlos sistemáticamente.

16.5. Limitaciones del evaluador de intercambios

`XCHNG` podía analizar pérdidas y ganancias materiales, pero no todas las combinaciones tácticas posibles.

16.6. Costo de ordenación

`SORTM` requería evaluar movimientos antes de explorarlos en profundidad.

En ciertas posiciones, el costo de esta evaluación preliminar podía resultar importante.

16.7. Biblioteca de aperturas mínima

La versión original no disponía de una base extensa de aperturas.

16.8. Ausencia de técnicas modernas

La arquitectura original no incorporaba elementos como:

- Tablas de transposición de gran capacidad.
- Redes neuronales.
- Evaluación NNUE.
- Búsqueda Monte Carlo.
- Paralelismo moderno.
- Aprendizaje a partir de partidas.
- Mecanismos avanzados de reducción selectiva comparables a los actuales.

No todas estas técnicas existían en 1978, de modo que su ausencia debe interpretarse históricamente.

17. Características principales

Característica	**SARGON original**
Autores	Dan y Kathe Spracklen
Año	1978
Computador original	Wave-Mate Jupiter III

Característica	SARGON original
Procesador	Zilog Z80
Memoria	8 KB
Lenguaje	Ensamblador
Tablero	Mailbox 10×12
Identificación de piezas	Un byte por casilla
Generador	GENMOV/MPIECE/PATH
Legalidad	INCHK y rutinas especiales
Búsqueda	Minimax
Optimización	Alfa-beta
Ordenación	SORTM
Evaluación	POINTS
Intercambios	XCHNG/NEXTAD
Clavadas	PINFND
Desarrollo	Heurísticas explícitas
Control de tablero	Diferencia de ataques
Aperturas	BOOK
Profundidad seleccionable	1–6 plies
Aprendizaje automático	No
Redes neuronales	No

18. Aplicación al proyecto Akkad

SARGON constituye una referencia particularmente apropiada para el proyecto Akkad porque su arquitectura separa tres responsabilidades fundamentales:

$$\text{Generación} \longrightarrow \text{Búsqueda} \longrightarrow \text{Evaluación}$$

Esta separación permite investigar experimentalmente qué parte de la fuerza ajedrecística procede del algoritmo de búsqueda y qué parte procede de la calidad de la evaluación.

En una implementación moderna pueden conservarse elementos históricos como:

- La representación mailbox.
- La generación basada en direcciones.
- La aplicación y restauración de movimientos.
- La búsqueda minimax con alfa-beta.
- La ordenación de movimientos.

- El análisis de capturas y clavadas.
- La evaluación de material y control posicional.

Al mismo tiempo pueden utilizarse implementaciones modernas de esos conceptos, sin reproducir las restricciones de memoria y tiempo del Z80.

La arquitectura de Akkad permite desacoplar el algoritmo de búsqueda del evaluador.

Por ejemplo:

Player	Evaluador	Objetivo experimental
SARGON	CLASSIC	Búsqueda con heurísticas convencionales
SARGON	Microchess	Comparar evaluación histórica alternativa
SARGON	NPZ	Búsqueda convencional con evaluación neuronal
SARGON-PLUS	Microchess	Estudiar mejoras de búsqueda manteniendo evaluación histórica
SARGON-PLUS	NPZ	Comparar búsqueda optimizada y evaluación aprendida

En estas combinaciones debe mantenerse una convención matemática única para la evaluación.

Si:

$$E(P) > 0$$

significa ventaja de blancas, independientemente del jugador que tenga el turno.

El player debe convertir esa evaluación a la perspectiva que corresponda para ejecutar negamax o minimax.

Esta separación también permite comparar calidad de juego y rendimiento computacional sin mezclar innecesariamente los cambios del buscador con los del evaluador.

En particular, las métricas de nodos por segundo, tiempo por nodo, profundidad alcanzada, tasa de victorias y resultado de enfrentamientos pueden utilizarse para medir diferencias entre configuraciones.

Sin embargo, **NPS no mide directamente la fuerza ajedrecística**: dos motores pueden contabilizar operaciones distintas como nodos.

La comparación más fiable requiere partidas controladas, posiciones tácticas reproducibles y una definición homogénea de las métricas.

El player SARGON y el player SARGON-PLUS de Akkad son implementaciones modernas inspiradas en esta familia de algoritmos. No debe suponerse que reproducen instrucción por instrucción el SARGON I original.

19. Conclusión

SARGON constituye uno de los ejemplos más importantes de la evolución del ajedrez informático desde programas extremadamente compactos hacia arquitecturas de búsqueda estructuradas.

Su contribución fundamental fue demostrar que una microcomputadora podía combinar:

1. Representación eficiente del tablero.

2. Generación legal de movimientos.
3. Evaluación material y posicional.
4. Reconocimiento de clavadas e intercambios.
5. Ordenación de movimientos.
6. Búsqueda minimax con poda alfa-beta.

La fortaleza del programa no residía exclusivamente en buscar más profundamente, sino en utilizar cuidadosamente la información disponible para mejorar tanto la evaluación como la eficiencia de la búsqueda.

La combinación de `POINTS`, `XCHNG`, `PINFND`, `SORTM` y `FNDMOV` representaba una arquitectura notablemente avanzada para un microcomputador de 8 bits.

Su principal limitación era la capacidad de procesamiento disponible, que restringía la profundidad de búsqueda y obligaba a emplear una evaluación relativamente económica.

Desde una perspectiva actual, SARGON demuestra que la fuerza ajedrecística depende de la interacción entre tres elementos:

$$\boxed{\text{Fuerza de juego} = f(\text{Búsqueda}, \text{Evaluación}, \text{Recursos})}$$

No existe una relación lineal simple entre estos componentes.

Un evaluador más sofisticado puede ser demasiado costoso; una búsqueda más profunda puede resultar inútil si evalúa incorrectamente las posiciones; y una arquitectura eficiente puede superar a otra teóricamente más ambiciosa bajo recursos limitados.

La importancia de SARGON consiste precisamente en haber alcanzado un equilibrio práctico entre esos factores.

---

20. Referencias bibliográficas

[1] Spracklen, Dan y Spracklen, Kathe (1978). *Sargon: A Computer Chess Program.* Hayden Book Company, Rochelle Park, New Jersey. ISBN 0-8104-5155-4. Obra primaria que contiene la descripción del programa, sus estructuras de datos, rutinas principales y código fuente en ensamblador Z80.

https://openlibrary.org/books/OL4747033M/Sargon_a_computer_chess_program

[2] Computer History Museum. *Sargon: A Computer Chess Program.* Registro histórico de la versión 1.0, publicada en 1978, y de su participación en el torneo de microcomputadoras de la West Coast Computer Faire.

https://www.computerhistory.org/chess/sft-431614f63c4c/

[3] Spracklen, Dan y Spracklen, Kathe (1978). *An Exchange Evaluator for Computer Chess.* BYTE, noviembre de 1978. Artículo técnico sobre la evaluación de intercambios y su importancia en el ajedrez computacional para microprocesadores.

https://www.worldradiohistory.com/Archive-Byte/70s/Byte-1978-11.pdf

[4] Computer History Museum (1978). *First Steps in Computer Chess Programming.* BYTE Publications. Material histórico relacionado con los conceptos de programación de ajedrez utilizados por los Spracklen.

https://www.computerhistory.org/chess/doc-431614f6da6d4/

[5] Computer History Museum. *Micros Become Masters.* Historia de la evolución de los programas de ajedrez para microprocesadores y su crecimiento en fuerza de juego.

https://www.computerhistory.org/chess/micros-become-masters/

[6] Forster, Bill. *Retro Sargon.* Proyecto de recuperación y adaptación del código fuente del SARGON de 1978, basado en el listado original publicado por Hayden.

https://github.com/billforsternz/retro-sargon

[7] ChessBase. *Sargon — A Chess Program from the Past.* Artículo histórico sobre las versiones de SARGON y las características de la implementación original.

https://en.chessbase.com/post/sargon-a-che-program-from-the-past/820

Nota metodológica: los datos de autoría, plataforma, organización del programa, representación mailbox, tabla PVALUE, rutinas y utilización de alfa-beta están respaldados por la publicación histórica de 1978. Las ecuaciones de minimax, negamax, complejidad y evaluación de intercambios son formalizaciones matemáticas modernas destinadas a explicar los mecanismos. No deben interpretarse como transcripciones literales del ensamblador Z80 ni como prueba de que las versiones posteriores de SARGON mantuvieran exactamente los mismos algoritmos.

Redes neuronales

Concepto y detalles de arquitecturas Representación Metodos numéricos

De las máquinas

Guiado por .pgn vs. aprendizaje al azar

Entrenamiento (Backpropagation)

Pytorch

Tensorflow

Tensorflow Lite

Arquitectura NNUE

Arquitectura MCTS

Arquitectura Transformer (Search less)

Destilado

Destilado con red

Destilado con motor

Comunicación

Sistema freechess.org

Sistema Universal Chess Interface (UCI)

Comunicación

Convenciones

Métodos de enlace

Repositorios de partidas

Describir Entrada-Salida estándar * TCP/IP

Motores vs. GUI

Reglas del ajedrez

Modos

Clásico, Blitz, Bullet y otros modos especiales significado de cada uno

Sistema de puntuación ELO

Explicar el sistema ELO y las fórmulas, como se usa en Ajedrez y como se usa en éste proyecto para evaluar la potencia relativa entre distintas alternaivas Explicar como calcular el ELO en forma relativa

El problema

Describir el ajedrez como problema de optimización y describir la dimensión de su dificultad

Explicar que el objetivo es encontrar algoritmos de razonable competitividad para plataformas pequenas

La competencia entre motores como validación de capacidades

Explicar el problema de evaluar el resultado de un algoritmo, concepto de antagonismo y competencia uso del sistema ELO

Evolución del proyecto

Explicar brevemente las iniciativas y sus problemas

Bobby

Nyx

Akkad

Dificultades de recursos

El problema del entrenamiento, uso de CPU y de GPU, doble problema acoplado optimizar y entrenar, importar redes, destilar redes.

Random

Heurística pura

Descripción y limitaciones

Ecuación de Bellman

Política y acción, descripción matemática exhaustiva Comparar Ta-Te-Ti con Damas y Ajedrez la maldición de la dimensionalidad El problema del entrenamiento Posibles alternativas (¿Bellman restringido?)

Minimax Alpha/Beta

Descripción y limitaciones Algoritmo Sargón en detalle

Negamax

.....

Estrategias de poda (pruning)

Heurísticas extremas

Redes neurales

Distintas estrategias para su uso

Redes NNUE

Descripción exhaustiva de su formulación arquitectura y entrenamiento Como representar el espacio de juego Distintos tipos y usos

Aqui hay que hablar algo de Cerebrum, npz y grumpy

Redes convolucionales

El ajedrez como un problema de interpretación visual y su aplicación a enfoque *search less* Arquitectura de las redes necesarias

Monte Carlo Tree Search

El ajedrez como un problema de política y valor Arquitectura de las redes necesarias

Algoritmos genéticos

Que se puede aportar aqui

Algoritmos cuánticos

Evaluación vs. Jugada

Ver la necesidad de partir la evaluación vs. la jugada

Implementación

Plataforma Sargon-NNUE

Descripción de arquitectura

''' ⬅️Akkad/ ├── .gitignore ├── .vscode/ ├── createPackage.ps1 ├── excluidos.txt ├── listZipFolders.ps1 │ ├── AlphaZero/ [NUEVO: migrado desde Sargon-NNUE/x86/AlphaZero] │ ├── CMakeLists.txt │ ├── README.md │ ├── README_ARM64.md │ ├── README_CONFIG_v0.1.2.md │ ├── README_GPU_v0.1.1.md │ ├── README_DEBIAN_ARM64.md │ ├── brain_example.json │ ├── build.ps1 [Windows] │ ├── build_debian.sh [Debian] │ ├── build_macos.sh │ ├── build_posix.sh │ ├── build_raspberrypi.sh │ ├── src/ │ │ ├── akkad_alphazero_main.cc │ │ └── config.h │ ├── tests/ │ │ └── uci_smoke.ps1 │ └── tools/ │ └── bash/ │ ├── build.sh │ ├── run.sh │ ├── diagnostics.sh │ ├── uci_smoke.sh │ └── run_benchmark.sh │ ├── Sargon-NNUE/ [orquestador Python / benchmark / match / rally] │ ├── examples/ │ │ ├── lc0/ │ │ └── lc0_uci/ │ ├── scripts/ │ │ ├── crypt/ │ │ ├── data/ │ │ ├── legacy/ │ │ ├── rally/ │ │ ├── sargon_benchmark.py │ │ ├── sargon_match.py │ │ ├── sargon_listen.ps1 │ │ └── ... │ ├── src/ │ │ ├── sargon_nnue/ │ │ │ ├── evaluators/ │ │ │ ├── players/ │ │ │ ├── brains.py │ │ │ ├── factory.py │ │ │ ├── sargon_listen.py │ │ │ ├── sargon_arbiter.py │ │ │ └── ... │ │ ├── sargon_nnue.egg-info/ │ │ └── **pycache**/ │ ├── tests/ │ ├── data/ [usado operativamente aunque no se vea completo en el ZIP] │ │ ├── brains/ │ │ │ ├── brains.json │ │ │ ├── brains_amelia.json │ │ │ └── ... │ │ ├── runs/ │ │ │ └── lc0/ │ │ │ └── t1-256x10-distilled-swa-2432500.pb.gz │ │ ├── benchmark/ │ │ ├── match/ │ │ │ └── ... │ └── x86/ [LEGADO / histórico] │ ├── AlphaZero/ [origen anterior de AlphaZero] │ │ ├── src/ │ │ │ └── tests/ │ └── picoKeras/ │ ├── build/ │ ├── include/ │ ├── scripts/ │ ├── src/ │ ├── tests/ │ └── tools/ │ ├── challenger/ │ ├── data/ │ ├── docs/ │ │ ├── AlphaZero_Algorithm/ │ │ ├── Sargon_Adaptativo/ │ │ ├── Sargon_Algorithm/ │ │ ├── Sargon_Learning_Algorithm/ │ │ └── StockFish_Algorithm/ │ ├── scripts/ │ ├── src/ │ │ └── challenger/ │ └── tests/ │ ├── fics-uci/ │ ├── archive/ │ ├── **pycache**/ │ ├── fics_uci.py │ ├── fics_stats.py │ ├── config.yml │ └── tools/ [AGREGADO operativamente] │ └── bash/ │ ├── run_fics_uci.sh │ └── run_fics_uci_amelia.sh │ ├── picoAlpha/ │ ├── ARM/ │ │ ├── config/ │ │ ├── include/ │ │ ├── src/ │ │ ├── tests/ │ │ └── tools/ │ └── x86/ │ └── picoAlpha/ │ ├── build/ │ ├── include/ │ ├── src/ │ ├── tests/ │ └── tools/ │ ├── picoKeras/ │ ├── ARM/ [placeholder / port futuro a arquitectura Pico] │ │ ├── include/ │ │ └── src/ │ ├── x86/ [motor válido de escritorio original] │ │ └── picoKeras/ │ │ ├── build/ │ │ ├── include/ │ │ ├── src/ │ │ └── third_party/ │ ├── debian/ [NUEVO: port Debian con TFLite] │ │ ├── CMakeLists.txt │ │ ├── model_tflite.cpp │ │ └── picoKeras/ [salida/estructura de build del port] │ ├── scripts/ │ ├── tools/ │ │ └── bash/ │ │ ├── build.sh │ │ ├── run.sh │ │ └── uci_smoke.sh │ └── README_DEBIAN.md │ └── picoSargon/ ├── .vscode/ ├── ARM/ │ ├── assets/ │ ├── build/ │ ├── config/ │ ├── include/ │ ├── reference/ │ ├── scripts/ │ ├── src/ │ └── tests/ ├── x86/ │ └── picoSargon/ │ ├── assets/ │ ├── build/ │ ├── build-host-windows/ │ ├── host_build_source/ │ ├── include/ │ └── src/ ├── debian/ [NUEVO: port Debian] │ └── picoSargon/ │ ├── CMakeLists.txt │ ├── assets/ │ │ └── grumpy.bin │ ├── build/ │ └── ... ├── tools/ │ ├── bash/ │ │

├── build.sh │ │ ├── run.sh │ │ ├── run_brain.sh │ │ ├── run_listen.sh │ │ ├── run_match.sh │ │ ├── run_benchmark.sh │ │ ├── uci_smoke.sh │ │ └── ... │ └── uci_screen_gui/ └── README_PORT_DEBIAN_v0.2.21.md '''

flowchart TD AK["Akkad / Raíz del proyecto"]

```
AK --> AZ["AlphaZero"]
AK --> SN["Sargon-NNUE"]
AK --> PS["picoSargon"]
AK --> PA["picoAlpha"]
AK --> PK["picoKeras"]
AK --> FI["fics-uci"]
AK --> CH["challenger"]

AZ --> AZS["src / Núcleo C++"]
AZ --> AZW["build.ps1 / Windows"]
AZ --> AZD["tools/bash / Debian"]
AZ --> AZT["tests"]

SN --> SNS["src/sargon_nnue"]
SN --> SNC["data/brains"]
SN --> SND["data/runs"]
SN --> SNT["scripts"]
SNS --> SNP["players"]
SNS --> SNE["evaluators"]
SNT --> SNB["benchmark"]
SNT --> SNM["match / rally / listen"]

PS --> PSW["x86/picoSargon"]
PS --> PSP["ARM / Pico W y 2W"]
PS --> PSD["debian/picoSargon"]
PS --> PST["tools/bash"]
PSD --> PSG["Grumpy NNUE"]
PSW -. "Fuentes C++ compartidos" .-> PSD

PA --> PAW["x86/picoAlpha"]
PA --> PAP["ARM / Pico W y 2W"]

PK --> PKW["x86/picoKeras"]
PK --> PKP["ARM / Placeholder Pico"]
PK --> PKD["debian"]
PK --> PKT["tools/bash"]
PKD --> PKL["TensorFlow Lite"]
PKW -. "Núcleo C++ compartido" .-> PKD

FI --> FIP["fics_uci.py"]
FI --> FIC["config_amelia.yml"]
FI --> FIS["Scripts Bash"]

CH --> CHS["src/challenger"]
CH --> CHT["tests / docs"]
```

```
SNM -. "UCI" .-> AZ
SNM -. "UCI" .-> PS
SNM -. "UCI" .-> PA
SNM -. "UCI" .-> PK
FIP -. "UCI/TCP" .-> PSD

classDef root fill:#253858,color:#fff,stroke:#172b4d
classDef core fill:#e1edff,color:#17365d,stroke:#5c91ca
classDef debian fill:#d8f3df,color:#14532d,stroke:#45a06c
classDef pico fill:#fff0cd,color:#7a4a06,stroke:#d49b37
classDef placeholder fill:#efefef,color:#666,stroke:#aaa,stroke-dasharray:5 3

class AK root
class AZ,SN,PS,PA,PK,FI,CH core
class AZD,PSD,PKD,PKL debian
class PSP,PAP pico
class PKP placeholder
```

Arquitecturas soportadas

ARM (Raspberry Pi Pico) X86 (Windows) MAC Debian (Raspberry Pi)

Ciclo de desarrollo

Desarrollo y evaluación en Python (Sargon-NNUE)

Desarrollo o adaptación de red neuronal o mecanismo de evaluación

Evaluación de performance

Competencia entre brains

Competencia abierta (lichess y fics)

Player y Evaluador con bus comun

Posibilidad de incorporar motores (Stockfish, lc0)

Posibilidad de importar "cerebros"

Definición de caracteristicas de un jugador (cerebro).

Interfaces (UCI TCP/IP)

Challenger

Ciclo de entrenamiento

Entrenamiento NNUE

Entrenamiento MCTS

Entrenamiento KERAS

Destilado

Peso de las redes (FP32 vs. INT8)

Players Genéricos

Random

Sargon

SargonPlus

Keras

Cerebrum

AlphaZero

StockFish

lc0

External

microChess

Players Optimizados

Keras

picoSargon (x86)

picoAlpha (x86)

Alphazero (x66) ELO 2500

picoSargon (ARM) ELO 1800

picoAlpha (ARM)

picoChess (ARM)

Evaluadores Genericos

Null

Classic

npz

Keras

Grumpy

Cerebrum-v1.1

lc0

microChess

Evaluadores especializados

tinyGyal

picoGyal

picoChess

Plataforma

Como se juntan e interconectn players con evaluadores

Definición de configuraciones (brains.json)

Evaluación de performance (benchmark)

Evaluación de capacidad mutua (match)

Evaluación de capacidad cruzada (rally)

Cruces entre player y evaluador

Al ser un bus comun se pueden intercambiar, algunas experiencias

Entrenamiento

Preparación

Lectura de PGN

Extracción de muestra

Valuación

Entrenamiento (PyTorch)

Auditoría

Servidor (TCP/IP)

Servidor (E/S Estándar)

Challenger

Bridges

lichess-bot

fics-uci

fics-stat

Motores optimizados

picoSargon (X86)

picoSargon (ARM)

picoSargon (Debian)

picoSargon (MacOS)

picoAlpha (X86)

picoAlpha (ARM)

picoAlpha (Debian)

picoAlpha (MacOS)

picoKeras (X86)

picoKeras (ARM)

picoKeras (Debian)

picoKeras (MacOS)

picoChess (X86)

picoChess (ARM)

picoChess (Debian)

picoChess (MacOS)

Motores experimentales

Bellman-Restricted

Quantum

Genetic

Construcción

Entrenamiento convencional

Entrenamiento destilado

Adopción de redes pre-entrenadas

Cerebrum

Red nativa npz

Red semi-nativa grumpy

Redes leela zero chess

Red muy grande

Red grande

tinyGyal

destilado de picoGyal

Redes transformadoras

medium y small.keras

Evaluación de performance

Capacidades relativas

Marcar que la referencia es sargon classic

Competencia abierta

Casos de lichess.org y fics

Apéndice A - Notación

Para registrar y leer las partidas de ajedrez se utiliza un sistema universal llamado Notación Algebraica. Este sistema sustituyó al antiguo sistema descriptivo (como "P4R" o "Peón 4 Rey") y es el estándar oficial de la FIDE (Federación Internacional de Ajedrez). A continuación, te presento un resumen detallado con ejemplos claros de cómo funciona:

Identificación del Tablero (Filas y Columnas)

El tablero se divide mediante un sistema de coordenadas similar a una batalla naval: • Columnas (verticales): Se nombran con letras minúsculas de la a a la h (de izquierda a derecha desde la perspectiva de las piezas blancas). • Filas (horizontales): Se nombran con números del 1 al 8 (la fila 1 es donde se colocan las piezas mayores blancas y la 8 donde se colocan las negras). • Casillas: Cada casilla se identifica combinando su letra y su número. Ejemplo: e4, c6, h7.

Identificación de las Piezas

Cada pieza se representa con la primera letra de su nombre en mayúscula. • R = Rey • D = Dama (o Reina) • T = Torre • A = Alfil • C = Caballo • Nota importante: Los peones no llevan ninguna letra. Si solo aparece el nombre de la casilla, significa que se movió un peón. (En libros internacionales es común ver la notación en inglés: K=King, Q=Queen, R=Rook, B=Bishop, N=Knight).

Reglas de Escritura y Ejemplos

Una jugada normal se escribe indicando la letra de la pieza + la casilla de destino.

Movimiento Simple

• Nf3 (en inglés) o Cf3 (en español): El Caballo se mueve a la casilla f3. • Be4 o Ae4: El Alfil se mueve a e4. • e4: Un peón se mueve a la casilla e4 (recuerda que no lleva letra).

Capturas (Símbolo x)

Cuando una pieza captura a otra, se coloca una x entre la letra de la pieza y la casilla donde se realiza la captura. • Cxf7: El Caballo captura la pieza que estaba en f7. • Axd5: El Alfil captura la pieza que estaba en d5. • Captura de peón: Se escribe la columna de origen del peón (en minúscula), la x y la casilla de destino. Ejemplo: exd5 (El peón de la columna e captura en d5).

Enroques

• 0-0: Enroque corto (hacia el flanco de rey). • 0-0-0: Enroque largo (hacia el flanco de dama).

Jaque (+) y Jaque Mate (#)

• Dh5+: La Dama se mueve a h5 y da jaque al Rey enemigo. • Dxf7#: La Dama captura en f7 y es Jaque Mate (fin de la partida).

Coronación o Promoción (Símbolo =)

Cuando un peón llega a la octava fila (o primera fila en el caso de las negras) y se transforma en otra pieza, se añade = seguido de la letra de la nueva pieza. • e8=D: El peón avanza a e8 y corona una Dama. • b1=C: El peón negro avanza a b1 y se transforma en Caballo.

Resolución de Ambigüedades

A veces, dos piezas iguales (por ejemplo, dos caballos) pueden moverse a la misma casilla. Para evitar confusiones, se especifica la columna o la fila de origen de la pieza que se va a mover: • Por columna: Si los caballos están en g1 y d2, y uno va a f3, se escribe: Cgf3 (El caballo de la columna g va a f3). • Por fila: Si los caballos están en g1 y g5, y uno va a f3, se escribe: C1f3 (El caballo de la fila 1 va a f3).

"El Mate del Pastor"

Así se escribiría una partida real completa utilizando esta notación:

1. e4 e5 (Ambos bandos mueven sus peones centrales)
2. Ac4 Cc6 (Las blancas desarrollan el alfil a c4; las negras desarrollan su caballo a c6)
3. Dh5 Cf6 (La dama blanca va a h5 amenazando mate; las negras desarrollan su otro caballo sin notar el peligro)
4. Dxf7# (La dama blanca captura el peón de f7 y es Jaque Mate)

Apéndice B - El formato Portable Game Location (PGN)

El formato PGN (**Portable Game Notation**) es el estándar universal absoluto para almacenar, registrar y compartir partidas de ajedrez en formato digital. Fue diseñado para ser fácilmente legible tanto por seres humanos como por computadoras, lo que permitió que millones de partidas históricas y modernas se guarden en archivos de texto plano sumamente ligeros. A continuación, te presento una descripción detallada de su origen, estructura técnica y un ejemplo real de su funcionamiento.

Origen del formato PGN

El formato PGN fue desarrollado y especificado en 1993 por Steven J. Edwards, un informático y entusiasta del ajedrez. Su objetivo principal era solucionar la enorme fragmentación que existía en el naciente software de ajedrez de principios de los años 90, donde cada programa utilizaba su propio formato propietario cerrado. Edwards diseñó PGN de uso libre y gratuito, basándose en la Notación Algebraica estándar, lo que garantizó una rápida adopción comunitaria. Hoy en día, cualquier plataforma de ajedrez (Chess.com, Lichess, ChessBase) o motor de inteligencia artificial (Stockfish, Lc0) exporta e importa archivos .pgn por defecto.

Estructura Técnica

Un archivo .pgn es simplemente un archivo de texto con la extensión modificada. Se divide estrictamente en dos secciones consecutivas por cada partida: la Sección de Etiquetas (Tags) y la Sección de Movimientos (Movetext).

Sección de Etiquetas (STR - Seven Tag Roster)

Al principio de la partida aparecen una serie de metadatos encerrados entre corchetes [ ]. Para que un archivo PGN sea válido a nivel internacional, debe incluir obligatoriamente siete etiquetas específicas (conocidas en inglés como el Seven Tag Roster), escritas en este orden exacto:

1. [Event "Name"]: El nombre del torneo o evento.
2. [Site "City, Region COUNTRY"]: El lugar geográfico o plataforma en línea donde se jugó.
3. [Date "YYYY.MM.DD"]: La fecha de inicio de la partida. Si se desconoce un dato, se usan signos de interrogación (ej. 2026.10.??).
4. [Round "Number"]: El número de ronda del torneo.
5. [White "LastName, FirstName"]: Nombre completo del jugador de las piezas blancas.
6. [Black "LastName, FirstName"]: Nombre completo del jugador de las piezas negras.
7. [Result "Result"]: El resultado final. Solo se admiten cuatro opciones: 1-0 (ganan blancas), 0-1 (ganan negras), 1/2-1/2 (tablas) o * (partida en desarrollo o resultado desconocido).

Sección de Movimientos (Movetext)

Justo debajo de las etiquetas se escriben las jugadas de forma secuencial. Cada turno consta del número de la jugada, un punto, el movimiento de las blancas y el movimiento de las negras separados por espacios. El texto finaliza obligatoriamente repitiendo el resultado exacto que se indicó en la etiqueta Result.

Archivo PGN del "Mate del Pastor"

Si abrieras un archivo llamado pastor.pgn en un bloc de notas de Windows, su contenido se vería exactamente así: pgn [Event "Partida Amistosa de Práctica"] [Site "Mar del Plata, ARG"] [Date "2026.10.30"] [Round "1"] [White "Perez, Juan"] [Black "Gomez, Carlos"] [Result "1-0"]

1. e4 e5 2. Ac4 Cc6 3. Dh5 Cf6 4. Dxf7# 1-0

Interpretación

1. Lectura de Metadatos: Cualquier software lee primero los corchetes para catalogar la partida en su base de datos. Sabrá instantáneamente que Juan Pérez le ganó a Carlos Gómez en Argentina el 30 de octubre de 2026.
2. Procesamiento del Tablero: El programa inicializa un tablero virtual con las 32 piezas en sus casillas de origen.
3. Ejecución paso a paso: El lector busca el número 1. y arrastra el peón blanco a e4, luego busca la respuesta negra a e5. Avanza secuencialmente interpretando las letras de las piezas y las casillas de destino.
4. Detección de Fin: Al encontrar el símbolo #, el programa detiene la partida, valida el Jaque Mate y confirma el cierre mediante el marcador numérico 1-0. Si te interesa, puedo mostrarte cómo se añaden variantes alternativas (jugadas que pudieron pasar) o comentarios de texto dentro de este mismo código PGN para que los programas los muestren de forma interactiva. ¿Te gustaría verlo?

Apéndice C - El formato Brains.json

Apéndice D - El formato Rally.json